N7 RACING TEAM

TUTORIEL ÉTUDIANT

N7RT CAN LAB 0.3.0

Projet : **Formula Student 2027**

Pôle : **DATA — CAN Lab**

Prendre en main N7RT CAN Lab 0.3 — commun aux quatre missions

Public visé : Étudiants des quatre missions (APPS, WSF, SAS, BPS)

Équipe N7 Racing Team · ENSEEIHT, Toulouse
Version N7RT CAN Lab 0.3.0 · 16/10/2026
Statut Document étudiant

Captures : application étudiante 0.3 (exécutable certifié caef4c51…), carte NUCLEO-H753ZI, campagne documentaire du 16/10/2026. Les exemples de valeurs sont fictifs.

Sommaire

1 À lire avant de commencer ... 3
1.1 Ce que ce document ne contient pas ... 3
1.2 Lire une figure ... 3
1.3 Vocabulaire ... 3
2 Installer et lancer ... 4
2.1 Récupérer et vérifier l'archive ... 4
2.2 Installer les outils de compilation ... 4
2.3 Brancher la carte ... 5
2.4 Premier lancement ... 5
2.5 Connecter la carte ... 5
2.6 Reconnexion ... 7
3 Ouvrir sa mission ... 8
3.1 Fermer et rouvrir la mission ... 11
4 L'interface en un coup d'œil ... 12
5 L'énoncé de la mission ... 15
6 La progression E0 → E5 ... 18
6.1 Les étapes et leurs états ... 18
6.2 La consigne et le bouton de vérification ... 18
6.3 Pendant une vérification ... 19
6.4 Le délai entre deux vérifications ... 19
6.5 Des tentatives liées à la session, à la carte et au code ... 20
6.6 Les étapes d'incident ... 20
7 La fiche ... 22
7.1 Ouvrir et parcourir la fiche ... 22
7.2 Enregistrer la fiche ... 23
7.3 Les informations de la partie nominale ... 25
7.4 La fiche d'incident (E1 à E3) ... 30
8 Le bus CAN ... 33
8.1 Démarrer et arrêter une session ... 33
8.2 Lire le tableau des trames ... 34
8.3 Résultats étudiants ... 35
8.4 La console et les onglets du bas ... 36
9 Le Physical Lab ... 37
9.1 Organisation de la vue ... 37
9.2 Barre d'outils et navigation ... 38
9.3 Sélectionner un composant et modifier un paramètre ... 39
9.4 Mesurer au multimètre ... 39
9.5 Remplacer un fil ... 40
9.6 Revenir au banc de la mission ... 43
9.7 Consigner la mesure ... 43
9.8 Reproduire l'anomalie ... 44
10 Le code STM32 ... 47
10.1 La page de code ... 47
10.2 Les boutons ... 48
10.3 Compiler et corriger une erreur ... 49
10.4 Flasher la carte et contrôler le CRC ... 50
10.5 Le squelette fourni ... 51
10.6 Les règles de compilation de l'IDE ... 53
10.7 Décoder une trame : méthodes générales ... 53
11 Vérifier une étape ... 57
11.1 Quand le bouton est actif ... 57
11.2 Pendant l'essai ... 58
11.3 Lire le verdict ... 59
12 La qualification E4 ... 62
13 Clôture E5 et export ... 66
14 Que faire quand ça échoue ? ... 68
15 FAQ de démarrage ... 69

1. À lire avant de commencer

Ce tutoriel vous rend autonome sur N7RT CAN Lab 0.3. Il ne résout pas votre mission : il vous apprend à installer l'application, à brancher et connecter la carte, à ouvrir la mission de votre groupe, à lire la progression et à tenir la fiche, puis, dans sa seconde partie, à observer le bus, à mesurer au banc, à programmer la carte et à faire vérifier votre travail. Il est commun aux quatre missions (APPS, WSF, SAS et BPS) : l'interface et les procédures sont les mêmes dans la vôtre ; seuls le contenu de la mission (masqué ici) et les valeurs (fictives ici) diffèrent.

L'application juge seule chaque étape : votre fiche contre la vérité scellée dans votre pack, votre circuit contre le banc de référence, votre carte contre des essais cachés. Le texte libre (justifications, pages citées, noms des signaux) n'entre jamais dans un verdict ; il sert à votre encadrant. Un verdict dit si l'étape est validée, sans détailler ce qui est faux : à vous d'observer, de mesurer, de corriger, puis de vérifier de nouveau.

1.1 Ce que ce document ne contient pas

- Aucune réponse : aucun identifiant de trame, aucune position de bit, aucun facteur ni offset, aucune valeur de supervision, aucune cible, catégorie ou valeur de mesure d'une mission réelle. Les établir fait partie de votre travail, à partir du bus, du banc et de la bibliothèque.
- Aucune méthode propre à une mission : les explications portent sur l'outil, sur les formats de saisie et sur la démarche générale.
- Sur les figures prises pendant une mission, tout ce qui appartient à la mission est recouvert d'un rectangle gris uni : titre et groupe, énoncé, bibliothèque, trafic du bus, résultats, valeurs choisies dans la fiche.
- Les valeurs lisibles dans les fiches des figures sont **fictives**, comme la trame `0x3C7` ou le signal *Température liquide (exemple)*. Elles ne correspondent à aucune mission : ne les recopiez pas.

1.2 Lire une figure

Chaque figure est une capture réelle de l'application 0.3. Un tracé rouge entoure chaque élément à regarder ou à utiliser : un cercle ou un cadre arrondi pour un bouton ou un champ, un rectangle pour une zone, un trait rouge épais posé sur le fil pour un fil du banc ; une flèche rouge désigne un petit élément. Une pastille rouge numérotée accompagne chaque tracé, et le texte cite les mêmes numéros (1, 2…) dans l'ordre où vous agissez.

Sous chaque figure, un tableau reprend :

- **Étapes** : ce que vous faites, dans l'ordre des pastilles ;
- **Ce qu'on doit observer** : ce que l'écran montre quand l'action a réussi ;
- **Si cela échoue** : les causes fréquentes et ce qu'il faut faire.

Les libellés de l'interface sont cités entre guillemets, tels qu'ils sont écrits dans l'application. Certains titres s'affichent en capitales : « Carte » apparaît CARTE dans la barre d'outils, « Sorties à publier » apparaît SORTIES À PUBLIER dans le panneau de la mission.

1.3 Vocabulaire

Terme	Sens dans ce tutoriel
Cockpit	La vue principale : mission, carte, session, trafic du bus, résultats et fiche.
Physical Lab	La vue du banc : le circuit simulé du véhicule, son inspecteur et son multimètre.
Code STM32	L'éditeur de `student_decoder.c` et `student_decoder.h`, la compilation et le flashage.
Carte	La NUCLEO-H753ZI de votre poste, qui exécute votre décodeur.
Session	Un échange en cours entre l'application et la carte : les trames du bus simulé partent vers la carte, ses résultats reviennent. « Start » la démarre, « Stop » l'arrête.
Pack, code du groupe	Le fichier `.n7rtpack` remis par l'encadrant (la mission de votre groupe, scellée) et le code remis à part qui l'ouvre.
Fiche	Le document que vous remplissez dans l'application : votre décodage (partie nominale), puis un diagnostic par incident.
Slot, tick	Un slot est une sortie numérotée de votre décodeur. Le tick est la période des appels de `student_on_tick` pendant une session (10 ms par défaut) : à chaque tick, la carte renvoie pour chaque slot la valeur publiée ou le rejet.

2. Installer et lancer

2.1 Récupérer et vérifier l'archive

L'application est livrée en deux fichiers, l'archive et son empreinte SHA-256 :

- `n7rt-canlab-0.3.0-windows-x64.zip` ;
- `n7rt-canlab-0.3.0-windows-x64.zip.sha256`.

Elle fonctionne sous Windows 10 ou 11, 64 bits.

1. Placez les deux fichiers dans le même dossier.
2. Ouvrez PowerShell dans ce dossier et tapez `Get-FileHash .\n7rt-canlab-0.3.0-windows-x64.zip`.
3. Comparez la valeur de la colonne « Hash » avec celle du fichier `.sha256` : elles doivent être identiques, aux majuscules près. Sinon, le téléchargement est abîmé : téléchargez de nouveau l'archive.
4. Décompressez toute l'archive (clic droit, « Extraire tout… ») dans un dossier où vous pouvez écrire, par exemple dans vos Documents, mais pas dans `Program Files`.

Élément de l'archive	Rôle
`n7rt-can-lab.exe`	L'application, édition étudiante.
`WebView2Loader.dll`	Le composant d'affichage de la fenêtre ; il reste à côté de l'exécutable.
`N7RT_CANLab_Student\`	Le SDK public et le projet de secours : le dossier `n7rt\` du SDK, fourni et vérifié à chaque compilation (n'y modifiez rien) ; `student\student_decoder.c` et `student\student_decoder.h`, le gabarit public de départ ; `LISEZMOI_ETUDIANT.txt` et le projet STM32CubeIDE de secours, réservé à l'encadrant (ne programmez pas la carte depuis STM32CubeIDE). Les fichiers que vous modifiez sont les copies de la vue « Code STM32 », enregistrées dans le profil local de l'application (seconde partie du tutoriel), et non ceux de ce dossier.
`tools\stm32\`	Les scripts Python qu'utilise la vue « Code STM32 » pour piloter le compilateur et STM32CubeProgrammer, à installer à part (section suivante).
`library\`	La liste des documents techniques de la bibliothèque : manifestes et liens vers les éditeurs. L'archive ne contient aucun PDF de documentation technique.
`shared\missions\`	Les énoncés publics des quatre missions (`enonce.json`), déjà intégrés à l'application ; n'y modifiez rien.
`docs\`	La documentation de la version : démarrage en 5 minutes, tutoriel de la progression 0.3 (Markdown), tutoriel de la vue « Code STM32 », problèmes connus, documents de conception, rapports de validation et captures.
`requirements-student-ide.txt`	La dépendance Python des outils (`pyserial`).
`LISEZMOI.txt`	Le résumé de l'installation.
`manifest.json`, `runtime-release.json`	L'empreinte SHA-256 de chaque fichier de l'archive ; la version et l'empreinte du runtime validé de la carte.
`CODEMIRROR-LICENSES.txt`	Les licences de l'éditeur de code intégré.

Attention Gardez tout le dossier décompressé intact : l'application cherche à côté d'elle `WebView2Loader.dll`, `library\`, `tools\` et `N7RT_CANLab_Student\`, et les outils de compilation lisent `runtime-release.json`. Ne déplacez jamais l'exécutable seul.

2.2 Installer les outils de compilation

La fiche, le Cockpit et le Physical Lab fonctionnent sans autre installation. Pour compiler et programmer la carte depuis la vue « Code STM32 », installez séparément, car aucun Python ni compilateur n'est embarqué :

1. Python 3.11 ou plus récent, accessible par la commande `python` ;
2. la chaîne GNU ARM, qui fournit `arm-none-eabi-gcc` ;
3. STM32CubeProgrammer ;
4. puis, dans le dossier décompressé, la dépendance des outils :
   `python -m pip install -r .\requirements-student-ide.txt`.

Astuce Une installation de STM32CubeIDE ou de STM32CubeCLT dans `C:\ST` fournit à la fois le compilateur et STM32CubeProgrammer : les outils de l'archive les y trouvent sans autre réglage. Ils les cherchent aussi dans le `PATH`.

Si Python ou sa dépendance manque, la vue « Code STM32 » répond *Outil de compilation indisponible : installez les prérequis du projet étudiant.* Si le compilateur manque, « Compiler » s'arrête sur un message en anglais du type `arm-none-eabi-gcc not found` ; si STM32CubeProgrammer manque, c'est « Flasher » qui s'arrête sur `STM32_Programmer_CLI not found`. Le document cité par ces messages n'est pas fourni : installez l'outil nommé ci-dessus, puis relancez l'opération.

2.3 Brancher la carte

1. Prenez la carte NUCLEO-H753ZI de votre poste : son identifiant (« UID MCU ») est enregistré avec chaque vérification, une autre carte n'est donc pas équivalente.
2. Branchez-la au PC par le connecteur USB de son ST-LINK, CN1, au milieu du petit côté de la carte, et non par l'autre connecteur USB.
3. Attendez que Windows ait reconnu le périphérique : un port série COM apparaît.

2.4 Premier lancement

1. Double-cliquez sur `n7rt-can-lab.exe`. L'exécutable n'est pas signé : si Windows affiche « Windows a protégé votre ordinateur », cliquez sur « Informations complémentaires », puis « Exécuter quand même ».
2. La fenêtre N7RT CAN Lab s'ouvre en mode étudiant (« Mode étudiant » dans la section « Mission » du Cockpit). Au premier lancement, elle s'ouvre sur la vue « Code STM32 » ; ensuite, elle rouvre la dernière vue utilisée.
3. Passez au Cockpit par le deuxième bouton de la barre d'activité, tout à gauche de la fenêtre (Figure 9).

Si la fenêtre reste blanche, le composant d'affichage de Windows manque : installez « Microsoft Edge WebView2 Runtime », puis relancez. Si elle reste noire ou figée, la mémoire du poste est peut-être saturée : fermez les applications inutiles et relancez ; la progression enregistrée est reprise.

À savoir Si une seule carte ST-LINK est branchée, l'application s'y connecte d'elle-même au démarrage. Avec plusieurs cartes, elle attend votre choix : *Plusieurs cartes détectées : choisissez-en une puis « Connecter ».*

2.5 Connecter la carte

La connexion se règle dans la barre d'outils, en haut de la fenêtre ; cette barre est la même dans les trois vues.

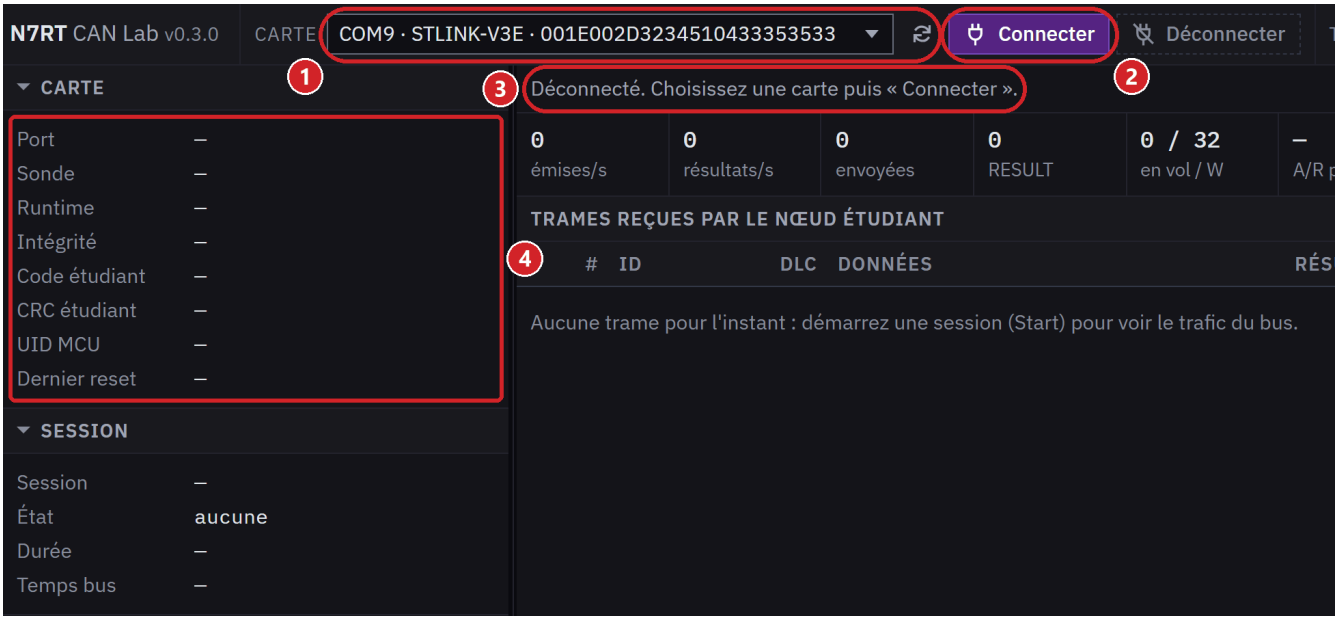


Figure 1. – Carte déconnectée : choisir la carte, puis « Connecter ».

Étapes	1 Dans la liste « Carte », choisissez votre carte : chaque ligne donne le port COM, la sonde ST-LINK et son numéro de série (`COM9 · STLINK-V3E · …`). Le bouton aux deux flèches circulaires, dans le même cadre à droite de la liste, est « Rechercher les cartes » : il relit la liste des ports si vous venez de brancher la carte. 2 Cliquez sur « Connecter ». 3 Lisez le message d'état, sous la barre d'outils : il dit à chaque instant ce que fait la liaison. 4 Tant que la carte n'est pas connectée, la section « Carte » du panneau latéral n'affiche que des tirets.
Ce qu'on doit observer	Avant la connexion, 3 affiche *Déconnecté. Choisissez une carte puis « Connecter ».* Après 2, l'état passe par « Poignée de main », puis « Prête » (Figure 2).
Si cela échoue	La liste 1 affiche « Aucun port série détecté » : vérifiez le câble sur CN1, puis cliquez sur « Rechercher les cartes » (les deux flèches de 1). Le message 3 dit que le port est déjà ouvert par une autre application (console série de STM32CubeIDE, seconde fenêtre du CAN Lab, moniteur série) : fermez-la, la connexion reprend seule.

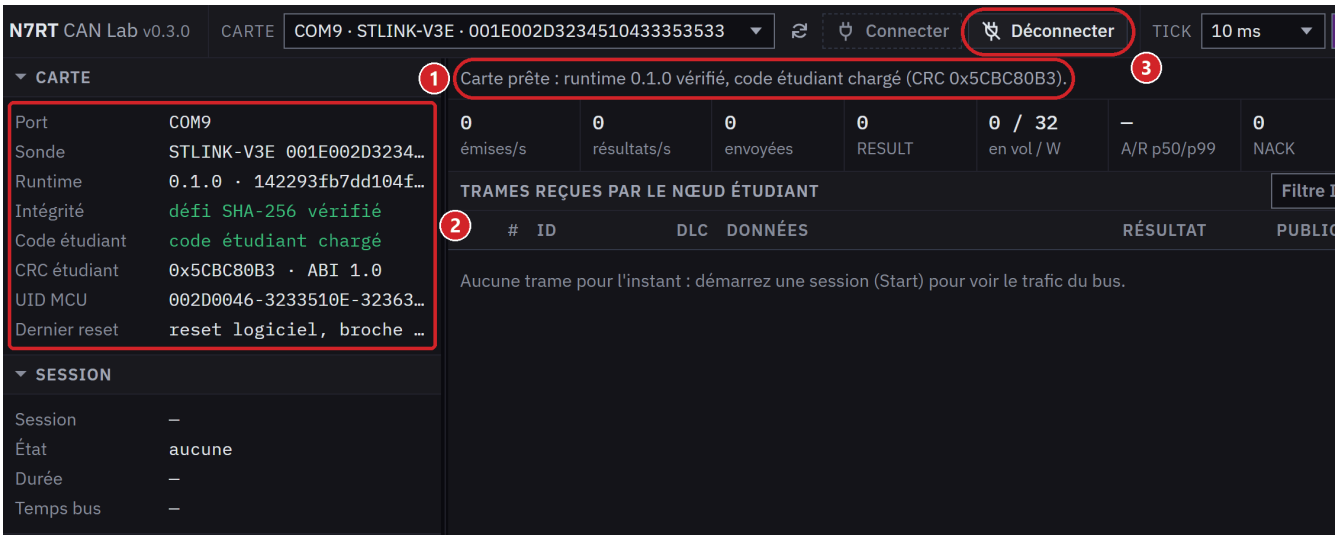


Figure 2. – Carte prête : le message d'état, la section « Carte » et « Déconnecter ».

Étapes	1 Le message d'état confirme la connexion : « Carte prête : runtime … vérifié, code étudiant chargé (CRC …). » 2 La section « Carte » détaille la carte connectée (tableau ci-dessous). 3 « Déconnecter » ferme la liaison. Pour passer à une autre carte, choisissez-la dans la liste « Carte », puis cliquez sur « Connecter ».

Ce qu'on doit observer	Le voyant d'état de la barre d'outils passe de « Déconnecté » à « Prête » (Figure 10). La barre d'état, en bas à gauche de la fenêtre, répète l'état, le port, le runtime vérifié et le CRC du code chargé.
Si cela échoue	L'état reste « Pas de code étudiant » : la carte n'a pas de programme étudiant valide ; programmez-la depuis la vue « Code STM32 » (« Compiler + flasher »). « Carte incompatible » ou « NON vérifié » à la ligne « Intégrité » de 2 : prévenez l'encadrant.

Ligne de 2	Ce qu'elle indique
« Port »	Le port série de la carte (COM…).
« Sonde »	Le modèle et le numéro de série du ST-LINK.
« Runtime »	La version du runtime N7RT installé sur la carte, son identifiant de construction et son ABI.
« Intégrité »	« défi SHA-256 vérifié », en vert, quand l'application a vérifié l'empreinte du runtime ; « NON vérifié » sinon.
« Code étudiant »	L'état du programme étudiant : « code étudiant chargé » quand un décodeur valide est présent.
« CRC étudiant »	L'empreinte CRC-32 du code étudiant présent sur la carte, suivie de son ABI. Elle change dès que le code programmé change (le même code redonne le même CRC) : comparez-la avec le « CRC compilé » de la vue « Code STM32 » pour savoir si la carte exécute bien votre dernière compilation.
« UID MCU »	L'identifiant unique du microcontrôleur, enregistré avec chaque vérification.
« Dernier reset »	La cause du dernier redémarrage de la carte.

2.5.1 États de la carte

Le voyant d'état de la barre d'outils (Figure 10) prend l'une des valeurs suivantes.

État affiché	Ce qui se passe	Que faire
« Déconnecté »	Aucune liaison n'est ouverte.	Choisissez la carte, puis « Connecter ».
« Carte absente »	La carte choisie n'est pas détectée : câble débranché, autre connecteur, port changé.	Vérifiez le câble sur CN1 ; l'application réessaie seule.
« Port occupé »	Une autre application a ouvert le port série de la carte.	Fermez cette application ; la connexion reprend seule.
« Poignée de main »	L'application et la carte s'identifient.	Attendez quelques secondes.
« Prête »	Runtime vérifié, code étudiant chargé.	Vous pouvez démarrer une session (« Start »).
« Pas de code étudiant »	Le runtime est vérifié, mais la carte n'a pas de programme étudiant valide : « Start » reste grisé.	Programmez la carte (vue « Code STM32 », « Compiler + flasher »), au besoin avec le modèle fourni.
« Code étudiant désactivé »	Votre code a provoqué une faute ; la carte l'a désactivé.	Corrigez et reprogrammez ; démarrer une session le réactive aussi.
« Carte incompatible »	Le runtime de la carte ne correspond pas à cette version de l'application.	Prévenez l'encadrant : vous ne pouvez pas le réinstaller vous-même.
« Session active »	Une session tourne : trames vers la carte, résultats en retour.	« Stop » l'arrête.
« Liaison perdue »	La carte a été débranchée ou son port a disparu.	Rebranchez la carte : l'application se reconnecte dès son retour.

2.6 Reconnexion

- Si le câble est débranché, l'état passe à « Liaison perdue » et le message annonce une reconnexion automatique dès le retour de la carte. Rebranchez-la : la liaison et, le cas échéant, la session en cours reprennent d'elles-mêmes. Pendant une vérification, en revanche, débrancher la carte interrompt l'essai, qui n'est pas compté (chapitre 6).
- Après chaque programmation, la carte redémarre et l'application refait seule la poignée de main : attendez « Prête », puis vérifiez que le « CRC étudiant » est égal au « CRC compilé » de votre dernière compilation.
- « Déconnecter » coupe la liaison volontairement : rien ne se reconnecte ensuite tant que vous n'avez pas cliqué sur « Connecter ».

3. Ouvrir sa mission

Tout se passe dans le Cockpit, panneau latéral, section « Mission », sous les sections « Carte » et « Session ». Une mission s'ouvre en trois temps : choisir la mission de la séance (une fois pour toutes), importer le pack du groupe (une fois), puis saisir le code du groupe (à chaque ouverture).

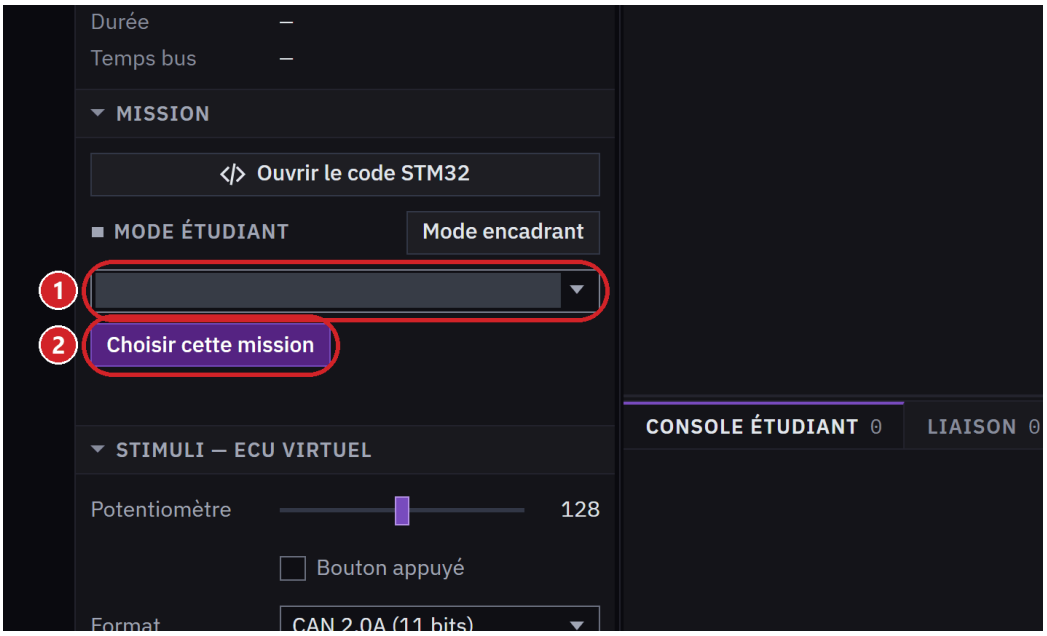


Figure 3. – Choisir la mission de la séance (le nom de la mission choisie est masqué).

Étapes	1 Dans la liste, sélectionnez la mission de votre groupe : « APPS — pédale d'accélérateur », « WSF — vitesse des roues », « SAS — angle du volant » ou « BPS — pression de freinage ». 2 Cliquez sur « Choisir cette mission ».
Ce qu'on doit observer	Une demande de confirmation s'affiche sous 2 (Figure 4). Rien n'est encore verrouillé.
Si cela échoue	Une mission grisée dans 1 porte sa raison en info-bulle (« Indisponible : … ») : prévenez l'encadrant.

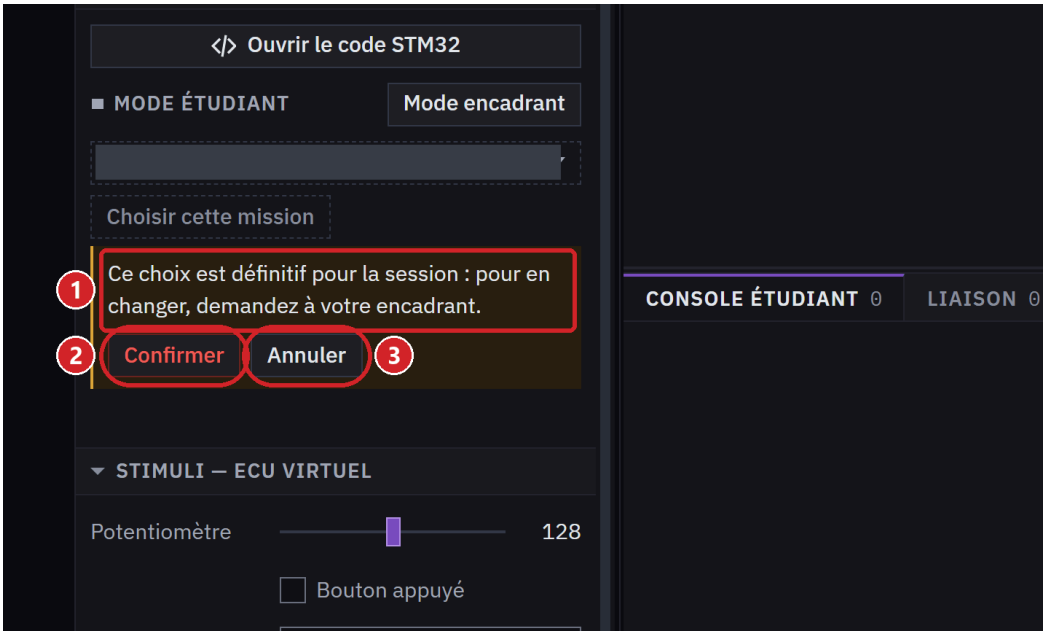


Figure 4. – Confirmer le choix de la mission : il est définitif.

Étapes	1 Lisez l'avertissement : « Ce choix est définitif pour la session : pour en changer, demandez à votre encadrant. » 2 Si la mission sélectionnée est bien celle de votre groupe, cliquez sur « Confirmer ». 3 Sinon, « Annuler » referme la confirmation : sélectionnez la bonne mission.
Ce qu'on doit observer	Après 2, la liste disparaît : la ligne « Mission » affiche la mission choisie et la ligne « Pack » indique « Aucun pack importé. » (Figure 5). Le choix reste verrouillé, même après un redémarrage de l'application.
Si cela échoue	Mauvaise mission confirmée : seul l'encadrant peut effacer ce choix sur le poste. Toute tentative d'ouvrir une autre mission répond « La mission de cette session est déjà choisie : … ».

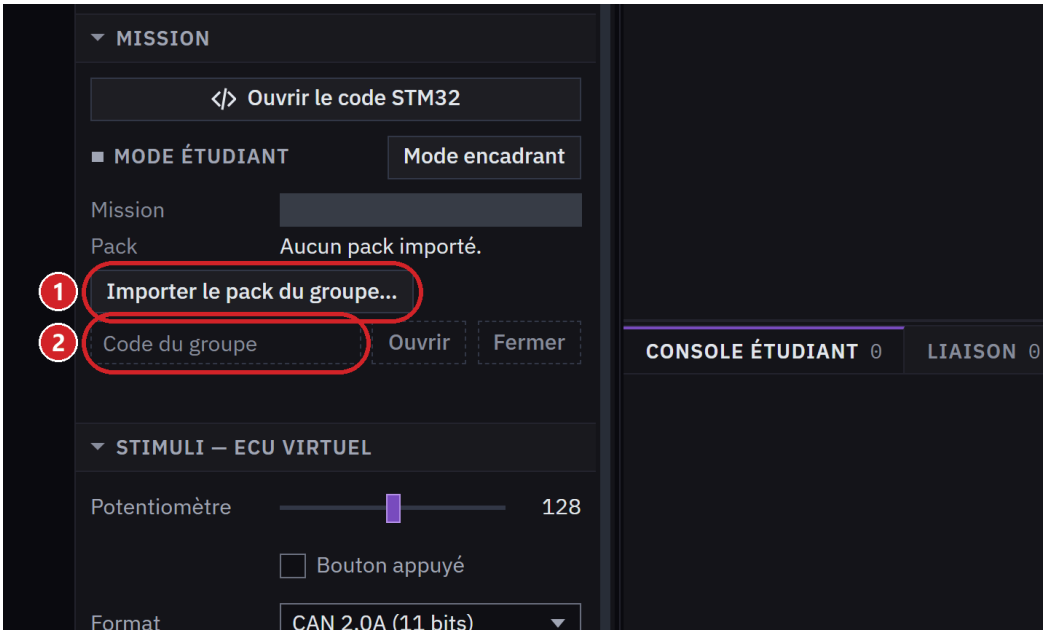


Figure 5. – Importer le pack du groupe (le nom de la mission est masqué).

Étapes	1 Cliquez sur « Importer le pack du groupe… », puis, dans la fenêtre de Windows, sélectionnez le fichier `.n7rtpack` de votre groupe, remis par l'encadrant. 2 Le champ « Code du groupe » reste grisé tant qu'aucun pack n'est importé.
Ce qu'on doit observer	La ligne « Pack », juste au-dessus de 1, passe de « Aucun pack importé. » au nom de votre groupe suivi de sa cohorte ; le champ 2 devient actif. L'import se fait une seule fois sur ce poste.

Si cela échoue	« Ce fichier n'est pas un pack de groupe N7RT. » : vous avez sélectionné un autre fichier. « Ce pack est destiné à une autre mission. » : vérifiez avec l'encadrant le fichier et la mission choisie. « Pack du groupe illisible : demandez un nouveau fichier à votre encadrant. » et « Fichier trop volumineux pour un pack de groupe. » : demandez un nouveau pack.

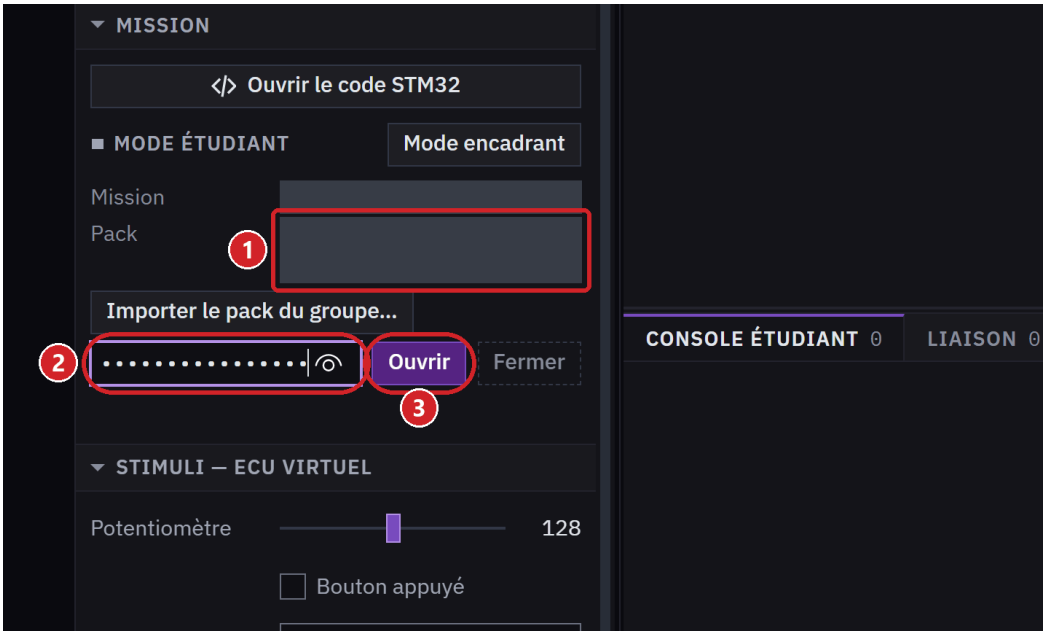


Figure 6. – Saisir le code du groupe, puis « Ouvrir » (mission et groupe masqués).

Étapes	1 Vérifiez la ligne « Pack » : le nom de votre groupe et sa cohorte (masqués sur la figure). 2 Tapez le code du groupe, reçu à part et jamais écrit dans le pack, dans « Code du groupe ». Les caractères sont masqués ; l'icône en forme d'œil les affiche. 3 Cliquez sur « Ouvrir », ou appuyez sur Entrée.
Ce qu'on doit observer	Le champ 2 se vide et la mission s'ouvre (Figure 7). Le bouton « Fermer », à droite de 3, refermera la mission.
Si cela échoue	« Code de groupe incorrect. » : retapez le code tel qu'il vous a été remis. Le bouton 3 grisé donne sa raison en info-bulle, par exemple « Saisissez le code du groupe. » ou « Importez d'abord le pack du groupe. » Ne communiquez ni le pack ni le code à un autre groupe.

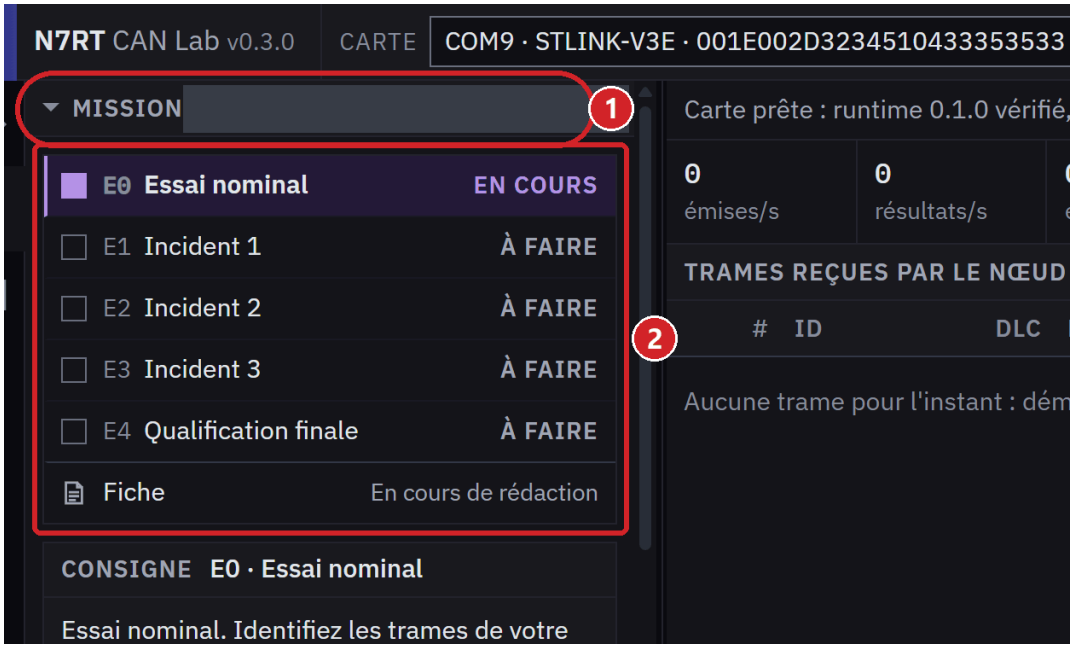


Figure 7. – Mission ouverte : le panneau de la mission en tête du panneau latéral (mission et groupe masqués).

Étapes	1 Le titre de la section donne la mission et votre groupe (« Mission … — groupe … »). 2 La progression : E0 « Essai nominal » est « EN COURS », les autres étapes sont « À FAIRE » ; la dernière ligne, « Fiche », est « En cours de rédaction ».
Ce qu'on doit observer	Sous 2 viennent la consigne, « Ouvrir la fiche », « Lancer la vérification », la liste des champs à remplir (chapitre 6), puis l'énoncé (chapitre 5). Les sections « Carte » et « Session » se replient et la section « Stimuli — ECU virtuel » disparaît : pendant une mission, le bus est celui du banc. La liste « Tick » est fixée par la mission. Le Physical Lab charge le banc de la mission.
Si cela échoue	La mission ne s'ouvre pas : lisez le message affiché sous les boutons de la section « Mission ». « Progression illisible sur ce poste : demandez à l'encadrant de réinitialiser la mission. » : prévenez l'encadrant.

3.1 Fermer et rouvrir la mission

- « Fermer », dans la section « Mission », referme la mission. Si la fiche contient des modifications non enregistrées, une fenêtre propose d'abord de les enregistrer (Figure 22).
- Pour rouvrir la mission, tapez de nouveau le code du groupe, puis « Ouvrir » : le pack reste importé et le choix de la mission reste verrouillé, même après un redémarrage de l'application.
- À la réouverture, tout ce qui a été enregistré est là : progression, fiche, banc et l'anomalie de l'incident en cours.
- Fermer l'application revient au même : rouvrez-la, puis tapez le code du groupe.

À savoir Le bouton « Mode encadrant », à droite de « Mode étudiant », est réservé à l'encadrant : il demande le code encadrant.

4. L'interface en un coup d'œil

La fenêtre est organisée comme un environnement de développement : une barre d'activité à gauche pour changer de vue, une barre d'outils en haut, un panneau latéral, une zone centrale, un panneau de droite, un dock en bas et une barre d'état. La figure ci-dessous montre le Cockpit sans mission, avec une session de démonstration.

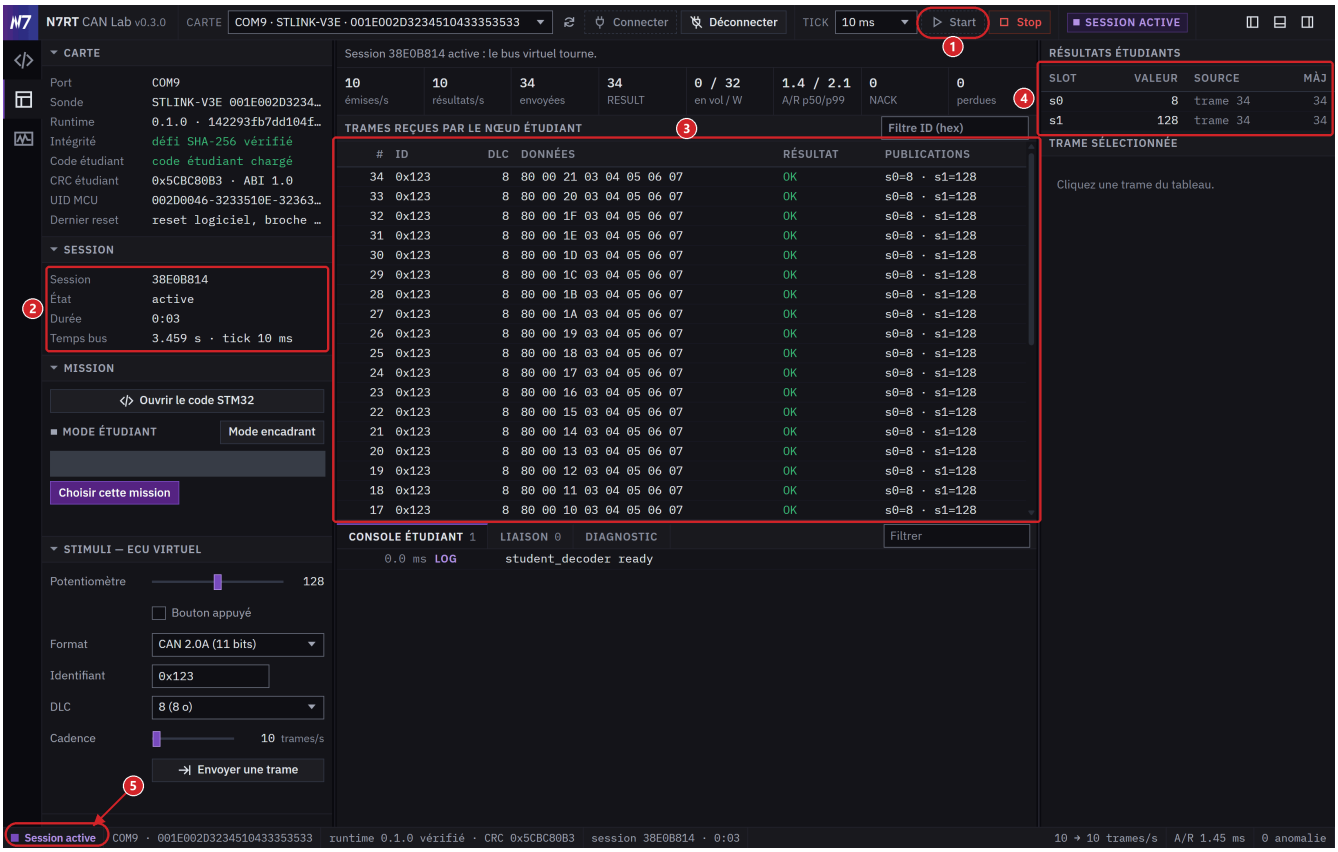


Figure 8. – Le Cockpit sans mission, session démarrée avec l'ECU virtuel (liste des missions masquée).

Étapes	❶ La barre d'outils, ici son bouton « Start » : elle porte la carte, la session (« Start », « Stop »), le voyant d'état et les boutons d'affichage des panneaux (Figure 10). ❷ Le panneau latéral, ici sa section « Session » : identifiant, état, durée, temps du bus et tick ; toutes ses sections sont décrites dans le tableau ci-dessous. ❸ La zone centrale, « Trames reçues par le nœud étudiant » : le trafic du bus tel que la carte le reçoit, avec ses compteurs au-dessus. ❹ Le panneau de droite, « Résultats étudiants » : la dernière valeur publiée par chaque slot de votre décodeur ; dessous, « Trame sélectionnée » détaille la trame cliquée dans ❸. ❺ La barre d'état, ici l'état de la carte (« Session active ») : elle donne aussi le port, le runtime et le CRC du code chargé, la session, le débit, le temps d'aller-retour et le nombre d'anomalies.
Ce qu'on doit observer	Sans mission, l'ECU virtuel du SDK envoie des trames de démonstration ; pendant une mission, c'est le banc du véhicule qui émet. Le dock du bas, sous ❸, est décrit plus loin (Figure 11). Pendant une mission, la barre d'état ❺ affiche aussi l'étape en cours ; un clic dessus ramène à la progression.
Si cela échoue	Le panneau de droite ❹ n'apparaît pas : la fenêtre fait moins de 1 280 pixels de large ; agrandissez-la, ou affichez le panneau avec le bouton « Résultats » de la barre d'outils.

Section du panneau latéral	Contenu
Panneau de la mission (en tête, mission ouverte)	La progression, la consigne, les boutons de la fiche et de la vérification, puis l'énoncé et la bibliothèque (chapitre 5 et suivants).
« Carte »	Les informations de la carte connectée (Figure 2).
« Session »	L'identifiant de la session, son état, sa durée, le temps du bus et le tick.
« Mission »	« Ouvrir le code STM32 », le mode (« Mode étudiant »), le choix de la mission, le pack, le code du groupe, « Ouvrir » et « Fermer » (chapitre 3).

Section du panneau latéral	Contenu
« Stimuli — ECU virtuel »	Sans mission seulement : les commandes de l'ECU virtuel du SDK (potentiomètre, bouton, format, identifiant, DLC, cadence, « Envoyer une trame ») pour essayer un décodeur sur des trames de démonstration. Elle disparaît pendant une mission.

Un clic sur le titre d'une section la replie ou la déplie. Les séparateurs entre les panneaux se déplacent à la souris ; l'application mémorise la disposition de chaque vue.

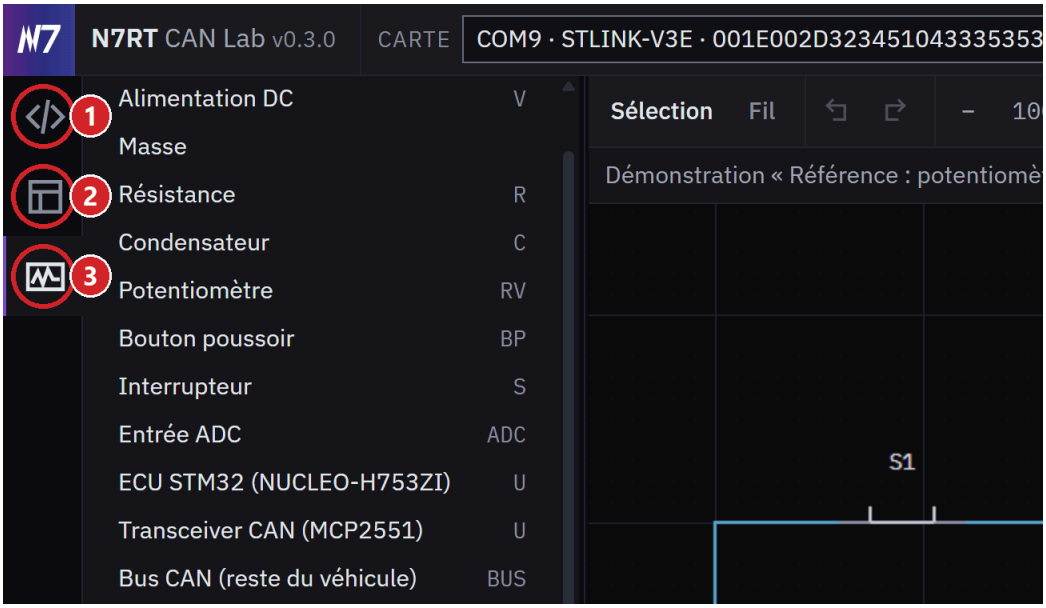


Figure 9. – La barre d'activité : les trois vues de l'application.

Étapes	1 « Code STM32 » : l'éditeur de vos deux fichiers, la compilation et la programmation de la carte. 2 « Cockpit » : la mission, la carte, la session, le trafic du bus, les résultats et la fiche. 3 « Physical Lab » : le banc du véhicule, son inspecteur et son multimètre.
Ce qu'on doit observer	Le bouton de la vue affichée est surligné : 3 sur la figure, prise dans le Physical Lab. La barre d'outils et la barre d'état restent les mêmes dans les trois vues ; le panneau latéral, la zone centrale et le panneau de droite changent. D'autres boutons mènent aux mêmes vues : « Ouvrir le code STM32 » dans la section « Mission », « Ouvrir le banc » dans le panneau de la mission.
Si cela échoue	Au premier lancement, la fenêtre s'ouvre sur « Code STM32 » : c'est normal ; cliquez sur 2 pour passer au Cockpit.

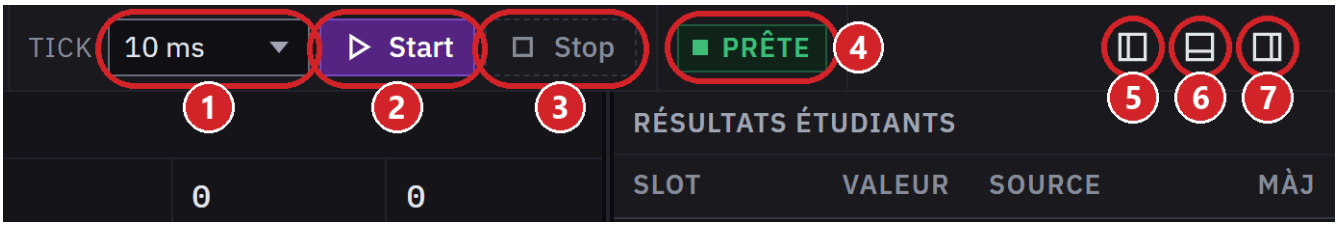


Figure 10. – La barre d'outils : session, voyant d'état de la carte et affichage des panneaux.

Étapes	1 « Tick » : la période des appels de `student_on_tick` pendant une session (10 ms par défaut) ; pendant une mission, elle est fixée par la mission et la liste est grisée. 2 « Start » démarre une session avec la carte connectée. 3 « Stop » l'arrête ; arrêtez la session avant de programmer la carte. 4 Le voyant d'état de la carte : « Prête », « Session active », « Déconnecté »… (tableau des états, section 2.5). 5, 6 et 7 affichent ou masquent le panneau latéral, le dock du bas (« Console ») et le panneau de droite (« Résultats »).
Ce qu'on doit observer	Un bouton grisé donne sa raison en info-bulle, par exemple « Connectez une carte prête pour démarrer une session. » sur « Start ». Les boutons 5 à 7 suivent la vue : dans le Physical Lab, leurs info-bulles deviennent « Bibliothèque », « Console » et « Inspecteur » ; dans « Code STM32 », « Explorateur de projet », « Console de compilation » et « Plan du code et carte ».

Si cela échoue	(2) reste grisé avec la carte connectée : l'état est « Pas de code étudiant » (programmez la carte) ou une session tourne déjà. Pendant une compilation ou un flashage, « Connecter », « Déconnecter » et « Start » sont grisés : « Compilation ou flashage en cours : la carte est occupée. »

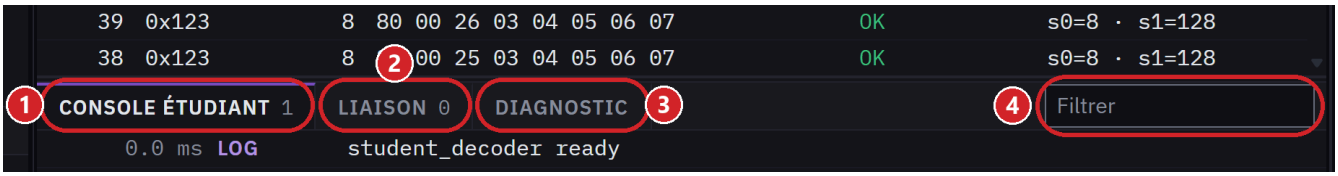


Figure 11. – Le dock du bas, sous la zone centrale.

Étapes	(1) « Console étudiant » : les lignes que votre décodeur écrit avec `n7rt_log`, et leur nombre. (2) « Liaison » : les compteurs de la liaison avec la carte (CRC, tramage, trous de séquence, débordements, journaux perdus) et les erreurs reçues. (3) « Diagnostic » : des mesures internes de la liaison et de l'affichage (files d'attente, contre-pression, temps de traitement), utiles pour décrire un problème à l'encadrant. (4) « Filtrer » ne garde que les lignes qui contiennent le texte tapé.
Ce qu'on doit observer	Pendant une mission, un onglet « Fiche » apparaît à gauche de (1) quand vous ouvrez la fiche (Figure 19). Le séparateur entre la zone centrale et le dock se déplace à la souris.
Si cela échoue	Le dock a disparu : réaffichez-le avec le bouton « Console » de la barre d'outils (Figure 10).

5. L'énoncé de la mission

L'énoncé de votre mission n'a pas de vue à lui : il se lit dans le panneau de la mission, en tête du panneau latéral du Cockpit, sous la progression (faites défiler le panneau). Il réunit le résumé, les exigences fonctionnelles, les sorties à publier, l'accès au banc et la bibliothèque. Lisez-le en entier avant de coder et de remplir la fiche.

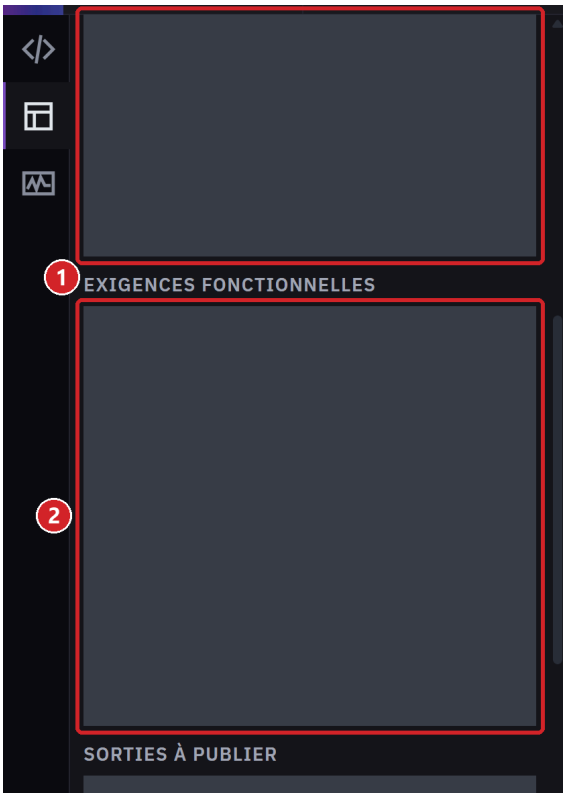


Figure 12. – Le résumé et les exigences fonctionnelles de la mission (texte masqué).

Étapes	1 Le résumé : le rôle de votre carte dans le véhicule, ce que votre programme reçoit et ce qu'il publie. 2 « Exigences fonctionnelles » : les règles que votre décodeur doit respecter (seuils, délais, conventions, conduite à tenir en cas de défaut).
Ce qu'on doit observer	Ces textes sont propres à votre mission. Avec le règlement de la bibliothèque, 2 fixe les seuils et les délais qu'il énonce et la conduite que vous reportez dans les « Décisions de conception » de la fiche. Les valeurs de supervision de la fiche n'y figurent pas : votre encadrant les communique à tous les groupes au lancement de l'E0, avec les éventuelles règles que l'énoncé ne détaille pas.
Si cela échoue	Le panneau de la mission est absent : aucune mission n'est ouverte (chapitre 3).

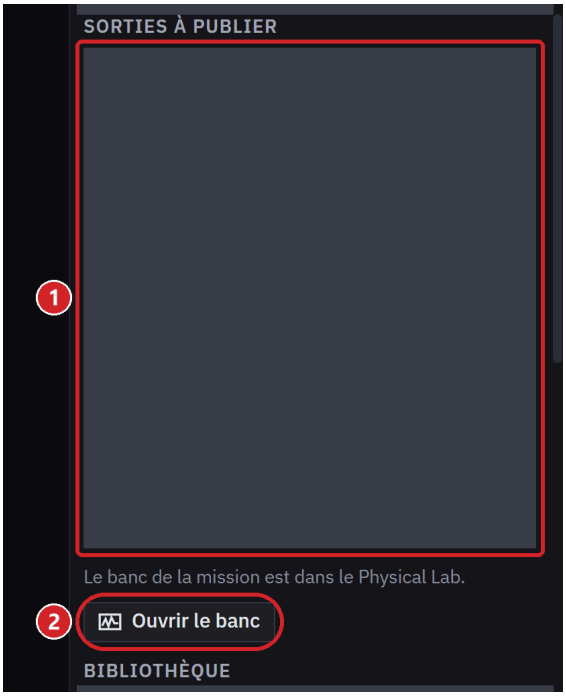


Figure 13. – « Sorties à publier » et accès au banc (liste masquée).

Étapes	1 « Sorties à publier » : une ligne par sortie de votre décodeur, de la forme `Slot n — code · libellé (unité)`. Le numéro de slot est celui où votre décodeur publie la grandeur à chaque tick et que la fiche demande dans « Slots » ; le code et le libellé nomment la grandeur ; l'unité est celle de la valeur publiée. 2 « Ouvrir le banc » affiche le banc de la mission dans le Physical Lab.
Ce qu'on doit observer	Pour le signal fictif de ce tutoriel, une ligne de 1 aurait la forme `Slot 7 — XX · Température liquide (exemple) (°C)`, où `XX` serait le code de la sortie. Dès que votre décodeur publie sur un slot de la mission, « Résultats étudiants » affiche ce slot avec son code et son libellé.
Si cela échoue	Un slot de 1 n'apparaît jamais dans « Résultats étudiants » : votre décodeur ne publie pas sur ce numéro ; vérifiez le numéro dans votre code et dans la fiche.

À savoir La partie nominale de la fiche n'a pas de champ d'unité : l'unité d'une sortie est donnée dans les « Sorties à publier » (Figure 13) et intervient dans le facteur et l'offset de votre décodage. Une unité se choisit seulement pour la mesure clé d'un incident.

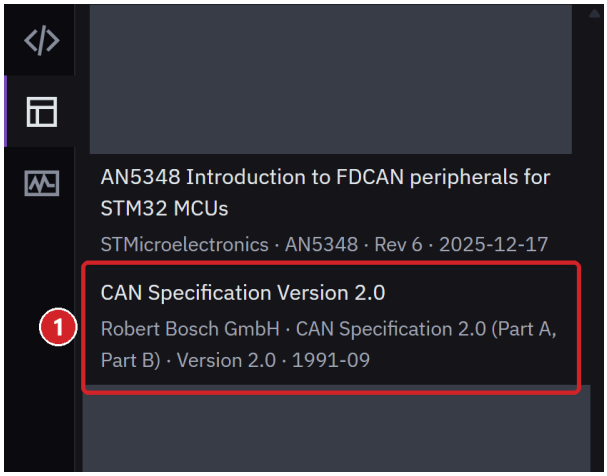


Figure 14. – La bibliothèque de la mission, un document par ligne (les documents propres à la mission sont masqués).

Étapes	1 Chaque document est un bouton : son titre, puis son éditeur, sa référence, sa révision et sa date. Un clic l'ouvre hors de l'application : sa copie locale si elle est présente et vérifiée, sinon la page de l'éditeur dans votre navigateur (connexion internet nécessaire).

Ce qu'on doit observer	La liste, triée par titre, réunit les documents de votre mission et des documents communs aux quatre missions, comme la spécification CAN de 1. Les documents ne disent jamais quoi saisir : vous y retrouvez l'information, puis vous citez les documents et les pages lus dans « Documents utilisés » de la fiche.
Si cela échoue	« Bibliothèque introuvable sur ce poste. » : prévenez l'encadrant. Un document ne s'ouvre pas sans internet : demandez sa copie locale à l'encadrant (procédure dans `library\README.md`).

Méthode : relevez le marquage écrit sur le symbole de chaque capteur dans le Physical Lab et cherchez-le (Ctrl+F) dans les fiches techniques, car certaines décrivent un autre capteur que celui du banc ; dans la bonne, cherchez la caractéristique de sortie et les diagnostics du capteur. Pour la protection des trames, servez-vous des spécifications de la bibliothèque ; pour les seuils et les délais, des « Exigences fonctionnelles » de l'énoncé, puis du règlement ; les valeurs de supervision n'y figurent pas, votre encadrant les communique au lancement de l'E0. Ne confondez pas cette bibliothèque de documents avec la « Bibliothèque » de composants du Physical Lab.

6. La progression E0 → E5

Une mission se déroule en six étapes, toujours dans le même ordre. Les étapes vérifiées (E0 à E4) ont chacune leur bouton ; E0 et chaque incident (E1 à E3) ont aussi leur partie de fiche.

6.1 Les étapes et leurs états

Étape	Nom affiché	Ce qui est demandé	Bouton
E0	« Essai nominal »	Identifier les trames de votre ECU sur le bus, les décoder, programmer la carte et documenter le décodage dans la partie nominale de la fiche.	« Lancer la vérification »
E1 à E3	« Incident 1 », « Incident 2 », « Incident 3 »	Le véhicule revient d'essai avec une anomalie : l'observer, la diagnostiquer dans la fiche d'incident, consigner la mesure clé, puis réparer le véhicule, votre code, ou les deux.	« Vérifier la réparation »
E4	« Qualification finale »	Une séquence d'essais cachée évalue votre code ; le résultat est donné par compétence. La fiche n'a pas de partie propre à E4.	« Lancer la qualification »
E5	« Clôture »	La mission est close et la fiche finale exportée.	« Ouvrir la fiche finale » (aucune vérification)

État d'une étape	Sens
« À FAIRE »	L'étape n'est pas encore ouverte : elle s'ouvrira quand la précédente sera validée.
« EN COURS »	L'étape sur laquelle vous travaillez ; aucune de ses vérifications n'a encore échoué.
« VALIDÉ »	L'étape est réussie ; sa date s'affiche dessous (`le JJ/MM/AAAA à HH:MM`). Elle ne se rouvre jamais.
« ÉCHEC À CORRIGER »	La dernière vérification de l'étape a échoué : corrigez, puis vérifiez de nouveau.

La ligne « Fiche », sous les étapes, donne l'état de votre fiche : « En cours de rédaction » pendant toute la mission, puis « Exportée le … » à la clôture (ou « Clôturée, exportation à reprendre » si l'export a échoué).

- **Une étape à la fois.** Les étapes s'enchaînent dans l'ordre E0, E1, E2, E3, E4, puis E5. Une étape ne s'ouvre qu'une fois la précédente validée ; une étape validée ne se rouvre pas ; aucune ne se saute.
- **Pas de note.** Il n'y a ni note, ni compteur, ni pourcentage : une étape est validée ou à reprendre. Le nombre de vérifications n'est pas limité.

6.2 La consigne et le bouton de vérification

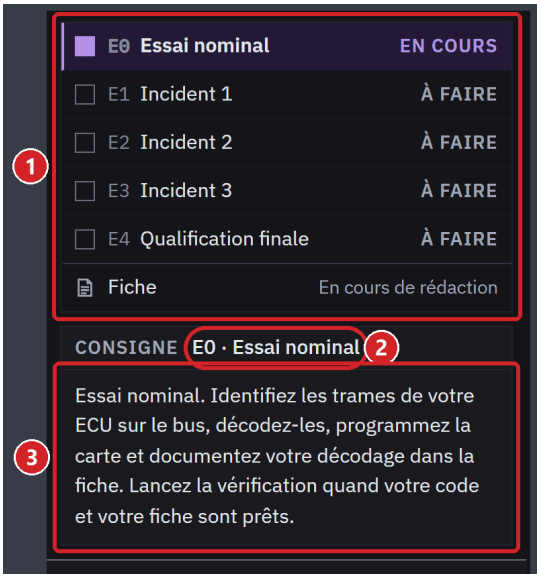


Figure 15. – Le haut du panneau de la mission : les étapes et la consigne.

Étapes	1 Les cinq étapes vérifiées, E0 à E4, avec leur état, puis la ligne « Fiche » et l'état de la fiche. 2 « Consigne », suivi de l'étape en cours (ici « E0 · Essai nominal »). 3 La consigne elle-même, c'est-à-dire l'ordre de travail de l'étape (ici celui de E0).

Ce qu'on doit observer	Au début de la mission, seule E0 est « EN COURS » dans 1. La consigne 3 change à chaque étape.
Si cela échoue	Le panneau de la mission est replié : cliquez sur son titre pour le déplier. Il est absent : la mission n'est pas ouverte (chapitre 3).

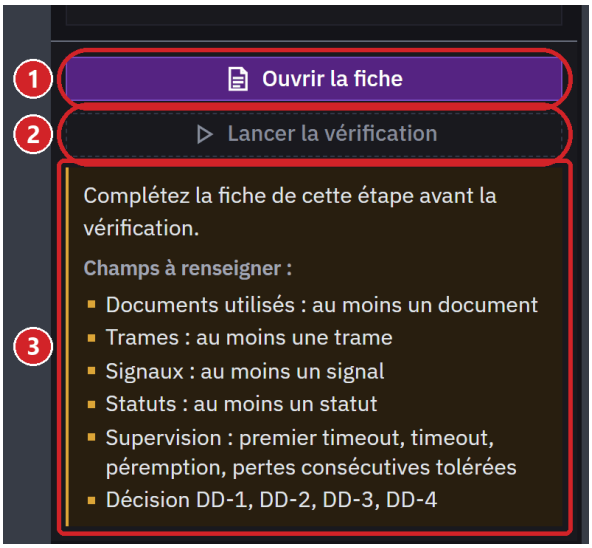


Figure 16. – Sous la consigne : la fiche, la vérification et ce qui la bloque.

Étapes	1 « Ouvrir la fiche » affiche la partie de fiche de l'étape en cours dans le dock du bas (chapitre 7) ; le bouton devient « Fermer la fiche ». 2 Le bouton de vérification de l'étape, ici « Lancer la vérification ». 3 La raison pour laquelle 2 est grisé ; ici « Complétez la fiche de cette étape avant la vérification. », suivi de « Champs à renseigner : » et d'une ligne par section ou par bloc incomplet.
Ce qu'on doit observer	La liste 3 porte sur la fiche enregistrée et se met à jour à chaque enregistrement. Les autres raisons possibles sont « Connectez la carte et vérifiez que votre programme est chargé. », « Prochaine vérification possible dans … » (Figure 17), « Corrigez les champs signalés dans la fiche. » (Figure 21) et « Vérification déjà en cours. »
Si cela échoue	2 reste grisé alors que la fiche vous semble complète : enregistrez-la (« Enregistrer la fiche »), car 3 ne tient compte que de la fiche enregistrée.

6.3 Pendant une vérification

Une vérification exige la carte connectée et prête, avec votre programme chargé, une partie de fiche complète et enregistrée, et la fin du délai depuis la vérification précédente. Le bouton de vérification enregistre d'abord la partie de fiche ouverte. Pendant l'essai :

- la zone centrale et les résultats sont masqués : « Essai en cours : la vue du bus est masquée jusqu'au verdict. » et « Essai en cours : les résultats sont masqués jusqu'au verdict. » ;
- sous le bouton, une barre de progression et le compte à rebours : « Essai en cours — … s. Ne débranchez pas la carte et ne fermez pas l'application. » ;
- la fiche est en lecture seule.

Le verdict s'affiche ensuite sous la consigne ; sa lecture est détaillée dans la seconde partie du tutoriel. Un essai interrompu (carte débranchée, application fermée) n'est pas compté.

6.4 Le délai entre deux vérifications

Deux vérifications d'une même étape sont séparées d'au moins 2 minutes pour E0 à E3, et de 5 minutes pour E4, comptées depuis la fin du dernier verdict de l'étape. Un essai interrompu ne déclenche pas ce délai, sauf au-delà de six interruptions de la même étape dans l'heure.

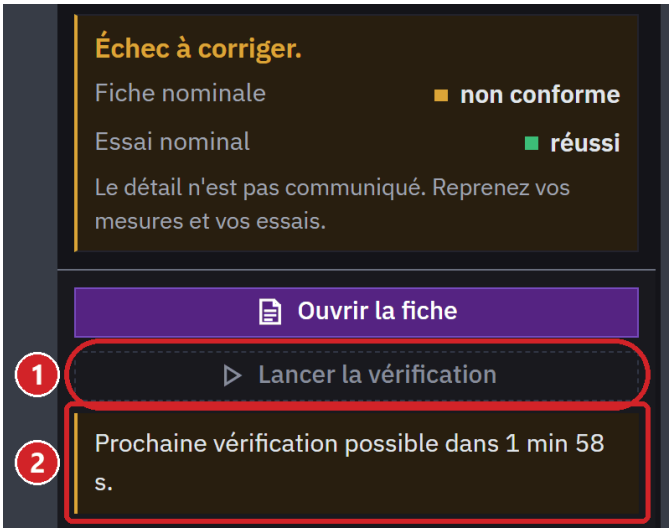


Figure 17. – Après un échec : le délai avant la vérification suivante.

Étapes	1 Le bouton de vérification reste grisé pendant le délai. 2 « Prochaine vérification possible dans … » donne le temps restant ; il se met à jour seul.
Ce qu'on doit observer	Au-dessus de 1, le verdict de la vérification précédente reste affiché : « Échec à corriger. » et ses lignes de contrôle. À la fin du délai, 1 redevient actif et 2 disparaît.
Si cela échoue	Le délai ne se raccourcit pas : profitez-en pour reprendre vos observations, vos mesures, votre code et votre fiche.

6.5 Des tentatives liées à la session, à la carte et au code

Chaque vérification est enregistrée comme une tentative, avec la version de la fiche, l'empreinte de votre circuit, le CRC du code étudiant présent sur la carte, l'identifiant de la session, l'identifiant (UID) de la carte et une empreinte du trafic du bus. Toutes les tentatives, réussies ou non, figurent dans la fiche finale. Vérifiez donc avec la carte de votre poste, programmée avec votre dernière compilation : un CRC qui ne correspond pas à votre code signale que la carte n'exécute pas ce que vous croyez.

6.6 Les étapes d'incident

Pendant les incidents, le panneau garde la même organisation : la consigne et le bouton de vérification changent, et un bouton s'ajoute sous la consigne.

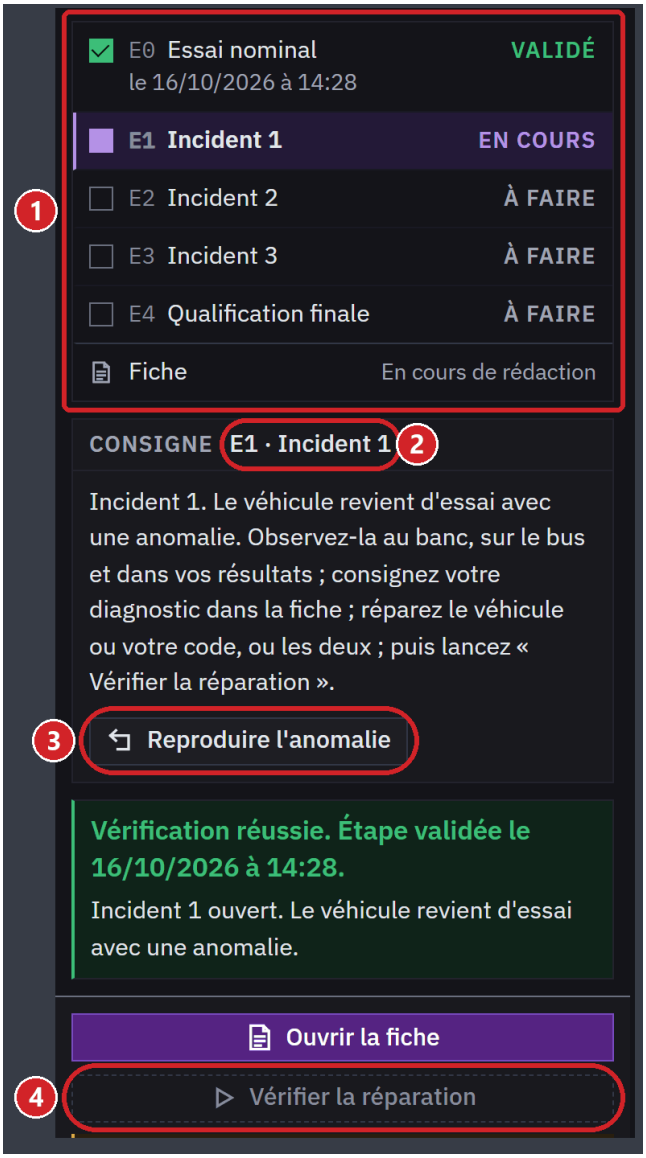


Figure 18. – Une étape d'incident : états, consigne et bouton « Vérifier la réparation ».

Étapes	1 E0 est « VALIDÉ », avec sa date ; E1 est l'étape en cours, ici « EN COURS » : elle vient de s'ouvrir, comme l'annonce le verdict vert de E0, encore affiché sous la consigne ; les étapes suivantes restent « À FAIRE ». Après une vérification manquée, E1 passerait à « ÉCHEC À CORRIGER ». 2 L'étape de la consigne, ici « E1 · Incident 1 », puis la consigne d'un incident : observer l'anomalie, consigner le diagnostic, réparer. 3 « Reproduire l'anomalie » remet le véhicule dans l'état où il est revenu d'essai ; son usage est décrit avec les incidents, dans la seconde partie. 4 Pendant les incidents, « Vérifier la réparation » remplace « Lancer la vérification ».
Ce qu'on doit observer	Pendant un incident, « Ouvrir la fiche » ouvre la fiche de l'incident en cours (Figure 26) ; la partie nominale reste consultable après E0. Sous 4, le délai avant la vérification suivante s'affiche comme en E0.
Si cela échoue	4 reste grisé : la liste sous le bouton nomme ce qui manque, par exemple « Mesure clé : consigner la mesure » tant que la mesure n'est pas consignée.

7. La fiche

La fiche est le document de votre groupe : la partie nominale, en E0, décrit votre décodage ; chaque incident, de E1 à E3, a sa propre partie de diagnostic. Elle s'ouvre dans le dock du bas du Cockpit. Le formulaire contrôle seulement la forme de chaque champ (un identifiant hexadécimal, un entier dans sa plage, un nombre décimal…), jamais la justesse : la justesse n'est jugée qu'à la vérification.

7.1 Ouvrir et parcourir la fiche

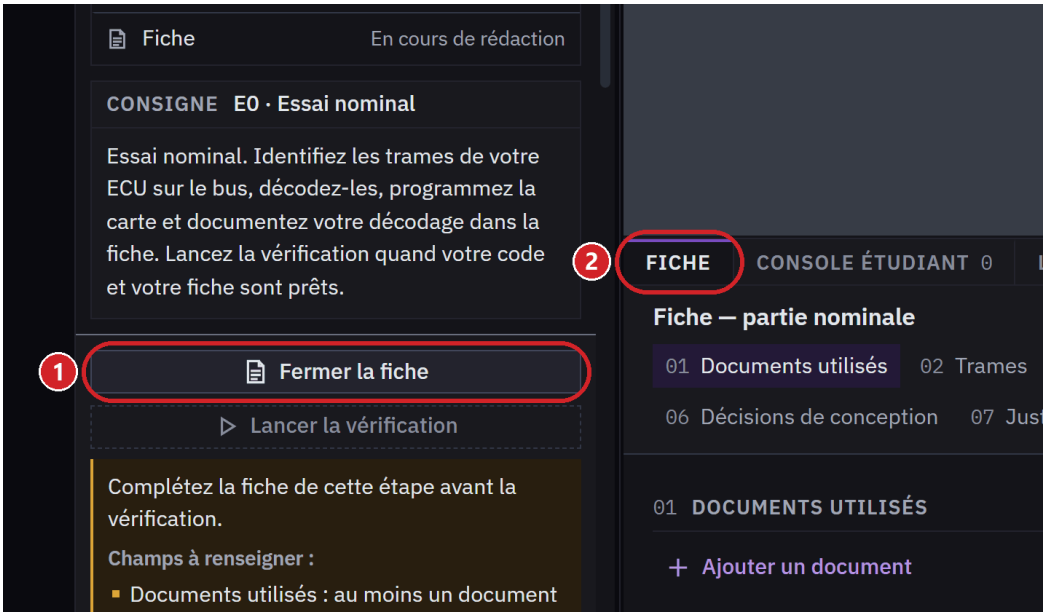


Figure 19. – Ouvrir la fiche : elle s'affiche dans l'onglet « Fiche » du dock.

Étapes	1 Cliquez sur « Ouvrir la fiche », sous la consigne ; une fois la fiche ouverte, ce bouton affiche « Fermer la fiche », comme sur la capture. 2 La fiche s'affiche dans l'onglet « Fiche » du dock, le premier à gauche ; le dock s'agrandit pour elle.
Ce qu'on doit observer	L'onglet 2 n'existe que pendant une mission. « Fermer la fiche », ou un clic sur un autre onglet, masque la fiche sans perdre ce que vous y avez tapé ; 1 ou 2 la réaffiche.
Si cela échoue	Le dock du bas était masqué : 1 le réaffiche de lui-même. Un message s'affiche sous le bouton de vérification au lieu de la fiche : lisez-le, puis fermez et rouvrez la mission avec le code du groupe.

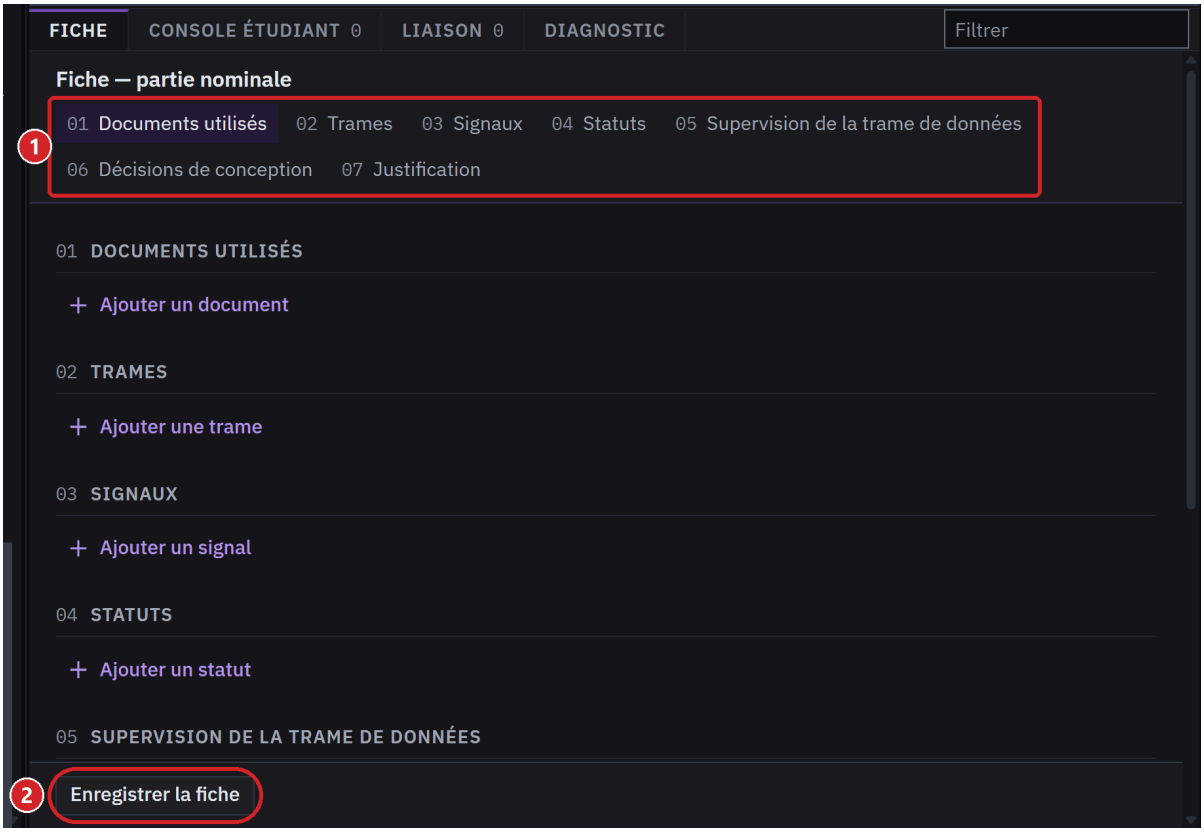


Figure 20. – La partie nominale vide : l'index des sept sections et « Enregistrer la fiche ».

Étapes	1 L'index des sections, numérotées 01 à 07 : « Documents utilisés », « Trames », « Signaux », « Statuts », « Supervision de la trame de données », « Décisions de conception », « Justification ». Un clic sur une entrée amène sa section en haut de la fiche ; l'entrée de la section affichée est surlignée. 2 « Enregistrer la fiche » reste visible en bas de la fiche.
Ce qu'on doit observer	Une fiche vide n'a aucun bloc : sous chaque titre, « Ajouter un document », « Ajouter une trame », « Ajouter un signal » ou « Ajouter un statut » crée un bloc (« Document 1 », « Trame 1 »…). Chaque bloc a son bouton « Supprimer » : un premier clic le change en « Confirmer la suppression », un second clic dans les 5 secondes supprime le bloc. Une section qui contient un champ mal formé apparaît en rouge dans l'index.
Si cela échoue	« Ajouter … » ne crée rien : un champ de la fiche est mal formé ; corrigez-le d'abord (Figure 21).

7.2 Enregistrer la fiche

- **Rien ne s'enregistre tout seul.** Dès qu'un champ change, « Modifications non enregistrées » s'affiche à droite de « Enregistrer la fiche », qui passe en couleur (Figure 23).
- « Enregistrer la fiche » enregistre tout le formulaire et affiche « Enregistré » à la place de « Modifications non enregistrées ». Une fiche incomplète s'enregistre : vous la complétez quand vous voulez.
- Le bouton de vérification, et « Consigner la mesure » pendant un incident, enregistrent aussi la partie ouverte ; sous le bouton de vérification, « La fiche sera enregistrée avant la vérification. » le rappelle.
- Un champ mal formé bloque tout l'enregistrement : la fiche répond « Certains champs ne sont pas au bon format : corrigez-les avant d'enregistrer. » et rien n'est enregistré (Figure 21).
- La fiche passe en lecture seule pendant une vérification (« Vérification en cours : la fiche n'est pas modifiable. »), quand sa partie est validée (« Partie validée : seuls la justification, les pages et les noms des signaux restent modifiables. » pour la partie nominale, « Partie validée : la fiche n'est pas modifiable. » pour un incident) et après la clôture (« La mission est clôturée : la fiche n'est plus modifiable. »).

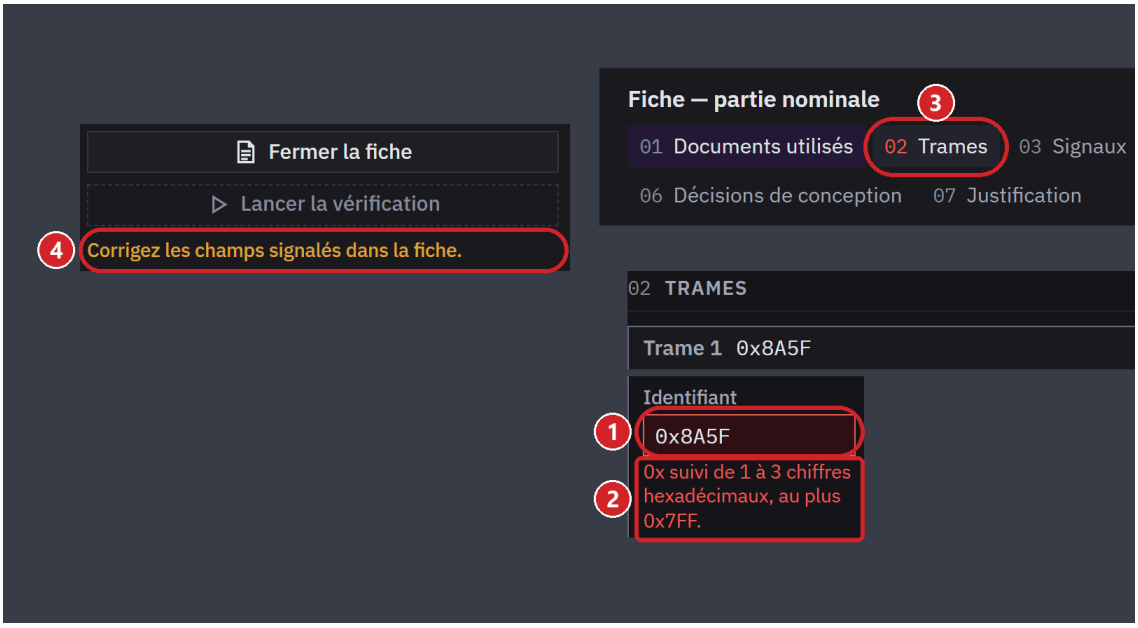


Figure 21. – Un champ mal formé (le reste de la fiche est masqué).

Étapes	1 Le champ refusé est encadré en rouge ; ici, un identifiant de quatre chiffres hexadécimaux. 2 Son message de format s'affiche dessous : « 0x suivi de 1 à 3 chiffres hexadécimaux, au plus 0x7FF. » 3 L'index marque en rouge la section qui contient le champ. 4 Sous le bouton de vérification, grisé : « Corrigez les champs signalés dans la fiche. »
Ce qu'on doit observer	Le contrôle a lieu quand vous quittez le champ et à l'enregistrement ; dès que vous retapez dans le champ, sa marque 1 disparaît. Une valeur bien formée mais fausse n'est jamais signalée ici : seul le verdict en tient compte.
Si cela échoue	Vous ne trouvez pas le champ fautif : cliquez sur « Enregistrer la fiche » ; la fiche amène le premier champ mal formé à l'écran et y place le curseur.

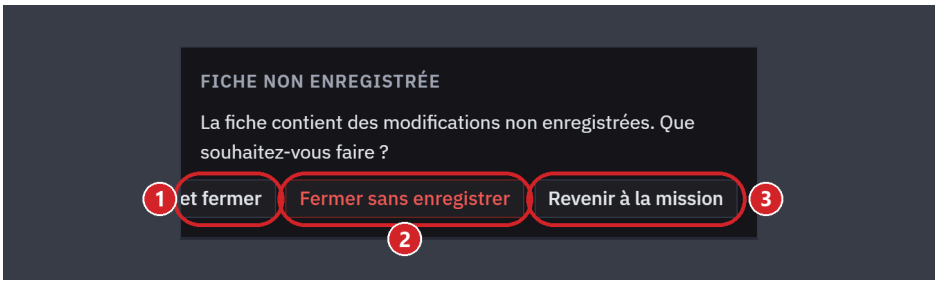


Figure 22. – Fermer la mission avec une fiche non enregistrée (l'arrière-plan est masqué).

Étapes	1 « Enregistrer et fermer » enregistre la fiche, puis ferme. Dans cette version, la fenêtre est trop étroite pour ses trois boutons : le début du libellé est coupé par son bord gauche et seul « et fermer » reste lisible. 2 « Fermer sans enregistrer » ferme en abandonnant les modifications ; ce bouton rouge reste inactif une fraction de seconde après l'ouverture de la fenêtre. 3 « Revenir à la mission » annule la fermeture : vous retrouvez la fiche telle quelle.
Ce qu'on doit observer	La fenêtre « Fiche non enregistrée » s'ouvre quand vous fermez la mission (« Fermer ») ou l'application alors que la fiche contient des modifications non enregistrées : « La fiche contient des modifications non enregistrées. Que souhaitez-vous faire ? » À la fermeture de l'application, 3 s'appelle « Revenir à l'application » ; ce bouton a le focus à l'ouverture, et la touche Échap a le même effet.
Si cela échoue	L'enregistrement demandé par 1 échoue (un champ mal formé, par exemple) : la fenêtre devient « Fermeture suspendue » et la raison est écrite au bas de la fiche ; cliquez sur « Revenir à la mission », corrigez, puis fermez de nouveau. Pour abandonner les modifications, fermez de nouveau et choisissez 2.

7.3 Les informations de la partie nominale

7.3.1 Positions des bits

Plusieurs champs décrivent la place d'une information dans les 8 octets de données d'une trame, par trois valeurs : « Ordre », « LSB » et « Longueur ».

- Le bit i de la trame (de 0 à 63) est le bit (i mod 8) de l'octet $\lfloor i/8 \rfloor$; l'octet 0 est transmis le premier et le bit 0 est le poids faible de chaque octet.
- « LSB » est la position du bit de poids faible du champ ; « Longueur » est son nombre de bits.
- En `INTEL`, les bits suivants du champ sont $i+1$, $i+2$… En `MOTOROLA`, ils remontent l'octet, puis continuent au bit 0 de l'octet précédent.

Champ fictif	Ordre, LSB, longueur	Bits occupés
CRC de la trame fictive `0x3C7`	`INTEL`, 40, 8	40 à 47 : tout l'octet 5.
Compteur de la trame `0x3C7`	`INTEL`, 36, 4	36 à 39 : les quatre bits de poids fort de l'octet 4.
Température liquide (exemple)	`MOTOROLA`, 22, 10	22 et 23 (bits 6 et 7 de l'octet 2, poids faible du champ), puis 8 à 15 (tout l'octet 1, poids fort).
Statut fictif	`INTEL`, 6, 2	6 et 7 : les deux bits de poids fort de l'octet 0.

La valeur physique se calcule à partir de la valeur brute lue dans ces bits : valeur physique = valeur brute × facteur + offset. Pour le signal fictif, le facteur vaut 0,25 et l'offset −40, en °C ; sa valeur brute maximale, 1023 (dix bits à 1), est ici son code invalide et ne se convertit pas.

7.3.2 Documents utilisés

Champ	Ce qu'il faut saisir	Erreurs fréquentes
« Document »	Un document de la liste, qui reprend la bibliothèque de votre mission. Ajoutez un bloc par document lu.	Laisser le choix vide : le bloc reste incomplet.
« Pages »	Texte libre, non évalué : les pages ou les sections lues (`p. …`).	Aucune : le champ n'est pas contrôlé.

Où chercher : la bibliothèque (Figure 14). Exemple fictif : dans « Document », la fiche technique d'un capteur ; dans « Pages », *section Caractéristiques de sortie*. La fiche exige au moins un document.

7.3.3 Trames

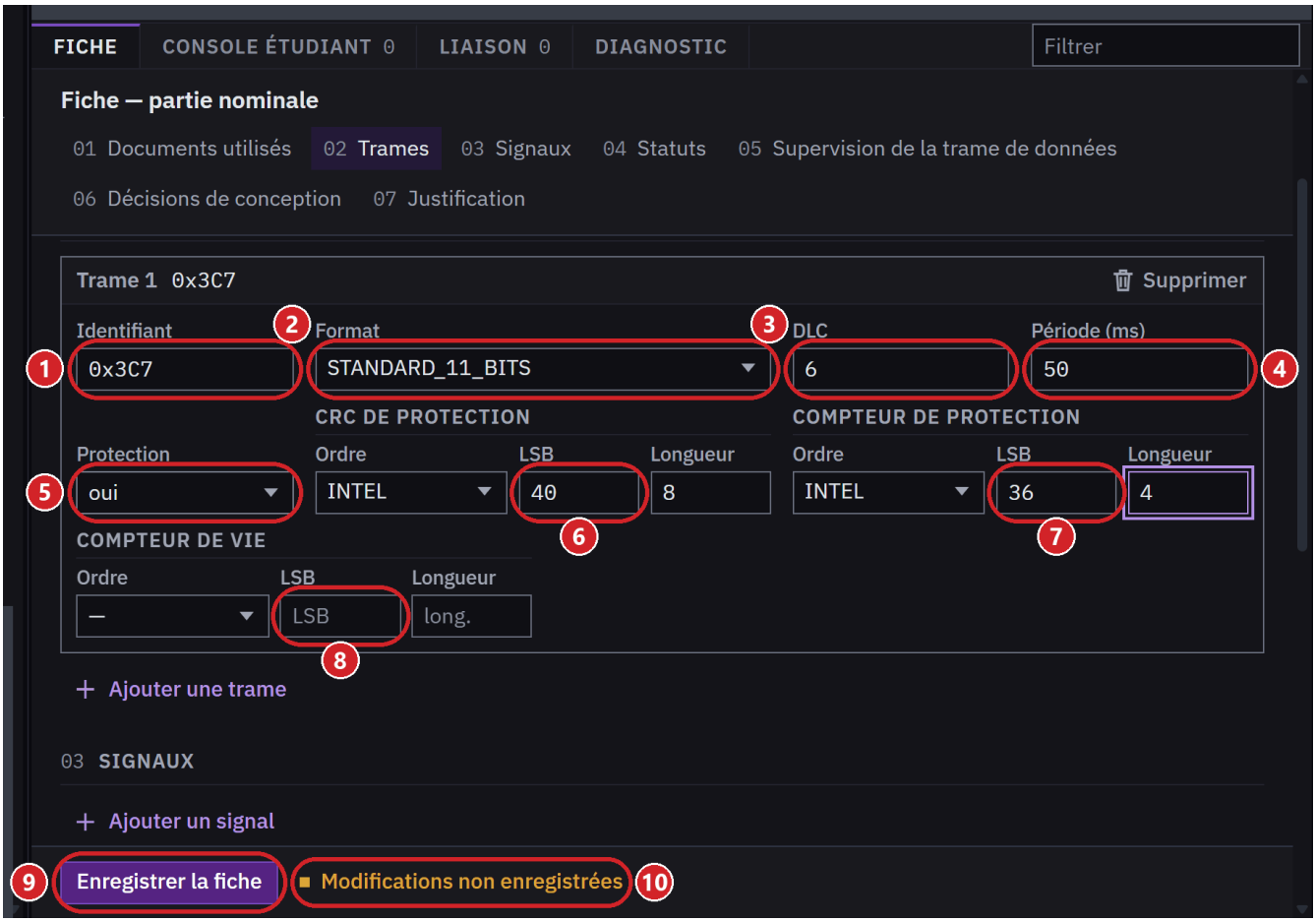


Figure 23. – Un bloc « Trame » rempli avec les valeurs fictives du tutoriel.

Étapes	1 « Identifiant ». 2 « Format ». 3 « DLC ». 4 « Période (ms) ». 5 « Protection ». 6 Le « LSB » du « CRC de protection », entre son « Ordre » et sa « Longueur ». 7 Le « LSB » du « Compteur de protection », avec les mêmes voisins. 8 Le « LSB » du « Compteur de vie », laissé vide comme ses voisins : la trame fictive n'en a pas. 9 « Enregistrer la fiche », en couleur tant que la fiche a changé. 10 « Modifications non enregistrées ».
Ce qu'on doit observer	Le titre du bloc reprend l'identifiant (« Trame 1 » suivi de `0x3C7`). Après 9, 10 laisse place à « Enregistré ».
Si cela échoue	Un champ s'encadre de rouge : sa forme est refusée ; lisez le message sous le champ et le tableau ci-dessous.

Champ	Sens et format exact	Erreurs fréquentes	Exemple fictif
« Identifiant »	L'identifiant CAN de la trame, sur 11 bits : `0x` suivi de 1 à 3 chiffres hexadécimaux, au plus `0x7FF`. La fiche l'affiche ensuite en majuscules, sur trois chiffres (`0x3c7` devient `0x3C7`).	Oublier `0x` ; écrire l'identifiant en décimal ; écrire quatre chiffres (`0x03C7`) ; dépasser `0x7FF`.	`0x3C7`
« Format »	Le format de la trame ; une seule valeur proposée, `STANDARD_11_BITS`.	Laisser le choix vide.	`STANDARD_11_BITS`
« DLC »	Le nombre d'octets de données : entier de 0 à 8.	Donner une longueur en bits.	`6`
« Période (ms) »	L'intervalle nominal entre deux trames de cet identifiant, en millisecondes : entier de 0 à 100 000, sans unité.	Écrire `50 ms` ou `50,0` ; recopier un écart isolé au lieu d'une moyenne sur plusieurs trames.	`50`
« Protection »	`oui` si la trame porte une protection de bout en bout (CRC et compteur), `non` sinon.	Répondre sans avoir observé la trame sur plusieurs envois.	`oui`

Champ	Sens et format exact	Erreurs fréquentes	Exemple fictif
« CRC de protection »	La place du CRC de protection : « Ordre » (`INTEL` ou `MOTOROLA`), « LSB » (entier de 0 à 63), « Longueur » (entier de 1 à 64). À remplir si la protection est `oui`.	Donner le numéro de l'octet au lieu de celui du bit ; un champ qui déborde au-delà du bit 63.	`INTEL`, `40`, `8`
« Compteur de protection »	La place du compteur de la protection : mêmes trois champs.	Le confondre avec un compteur de vie.	`INTEL`, `36`, `4`
« Compteur de vie »	Un compteur distinct du compteur de protection, s'il existe ; sinon, laissez les trois champs vides. Ils sont tous vides ou tous remplis.	Remplir un seul des trois champs ; y recopier le compteur de protection.	vides

Où chercher : le tableau « Trames reçues par le nœud étudiant » (colonnes « ID », « DLC », « Données », et « t (ms) » pour la période, en moyennant plusieurs écarts), le détail « Trame sélectionnée », et, pour la protection, les spécifications de la bibliothèque. La fiche exige au moins une trame.

7.3.4 Signaux

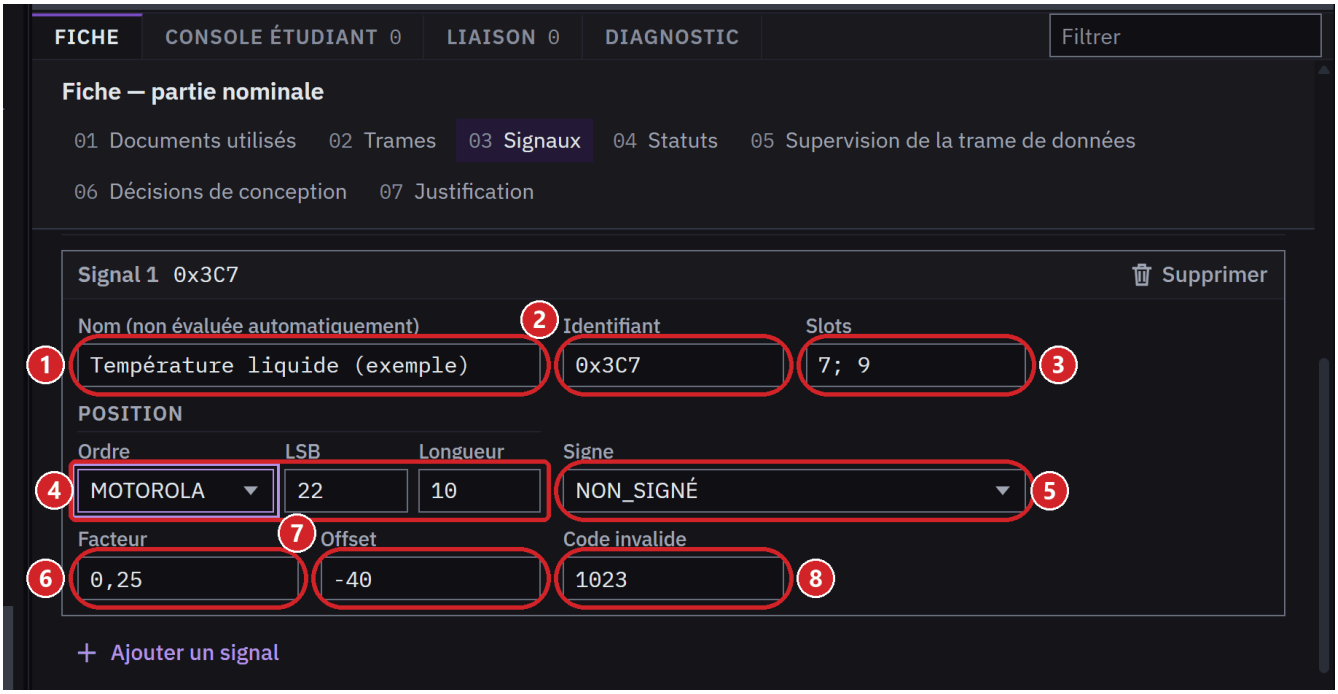


Figure 24. – Un bloc « Signal » rempli avec le signal fictif du tutoriel.

Étapes	1 « Nom », texte libre non évalué. 2 « Identifiant » de la trame qui porte le signal. 3 « Slots ». 4 « Position » : « Ordre », « LSB » et « Longueur ». 5 « Signe ». 6 « Facteur ». 7 « Offset ». 8 « Code invalide ».
Ce qu'on doit observer	Le titre du bloc reprend l'identifiant de 2. Le champ 3 montre la saisie de deux slots séparés par un point-virgule ; la virgule et l'espace sont aussi acceptés. Le signal fictif du tutoriel n'a qu'un slot, `7`.
Si cela échoue	6 ou 7 refusé : écrivez un seul nombre, sans unité ni fraction. 8 refusé : un entier, `0x…` ou `AUCUN`.

Champ	Sens et format exact	Erreurs fréquentes	Exemple fictif
« Nom »	Le nom que vous donnez au signal ; texte libre, non évalué.	Aucune.	*Température liquide (exemple)*
« Identifiant »	La trame qui porte le signal ; même format que l'identifiant d'une trame.	Un identifiant absent de la section « Trames ».	`0x3C7`
« Slots »	Le ou les numéros de slot où ce signal est publié, pris dans les « Sorties à publier » : entiers de 0 à 31, séparés par une virgule, un espace ou un point-virgule.	Écrire le code de la sortie au lieu de son numéro ; un intervalle écrit avec un tiret ; un numéro au-delà de 31.	`7`

Champ	Sens et format exact	Erreurs fréquentes	Exemple fictif
« Position »	« Ordre », « LSB », « Longueur » du signal (voir les positions des bits).	Inverser `INTEL` et `MOTOROLA` ; donner le bit de poids fort au lieu du LSB.	`MOTOROLA`, `22`, `10`
« Signe »	`NON_SIGNÉ` ou `COMPLÉMENT_À_DEUX` : la façon de lire la valeur brute.	Confondre le signe de la valeur brute avec celui de la grandeur physique : un offset négatif peut rendre la grandeur négative avec une valeur brute non signée.	`NON_SIGNÉ`
« Facteur »	Le multiplicateur de la valeur brute : nombre décimal, virgule ou point, exposant accepté (`2.5e-1`).	Une fraction (`1/4`) ; une unité (`0,25 °C`) ; un espace entre les milliers.	`0,25`
« Offset »	La constante ajoutée après le facteur, dans l'unité de la grandeur publiée ; nombre décimal.	Inverser l'ordre des opérations (l'offset s'ajoute après le facteur).	`-40`
« Code invalide »	La valeur brute qui signale une valeur indisponible : entier décimal, `0x…` en hexadécimal, ou `AUCUN` s'il n'y en a pas.	Laisser le champ vide au lieu d'écrire `AUCUN` ; donner une valeur physique au lieu de la valeur brute.	`1023`

Où chercher : les fiches techniques des capteurs (caractéristique de sortie, plage, diagnostics), les « Sorties à publier » (slots et unités), et le bus : faites varier la grandeur au banc et observez quels bits changent. La fiche exige au moins un signal.

7.3.5 Statuts

Un statut est un champ de la trame par lequel la source dit si sa valeur est valable. Il se décrit comme un signal, avec moins de champs : un bloc « Statut » (« Ajouter un statut ») a les mêmes champs « Identifiant », « Slots » et « Position » que le bloc « Signal » (Figure 24), puis « Code OK ».

Champ	Sens et format exact	Erreurs fréquentes	Exemple fictif
« Identifiant »	La trame qui porte le statut.	—	`0x3C7`
« Slots »	Les slots concernés par ce statut ; même format que pour un signal.	Oublier un slot concerné.	`7`
« Position »	« Ordre », « LSB », « Longueur » du statut.	Voir les signaux.	`INTEL`, `6`, `2`
« Code OK »	La valeur brute qui signifie que tout va bien : nombre entier, en décimal.	Donner le code d'un état de défaut ; écrire le code en binaire (`10` est lu dix) ou en hexadécimal.	`2`

Où chercher : les diagnostics décrits dans les fiches techniques et la spécification de la source, et le bus. La fiche exige au moins un statut.

7.3.6 Supervision de la trame de données



Figure 25. – La supervision fictive du tutoriel, au-dessus des décisions de conception encore vides.

Étapes	1 « Premier timeout (ms) ». 2 « Timeout (ms) ». 3 « Péremption (ms) ». 4 « Pertes consécutives tolérées ». 5 « Décisions de conception » : quatre listes, de DD-1 à DD-4.
Ce qu'on doit observer	Les quatre champs de supervision sont des entiers de 0 à 100 000, sans unité. Les listes de 5 commencent sur « — » : aucune décision n'est choisie.
Si cela échoue	« Entier de 0 à 100 000. » sous un champ : vous avez tapé une unité, une virgule ou un nombre négatif.

Champ	Sens	Erreurs fréquentes	Exemple fictif
« Premier timeout (ms) »	Le délai maximal accepté pour recevoir la première trame valide, au démarrage.	Le confondre avec le timeout, qui ne s'applique qu'après la première trame valide.	`300`
« Timeout (ms) »	Le délai maximal accepté entre deux trames valides, ensuite.	Écrire la durée en secondes ou avec son unité.	`150`
« Péremption (ms) »	L'âge au-delà duquel une donnée reçue n'est plus considérée comme fraîche (voir DD-4).	La confondre avec le timeout : la péremption porte sur l'âge de la donnée, le timeout sur l'écart entre deux trames valides.	`90`
« Pertes consécutives tolérées »	Le nombre de trames consécutives qui peuvent manquer sans que la supervision déclare un défaut.	Donner une durée au lieu d'un nombre de trames.	`2`

Où chercher : ces quatre valeurs ne figurent ni dans l'énoncé ni dans la bibliothèque. Votre encadrant les communique à tous les groupes au lancement de l'E0 ; notez-les, car la fiche et le code de votre carte doivent appliquer exactement ces valeurs-là. Si vous ne les avez pas reçues, demandez-les-lui.

7.3.7 Décisions de conception

Chaque décision fixe la conduite de votre décodeur dans une situation précise ; elle est contrôlée comme les autres champs et doit correspondre à ce que fait votre code.

Décision	Choix proposés et effet de chacun dans votre code
« DD-1 Valeur égale au code invalide »	`REJETER` : le slot reçoit un rejet (`n7rt_reject`) ; `PUBLIER` : la valeur est publiée comme les autres.
« DD-2 Trame à protection non conforme »	`IGNORER_SES_DONNÉES` : les données de cette trame ne sont pas utilisées ; `UTILISER_SES_DONNÉES` : elles sont décodées malgré tout.
« DD-3 Valeur hors plage sous statut OK »	`REJETER` : rejet ; `BORNER` : la borne la plus proche de la plage est publiée ; `PUBLIER` : la valeur est publiée telle quelle.

Décision	Choix proposés et effet de chacun dans votre code
« DD-4 Donnée plus vieille que la péremption »	`REJETER` : rejet ; `PUBLIER_LA_DERNIÈRE` : la dernière valeur reste publiée.

Où chercher : les « Exigences fonctionnelles » de l'énoncé, puis le règlement. Erreur fréquente : choisir une décision que le code n'applique pas, ou l'inverse. Aucun exemple n'est donné : tout choix pourrait être la réponse d'une mission.

7.3.8 Justification

« Justification » (non évaluée automatiquement) : un texte libre où vous expliquez votre raisonnement, les mesures faites et les documents utilisés. La vérification ne le lit jamais ; votre encadrant, si. Il reste modifiable après la validation de E0, comme les pages et les noms des signaux.

7.4 La fiche d'incident (E1 à E3)

Pendant un incident, « Ouvrir la fiche » ouvre la fiche de l'incident en cours. Elle a cinq blocs. Les listes de catégories et d'états observés sont les mêmes pour les quatre missions.

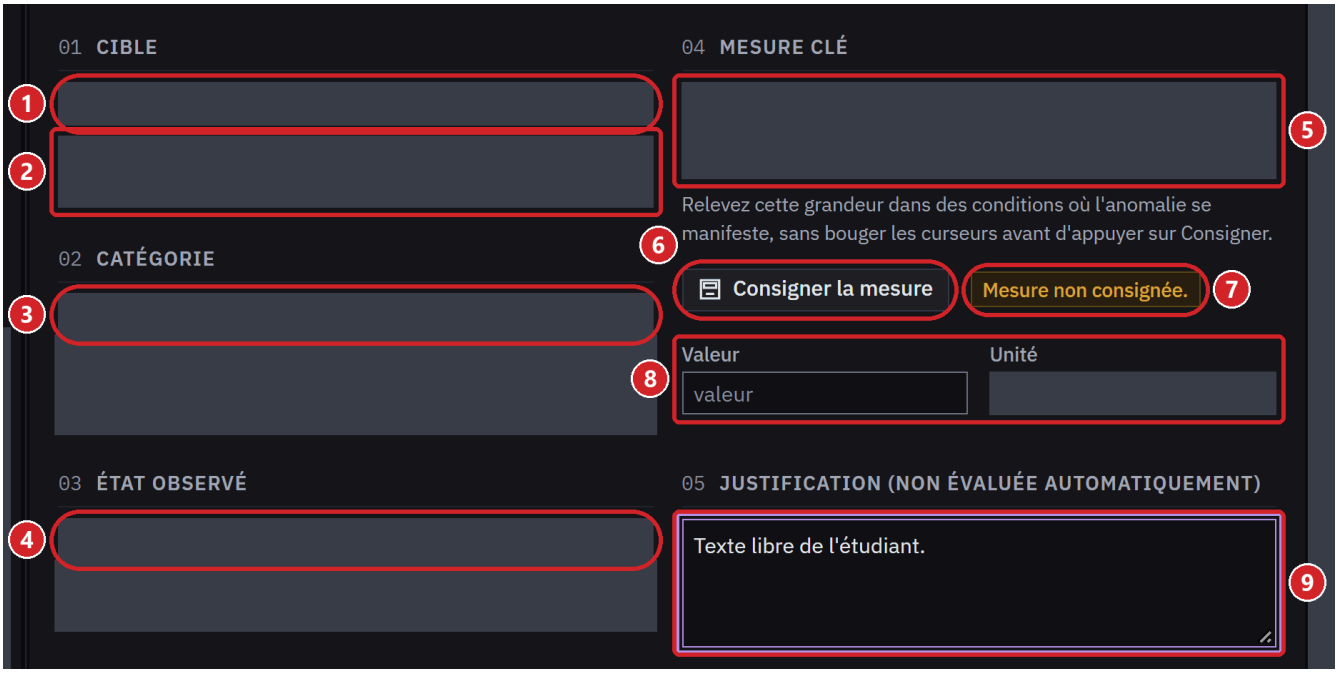


Figure 26. – La fiche d'un incident : les cinq blocs (toutes les valeurs choisies sont masquées).

Étapes	1 « Cible » : choisissez d'abord le type de cible. 2 Puis désignez la cible : deux broches, un composant ou un identifiant, selon le type. 3 « Catégorie » : la famille de la cause ; sa définition s'affiche sous la liste. 4 « État observé » : ce que vous constatez ; sa définition s'affiche sous la liste. 5 La grandeur imposée par l'état observé : « Grandeur imposée : … ». 6 « Consigner la mesure » scelle les conditions de votre mesure. 7 L'état de la consignation : ici « Mesure non consignée. » ; après un clic sur 6, « Mesure consignée le … », suivi de la date et de l'heure. 8 « Valeur » et « Unité » de la mesure clé. 9 La justification, en texte libre.
Ce qu'on doit observer	Tant qu'aucun état n'est choisi, 5 affiche « Choisissez l'état observé : il impose la grandeur à relever. » Sous 5, la consigne de mesure : « Relevez cette grandeur dans des conditions où l'anomalie se manifeste, sans bouger les curseurs avant d'appuyer sur Consigner. » La liste « Unité » de 8 ne propose que les unités de la grandeur imposée.
Si cela échoue	« La mesure ne peut être consignée que pendant l'incident en cours, hors vérification. » : attendez la fin de l'essai. « Aucune trame récente sur le bus : lancez le véhicule, puis consignez la mesure. » : démarrez une session (« Start »), attendez au moins 3 s, puis consignez de nouveau.

Type de cible	Désignation et format	Exemple fictif
« Conducteur (ses deux broches) »	Un fil ou une liaison du banc, désigné par ses deux extrémités, « Broche A » et « Broche B », choisies dans les listes des broches du banc sous la forme `désignateur.broche`.	`R9.p1` – `U7.in3`
« Composant (son désignateur) »	Un composant du banc, choisi par son désignateur dans la liste « Composant ».	`U7`
« Identifiant (trame du bus) »	Une trame du bus, par son identifiant : même format que dans la partie nominale.	`0x3C7`

Catégorie	Définition affichée sous la liste
`SENSOR/PHYSICAL`	Défaut du capteur, de son faisceau, de son montage ou de la grandeur physique. Les trames de l'ECU source restent conformes à leur protection et à leur rythme.
`SOURCE_ECU/APPLICATION`	Défaut de l'ECU source lui-même (logique, configuration, étalonnage, alimentation de l'ECU). Ses trames restent conformes à leur protection ; leur contenu ou leur présence est faux.
`MESSAGE/E2E`	Le message ne passe pas le contrôle de protection de bout en bout, ou un autre émetteur utilise le même identifiant.
`CAN_BUS/PROTOCOL`	Défaut de la couche CAN : trames d'erreur, acquittement, bus-off, débit.

Les états observés sont rangés dans un ordre : chaque définition suppose que les précédentes ne s'appliquent pas, et la dernière se lit « Rien de ce qui précède ». L'état choisi impose la grandeur de la mesure clé et ses unités (tableau ci-dessous).

État observé	Définition	Grandeur imposée	Unités
`TRAMES_ABSENTES`	Les trames n'apparaissent plus sur le bus.	Nombre de trames de l'identifiant cible (pour une cible composant : de la trame DATA de l'ECU cible) reçues en 1,000 s.	trames/s
`TRAMES_FIGÉES`	Présentes à leur rythme, contenu et compteur identiques d'une trame à l'autre.	Incrément du compteur entre deux trames consécutives de la trame concernée.	sans unité
`PROTECTION_NON_CONFORME`	Au moins une trame sur 100 a une protection non conforme.	Proportion des trames concernées dont la protection est conforme, sur 100 trames consécutives.	%, fraction
`SÉQUENCE_NON_CONFORME`	Protection conforme, mais compteur discontinu ou trames en surnombre.	Nombre de trames concernées reçues en 1,000 s.	trames/s
`DÉFAUT_SIGNALÉ_PAR_LA_SOURCE`	Trames conformes ; la source signale un défaut (statut différent de « OK » ou code d'invalidité).	Tension continue, par rapport à la broche de masse de l'ECU source, à la broche de l'ECU source où aboutit la cible (pour un composant : reliée à sa sortie).	V, mV
`VALEUR_HORS_PLAGE`	Trames conformes, statut OK, valeur hors de la plage codable ou code d'invalidité sous statut OK.	Valeur brute (entier) du signal concerné.	sans unité
`VALEUR_INCOHÉRENTE`	Trames conformes, statut OK, valeur dans la plage mais contraire à la redondance, à la grandeur imposée au banc ou à une autre grandeur transmise.	Valeur physique transmise par l'ECU source pour le signal qui dépend de la cible, moyenne sur 1 s, aux conditions capturées.	%, fraction, tr/min, Hz, °, rad, bar, kPa, 0/1
`AUCUNE_ANOMALIE_SUR_LE_BUS`	Rien de ce qui précède.	Tension à la broche LV_IN de l'ECU source.	V, mV

Où chercher, et erreurs fréquentes, pour les trois premiers blocs :

- **Cible** : les désignateurs et les broches du schéma du banc, dans le Physical Lab (inspecteur et multimètre), ou le tableau « Trames reçues par le nœud étudiant » pour un identifiant. Erreurs fréquentes : désigner un composant quand le défaut porte sur une liaison ; prendre une broche voisine, ou une broche d'un autre net.

- **Catégorie** : la définition affichée sous la liste, confrontée à vos observations du bus (les trames restent-elles conformes à leur protection et à leur rythme ?) et du banc. Erreur fréquente : classer l'incident d'après le symptôme visible dans vos résultats plutôt que d'après sa cause.
- **État observé** : le tableau des trames et le détail « Trame sélectionnée » (présence et rythme des trames, compteur, protection, statut, valeur), en parcourant les états dans l'ordre de la liste. Erreur fréquente : retenir un état alors qu'un état précédent de la liste s'applique.
- **Mesure clé lue sur le bus** : comptez les trames de l'identifiant sur 1,000 s avec la colonne « t (ms) » (le champ « Filtre ID (hex) » ne garde que cet identifiant), comparez les « Données » de deux trames consécutives pour un compteur, lisez la valeur brute dans les bits du signal. Une tension se relève au multimètre du Physical Lab.

Champ	Sens et format exact	Erreurs fréquentes
« Valeur »	La valeur relevée de la grandeur imposée : nombre décimal, virgule ou point, sans unité.	Écrire l'unité dans la valeur (`2,48 V`) ; arrondir trop tôt.
« Unité »	L'unité de la valeur, parmi celles de la grandeur imposée. L'application convertit les notations d'une même grandeur (V et mV, % et fraction…).	Laisser le choix vide ; choisir l'unité avant l'état observé (la liste change avec lui).
« Consigner la mesure »	Pendant l'incident en cours, carte connectée et session démarrée depuis au moins 3 s (« Start ») : l'application enregistre la grandeur, les états des commandes du banc et la lecture du multimètre, et scelle les trames du bus des dernières secondes et votre circuit. Une nouvelle consignation remplace la précédente. Le même bouton existe sous le multimètre du Physical Lab.	Consigner après avoir réparé, ou après avoir bougé un curseur : la mesure ne correspond plus à l'anomalie.
« Justification »	Votre raisonnement, en texte libre ; jamais lu par la vérification.	—

Exemple fictif complet : cible `R9.p1` – `U7.in3` (type conducteur), mesure clé `2,48` en `V`. Aucun exemple de catégorie ni d'état observé n'est donné : toute valeur pourrait être la réponse d'un incident réel. La mesure au multimètre et la consignation pas à pas sont détaillées dans la seconde partie du tutoriel.

8. Le bus CAN

Le trafic du bus et les résultats de votre décodeur s'observent dans la vue **Cockpit**, deuxième bouton de la barre d'activité. Le tableau central montre les trames que reçoit le nœud étudiant, c'est-à-dire votre carte ; le panneau de droite montre ce que votre code en a tiré. Les captures de cette section ont été prises sans mission : le trafic est celui de l'ECU virtuel du SDK (section « Stimuli — ECU virtuel » du panneau latéral), qui émet une trame d'exemple. La carte exécute le gabarit fourni (Le squelette fourni), qui publie la longueur de chaque trame dans le slot 0 et son premier octet dans le slot 1.

8.1 Démarrer et arrêter une session

Une session N7RT-Link relie l'application et la carte : l'application transmet chaque trame du bus à votre nœud, la carte exécute votre code et renvoie ce qu'il publie. Sans session, le tableau reste vide et affiche « Aucune trame pour l'instant : démarrez une session (Start) pour voir le trafic du bus. »

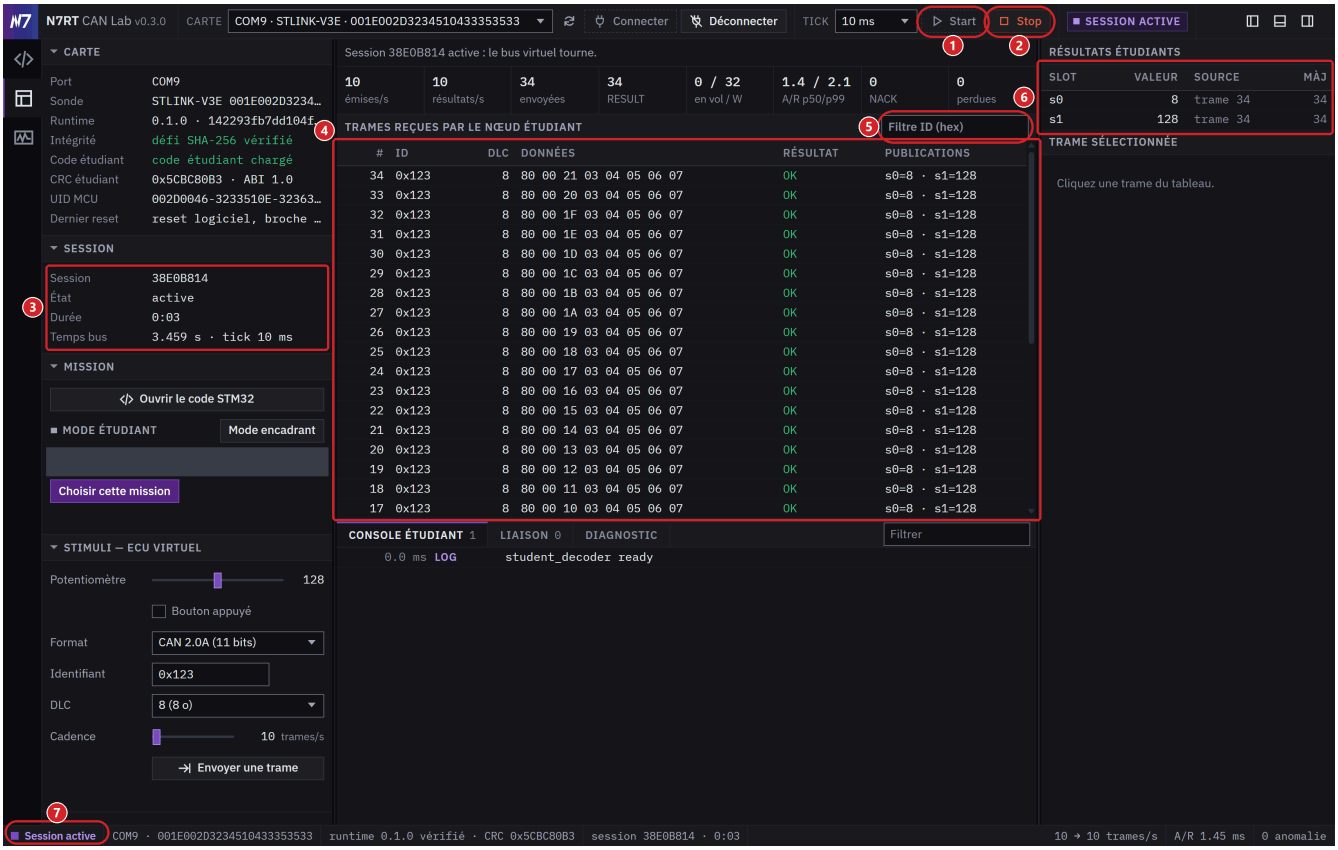


Figure 27. – Une session ouverte sur l'ECU virtuel du SDK.

Étapes	1. Vérifiez que l'état de la carte, en haut à droite de la fenêtre, est « Prête », puis cliquez **Start** 1. Le sélecteur **Tick**, juste à sa gauche, fixe la période des appels de `student_on_tick` (10 ms par défaut). 2. Pour arrêter, cliquez **Stop** 2. Arrêtez toujours la session avant de programmer la carte.
Ce qu'on doit observer	**Start** 1 se grise et **Stop** 2 s'active. Le bloc « Session » 3 passe à « active » : « Durée » compte le temps écoulé, « Temps bus » donne le temps du bus et la période du tick. Le tableau 4 se remplit, la trame la plus récente en haut ; le champ « Filtre ID (hex) » 5 le restreint à un identifiant. « Résultats étudiants » 6 affiche la dernière valeur de chaque slot écrit par votre code. La barre d'état 7 indique « Session active ».
Si cela échoue	**Start** reste grisé : survolez-le, la raison s'affiche (« Connectez une carte prête pour démarrer une session. », « Une session est déjà en cours. »). Si l'état de la carte est « Pas de code étudiant », programmez d'abord la carte (Le code STM32).

La ligne de compteurs au-dessus du tableau résume la session : « émises/s » et « résultats/s » (trames envoyées à la carte et réponses reçues, par seconde), « envoyées » et « RESULT » (totaux de la session), « en vol / W » (trames en attente de réponse et fenêtre de crédits), « A/R p50/p99 » (aller-retour d'une trame à sa réponse, en

ms), « NACK » (trames refusées par la carte) et « perdues » (trames restées sans réponse). En fonctionnement normal, « NACK » et « perdues » restent à 0 ; survolez un compteur pour lire sa définition.

À savoir **Start** et **Stop** pilotent la session avec la carte. Ils sont indépendants des boutons **RUN**, **STOP** et **RESET** du Physical Lab, qui pilotent seulement la simulation du circuit (Le Physical Lab).

Pendant une mission, le bus est celui du véhicule simulé : les stimuli manuels sont refusés (« Pendant une mission, le bus est celui du banc : pas de trame manuelle. ») et le **Tick** est imposé par la mission (sélecteur grisé, « Période de tick fixée par la mission »). Chaque **Start** remet à zéro la mémoire de votre code et rappelle `student_init` : vos variables statiques reprennent leur valeur initiale.

8.2 Lire le tableau des trames

À la largeur des captures, le tableau « Trames reçues par le nœud étudiant » affiche six colonnes :

Colonne	Contenu
#	numéro de la trame dans la session, croissant ;
ID	identifiant de la trame, en hexadécimal ;
DLC	code de longueur transmis sur le bus (pour une trame classique, le nombre d'octets de données) ;
Données	les octets reçus, en hexadécimal, l'octet 0 à gauche ;
Résultat	« OK » : la carte a livré la trame à votre code et renvoyé sa réponse ; « +débordement » : votre code a écrit plus de 8 entrées pendant cet appel (les suivantes sont perdues) ; « +budget » : l'appel a dépassé son budget de temps ; « NACK », « perdue » ou « non livrée » : la trame n'a pas été traitée normalement (voyez l'onglet « Liaison ») ;
Publications	ce que `student_on_frame` a publié pour cette trame, sous la forme `s7=…` (une valeur) ou `s7=rejet` ; « — » si rien.

Quand le tableau est plus large, cinq colonnes s'ajoutent : « t (ms) » (instant de réception, en millisecondes de temps bus), « Type » (format de la trame), « A/R ms », « Cycles » (cycles processeur de votre `student_on_frame`) et « Session ». Pour les faire apparaître, agrandissez la fenêtre ou masquez un panneau avec les boutons en haut à droite de la fenêtre (« Panneau latéral », « Résultats »).

- **Filtrer un identifiant** : tapez-le dans « Filtre ID (hex) », avec ou sans `0x` ; seules ses trames restent. Un texte qui n'est pas hexadécimal est encadré en rouge. Videz le champ pour revoir tout le trafic.
- **Figer la lecture** : dès que vous faites défiler le tableau vers le bas, la vue reste immobile ; les nouvelles trames s'ajoutent au-dessus. Remontez tout en haut pour reprendre le défilement.
- **Débit élevé** : le titre peut indiquer « non affichées (débit > affichage) ». L'application saute alors des lignes à l'écran ; la carte, elle, reçoit toutes les trames.
- **Mesurer une période** : filtrez l'identifiant, puis faites la différence entre les « t (ms) » de deux trames successives, ou entre les « Temps bus » de deux trames cliquées (Figure 28). Moyennez sur plusieurs trames : l'émission varie légèrement.

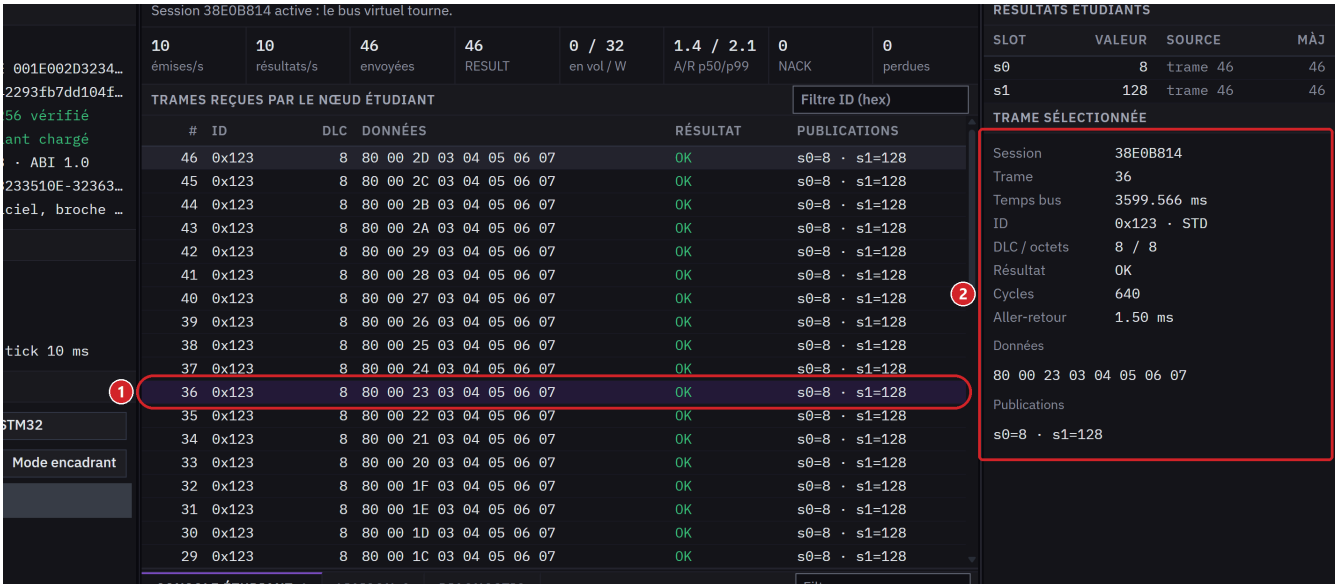


Figure 28. – Le détail d'une trame cliquée.

Étapes	1. Cliquez une ligne du tableau 1.
Ce qu'on doit observer	La ligne cliquée reste surlignée (ici la trame 36 : les trames arrivées depuis l'ont fait descendre). « Trame sélectionnée » 2 donne son numéro (« Trame »), son instant de réception (« Temps bus », en ms depuis le début de la session), son identifiant suivi du format (« STD » : identifiant standard de 11 bits), « DLC / octets », « Résultat », « Cycles », « Aller-retour », puis ses « Données » et ses « Publications ».
Si cela échoue	« Cliquez une trame du tableau. » reste affiché : aucune trame n'a encore été reçue, ou le filtre n'en laisse passer aucune.

8.3 Résultats étudiants

Le panneau de droite « Résultats étudiants » garde la dernière valeur écrite dans chaque slot depuis le début de la session :

Colonne	Contenu
Slot	numéro du slot (`s0`, `s1`…) ;
Valeur	la dernière valeur publiée, ou « rejet » ;
Source	« trame … » : écrite par `student_on_frame` en traitant la trame de ce numéro ; « tick … » : écrite par `student_on_tick` au tick de cet instant (en ms de temps bus) ;
Màj	nombre de mises à jour du slot.

Quand une mission est ouverte, un slot de la mission affiche son code et son nom, tels qu'ils figurent dans les « Sorties à publier » de l'énoncé, et sa valeur avec son unité ; la source s'affiche au survol de la ligne.

Attention Les vérifications ne lisent que ce que votre code publie **à chaque tick**, dans `student_on_tick` (source « tick … »). Une valeur publiée seulement dans `student_on_frame` s'affiche ici, mais ne compte pas (Quand publier, quand rejeter).

8.4 La console et les onglets du bas

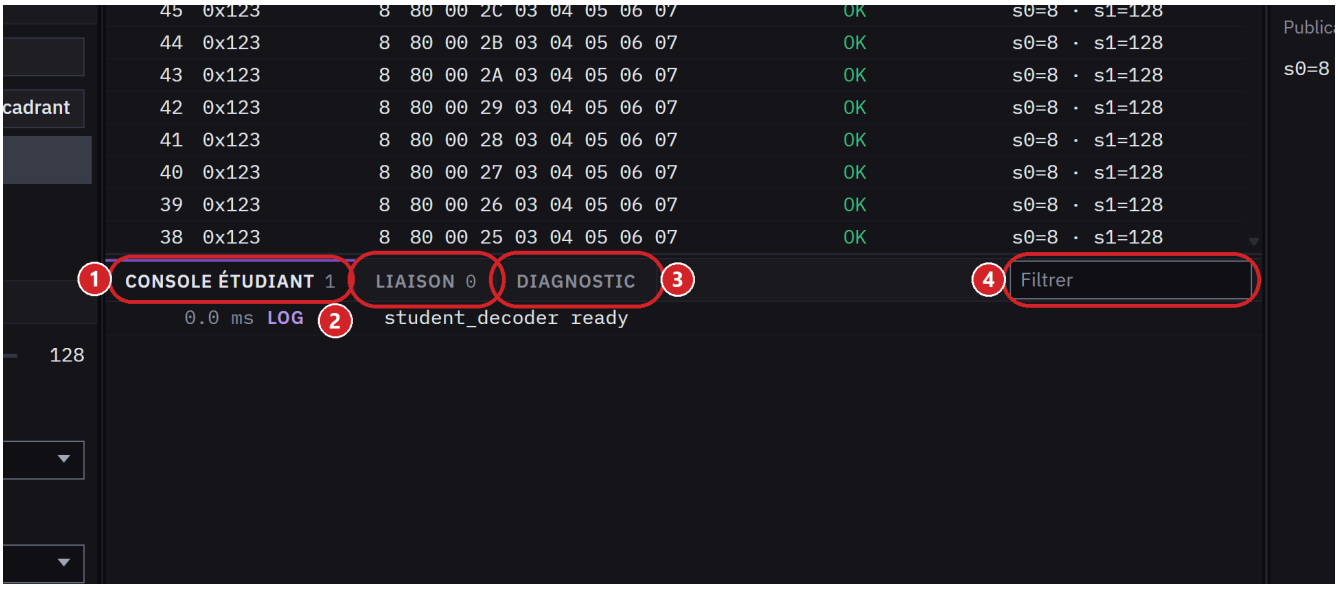


Figure 29. – Les onglets du bas du Cockpit.

Étapes	1. Cliquez **Console étudiant** 1 pour lire les lignes que votre code écrit avec `n7rt_log`. 2. Cliquez **Liaison** 2 ou **Diagnostic** 3 pour contrôler la liaison avec la carte. 3. Tapez un mot dans « Filtrer » 4 : seules les lignes de console qui le contiennent restent affichées.
Ce qu'on doit observer	1 chaque ligne de console porte son instant (temps bus) et son texte ; le nombre à côté du titre compte les lignes reçues (ici 1 : « student_decoder ready », écrite par le gabarit au début de la session). 2 les compteurs « CRC (carte) », « tramage (carte) », « trous de seq (carte) », « débordements UART », « logs perdus » et « erreurs reçues (PC) », puis la liste des incidents de liaison : ils concernent la liaison série entre le PC et la carte, pas la protection des trames de votre mission, et restent normalement à 0. 3 des mesures internes de l'application (files d'attente, délais), utiles à l'encadrant si l'affichage ralentit.
Si cela échoue	Aucune ligne n'arrive : la session n'est pas démarrée (les lignes n'arrivent que pendant une session), votre code n'appelle pas `n7rt_log`, ou il écrit plus de 20 lignes par seconde (l'excédent est perdu et « logs perdus » augmente). Une ligne est limitée à 120 caractères.

Pendant une mission, un quatrième onglet, **Fiche**, s'ajoute à gauche de ces onglets : c'est là que s'ouvre la fiche de l'étape en cours.

9. Le Physical Lab

Le Physical Lab, troisième bouton de la barre d'activité, simule un circuit : en mission, le banc de votre véhicule ; hors mission, un circuit de démonstration ou le vôtre. Les captures de cette section utilisent la démonstration « Référence : potentiomètre, filtre RC et ADC », chargée hors mission avec la liste « Démonstration » et le bouton **Charger** de la rubrique « Fichier ». Les gestes sont les mêmes sur le banc d'une mission.

9.1 Organisation de la vue

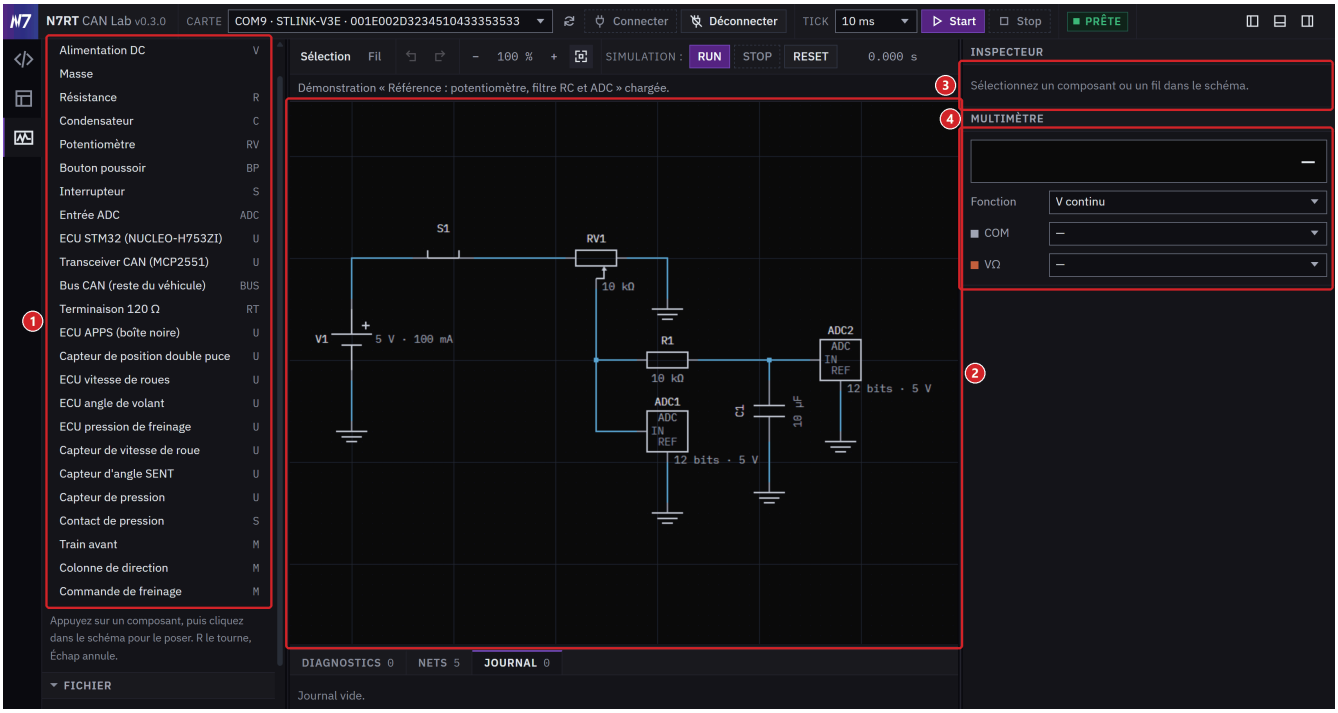


Figure 30. – Les quatre zones du Physical Lab.

Étapes	1. Repérez la bibliothèque de composants 1, le schéma 2, l'inspecteur 3 et le multimètre 4.
Ce qu'on doit observer	1 « Bibliothèque » : les composants à poser (« Appuyez sur un composant, puis cliquez dans le schéma pour le poser. R le tourne, Échap annule. ») ; ne la confondez pas avec la bibliothèque de documents de la mission. Plus bas, la rubrique « Fichier » (**Enregistrer**, **Nouveau**, **Ouvrir**, « Démonstration »). 2 le schéma, sous la barre d'outils (Figure 31) et une ligne de message ; dessous, les onglets « Diagnostics » (problèmes du circuit), « Nets » et « Journal » (événements de la simulation). 3 l'« Inspecteur » : les propriétés de ce qui est sélectionné. 4 le « Multimètre ».
Si cela échoue	Un panneau manque : les boutons en haut à droite de la fenêtre (« Bibliothèque », « Console », « Inspecteur ») affichent ou masquent chaque panneau.

9.2 Barre d'outils et navigation

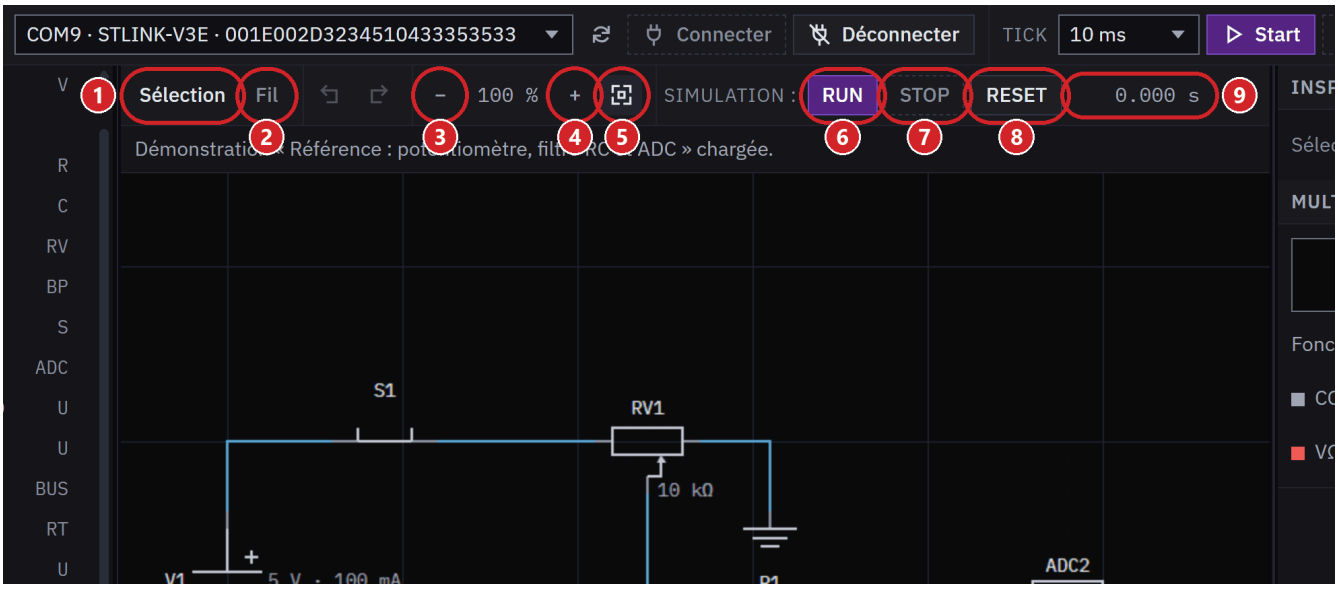


Figure 31. – La barre d'outils du schéma.

Étapes	1. Choisissez l'outil : **Sélection** 1 (touche Échap) pour sélectionner, déplacer et régler ; **Fil** 2 (touche W) pour tracer un fil. 2. Zoomez avec — 3 et + 4 ; le bouton d'ajustement 5 (touche F) cadre tout le schéma. 3. Pilotez la simulation : **RUN** 6 la lance, **STOP** 7 l'arrête, **RESET** 8 la remet à zéro ; le temps simulé s'affiche à droite 9.
Ce qu'on doit observer	Entre **Fil** 2 et — 3, deux flèches annulent et rétablissent la dernière modification (Ctrl+Z, Ctrl+Y). Entre — 3 et + 4, le pourcentage de zoom. Ici, le temps 9 vaut 0 : la simulation n'est pas lancée, et **RUN** 6 est en évidence.
Si cela échoue	Une touche reste sans effet : cliquez d'abord dans le schéma. Un champ de saisie (inspecteur, multimètre) garde les touches pour lui.

Au clavier et à la souris :

- molette : zoom autour du pointeur ; Maj + molette : déplacement latéral ;
- bouton du milieu enfoncé, ou barre d'espace maintenue avec le bouton gauche, puis glisser : déplacer la vue ;
- F : ajuster le schéma à la vue ; 1 : zoom à 100 % ;
- R : tourner la sélection ; Ctrl+A : tout sélectionner ; Ctrl+Z et Ctrl+Y : annuler et rétablir.

RUN, **STOP** et **RESET** agissent sur la simulation du circuit seulement. **RESET** remet le temps à zéro et décharge les condensateurs, sans toucher aux réglages que vous avez faits (curseurs, interrupteurs) ; si la simulation tournait, elle continue.

9.3 Sélectionner un composant et modifier un paramètre



Figure 32. – Un composant sélectionné et son inspecteur.

Étapes	1. Outil **Sélection** actif, cliquez le corps du composant 1 dans le schéma. 2. Lisez son désignateur 2 et ses paramètres 3 dans l'inspecteur.
Ce qu'on doit observer	Le composant 1 est surligné. L'inspecteur titre « R1 · Résistance » ; le champ « Désignateur » 2 porte son nom dans le schéma ; la rubrique « Paramètres » 3 liste ses valeurs avec leur unité. Plus bas : « Brochage » (chaque broche, son net et, hors mission, sa tension et son courant), « Mesures », « État de fonctionnement » et « Sources » (le modèle du composant).
Si cela échoue	L'inspecteur affiche « Sélectionnez un composant ou un fil dans le schéma. » : le clic est tombé à côté du composant. Cliquez au milieu de son corps.

Pour modifier un paramètre :

1. Cliquez sa valeur dans « Paramètres » 3 (Figure 32) : le champ affiche la valeur exacte.
2. Tapez la nouvelle valeur, avec un point décimal ; les préfixes comme k ou m sont acceptés (par exemple `4.7k`). Le survol du champ indique la plage permise.
3. Appuyez sur Entrée. Échap annule la saisie ; quitter le champ la valide aussi.
4. Une valeur refusée est encadrée en rouge et la raison s'affiche sous le tableau ; corrigez-la ou appuyez sur Échap.

Une valeur notée « valeur fixe (catalogue) » ne se modifie pas. Les composants que l'on manipule (potentiomètre, interrupteur, bouton poussoir, alimentation, carte, commandes mécaniques) ont en plus une rubrique « Commandes » : curseur ou champ de position, case à cocher, bouton « Appuyer (maintenir) ».

Attention Sur le banc d'une mission, chaque vérification, essai nominal compris, compare votre banc au banc de référence. Curseurs, interrupteurs et commandes sont libres. Les liaisons et les valeurs de la rubrique « Paramètres » doivent correspondre au banc de référence : ne modifiez une liaison ou un paramètre que si votre diagnostic le justifie, et notez sa valeur d'origine avant de la changer. Ctrl+Z annule une modification, mais l'historique est perdu quand la mission est rouverte.

9.4 Mesurer au multimètre

Le multimètre, sous l'inspecteur, est votre seul instrument de tension en mission : sur le banc d'une mission, les tensions des broches et des nets ne sont pas affichées.

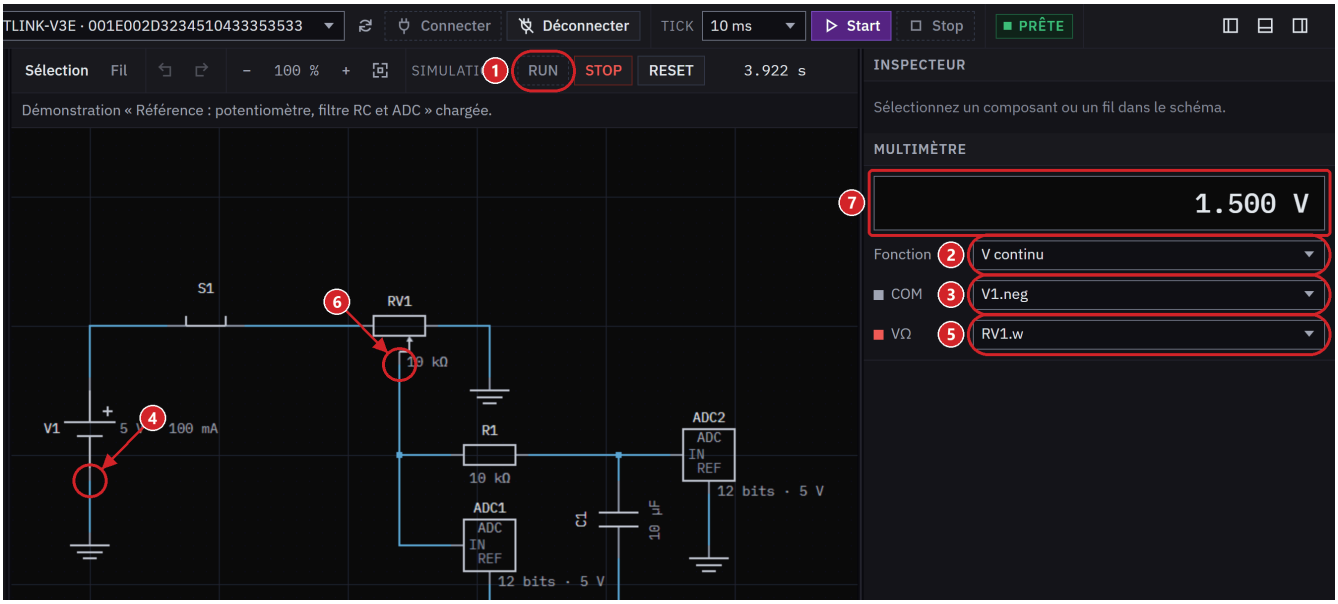


Figure 33. – Une tension mesurée au multimètre sur la démonstration.

Étapes	1. Lancez la simulation avec **RUN** 1 (ici, elle tourne déjà : **STOP** est actif). 2. Dans « Fonction » 2, choisissez « V continu ». 3. Dans « COM » 3 (sonde noire), choisissez la broche de référence, ici la broche négative de l'alimentation 4. 4. Dans « VΩ » 5 (sonde rouge), choisissez la broche mesurée, ici le curseur du potentiomètre 6. 5. Lisez l'afficheur 7.
Ce qu'on doit observer	Les listes 3 et 5 nomment chaque broche « désignateur.broche » (« V1.neg », « RV1.w »). L'afficheur 7 donne la moyenne de la tension entre la sonde rouge et la sonde noire sur les 100 dernières trames de simulation : une ligne numérique active s'y lit à sa valeur moyenne.
Si cela échoue	L'afficheur montre « — » : une sonde n'est pas placée, ou la fonction Ω est choisie sur un banc alimenté.

La fonction « Ω (banc hors tension) » ne mesure que sur un banc éteint, comme un vrai ohmmètre : dans l'inspecteur (rubrique « Commandes »), décochez la sortie de chaque alimentation et l'alimentation USB de la carte ; sinon l'afficheur montre « — ». Rallumez ensuite le banc : banc éteint, l'ECU ne transmet plus, et une consignation ou une vérification porterait sur ce banc éteint. Pendant un incident, un bloc « Mesure clé » s'ajoute sous le multimètre, avec le bouton « Consigner la mesure » (Consigner la mesure).

9.5 Remplacer un fil

Un fil se remplace en deux temps : le supprimer, puis en tracer un nouveau. Les figures suivantes refont le fil qui relie l'alimentation `V1` à l'interrupteur `S1` de la démonstration.

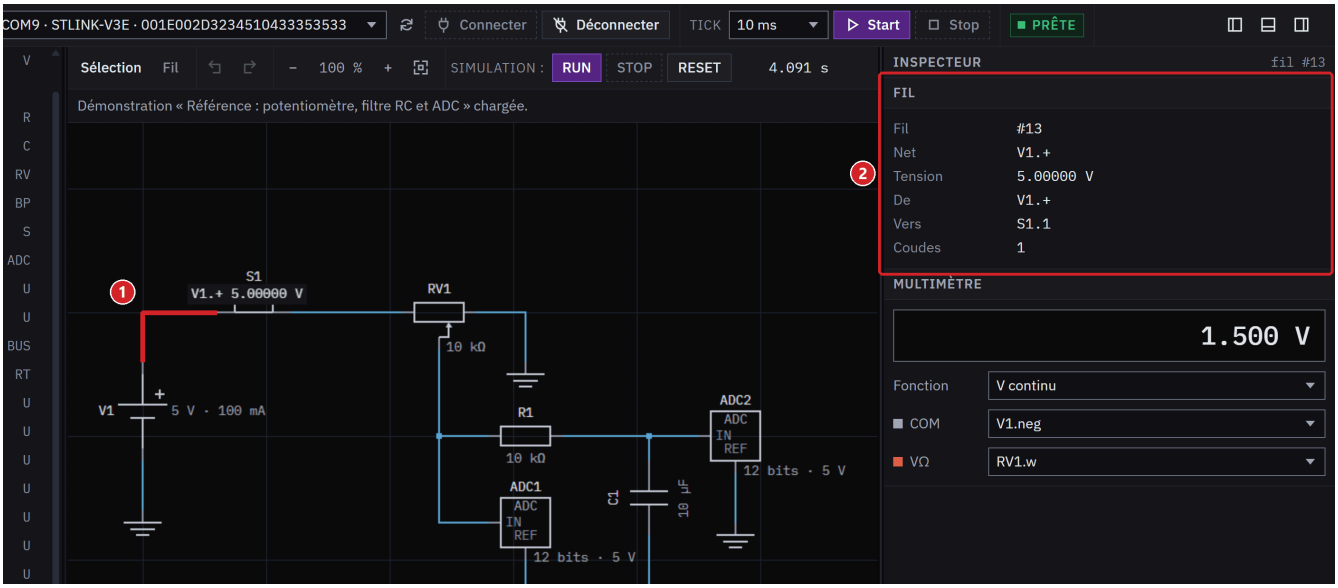


Figure 34. – Un fil sélectionné.

Étapes	1. Outil **Sélection** actif, cliquez le fil 1. 2. Lisez ses extrémités dans l'inspecteur 2, puis appuyez sur la touche Suppr.
Ce qu'on doit observer	Le fil 1 est surligné. L'inspecteur 2 affiche la rubrique « Fil » : son numéro, son net, sa tension (hors mission seulement), ses extrémités (« De », « Vers ») et son nombre de « Coudes ». Notez les deux extrémités avant de le supprimer.
Si cela échoue	La touche Retour arrière ne supprime pas un fil sélectionné : utilisez Suppr. Si le clic sélectionne un composant, cliquez exactement sur le trait du fil.

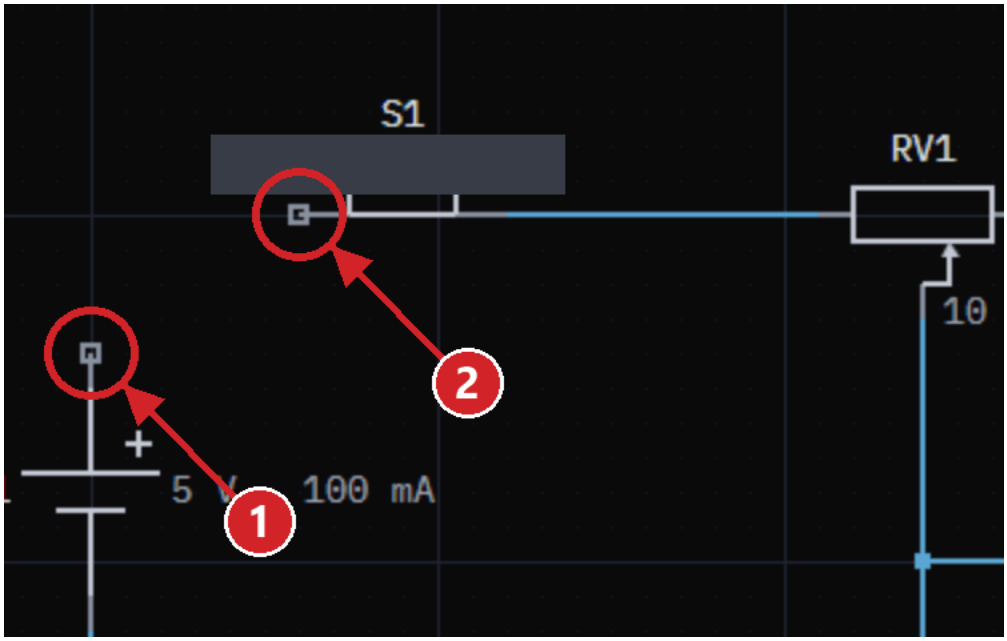


Figure 35. – Le fil supprimé : ses deux broches sont libres.

Étapes	1. Après Suppr, repérez les deux broches que reliait le fil : celle de l'alimentation 1 et celle de l'interrupteur 2.
Ce qu'on doit observer	Le fil a disparu. Une broche libre se dessine comme un petit carré au bout de la patte du composant 1 2.
Si cela échoue	Supprimé par erreur : Ctrl+Z le rétablit.

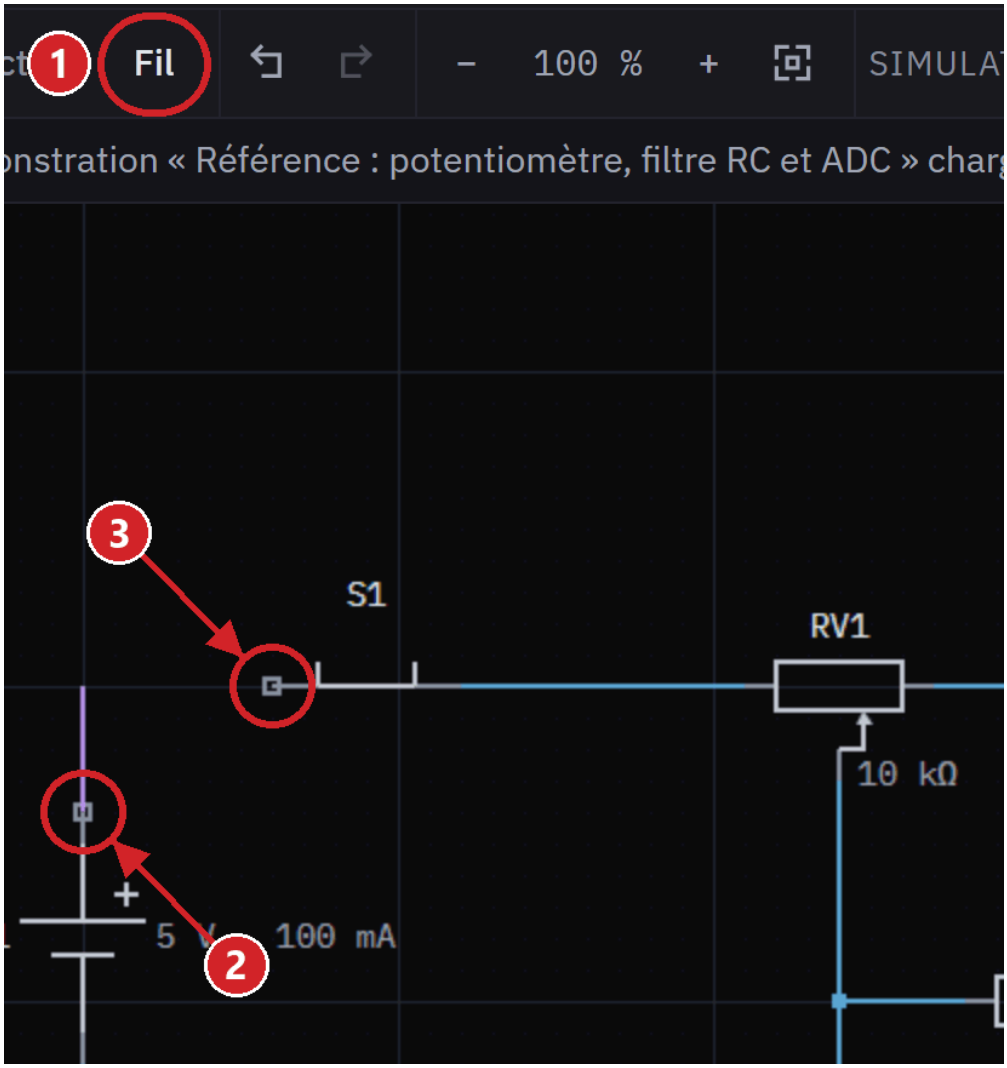


Figure 36. – Un fil en cours de tracé.

Étapes	1. Choisissez l'outil **Fil** 1 (touche W). 2. Cliquez la première broche 2 : un segment suit le pointeur. 3. Pour chaque coude, cliquez dans le vide ; puis cliquez la broche d'arrivée 3.
Ce qu'on doit observer	Le segment part de la broche 2 en équerre : la touche / inverse l'équerre du segment en cours ; Retour arrière retire le dernier segment ; Échap abandonne le tracé. Un clic sur la broche 3 termine le fil.
Si cela échoue	Le fil est refusé et la raison s'affiche au-dessus du schéma, par exemple quand il traverserait le corps d'un composant : recommencez en cliquant un point libre à chaque coude pour le contourner.

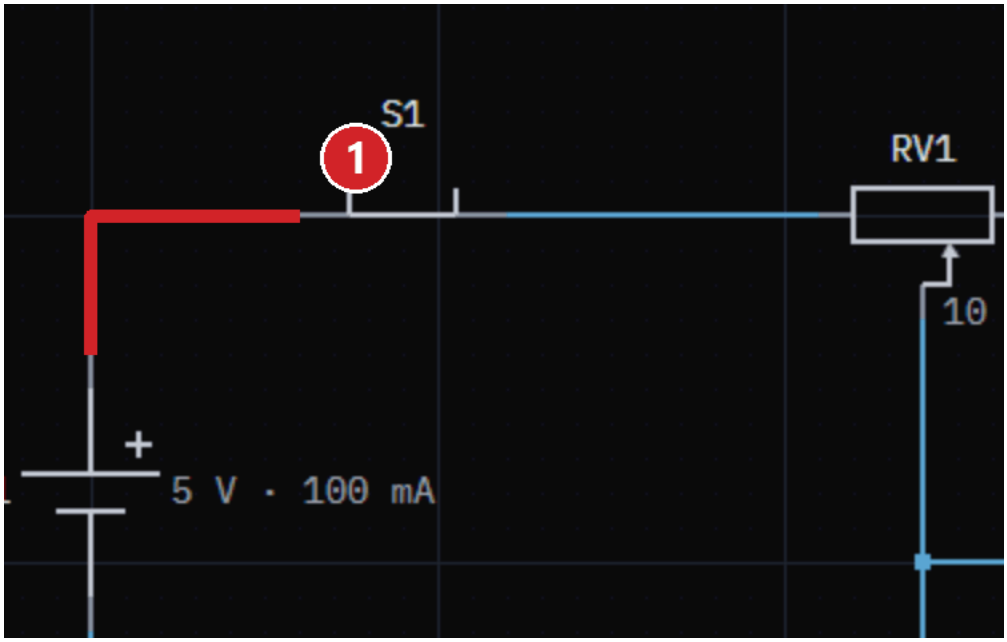


Figure 37. – Le nouveau fil.

Étapes	1. Vérifiez le nouveau fil 1, puis appuyez sur Échap pour revenir à l'outil **Sélection**.
Ce qu'on doit observer	Le fil 1 relie de nouveau les deux broches ; cliquez-le pour lire ses extrémités dans l'inspecteur (Figure 34).
Si cela échoue	Le fil aboutit à la mauvaise broche : sélectionnez-le, Suppr, et retracez-le.

9.6 Revenir au banc de la mission

En mission, le banc de votre véhicule s'ouvre par le bouton « Ouvrir le banc » du panneau de la mission (Cockpit, sous les « Sorties à publier »), par le troisième bouton de la barre d'activité, ou par « Banc Physical Lab » dans la vue **Code STM32**. Au-dessus du schéma, le bandeau « Banc de la mission — mode mission : les grandeurs internes ne sont pas affichées. » le signale : pas de tension aux broches, pas de journal ; mesurez au multimètre.

- Le banc d'une mission ne s'enregistre pas sous un autre nom (« Enregistrement indisponible en mission. ») : il est conservé avec votre progression.
- N'utilisez pas **Nouveau**, **Ouvrir** ni **Charger** pendant une mission : ils remplacent le banc par un autre document, et consigner, reproduire l'anomalie ou vérifier sont alors refusés (« Le banc de la mission n'est plus ouvert dans le Physical Lab »). Pour retrouver votre banc tel que vous l'aviez laissé, cliquez **Fermer** sous la mission (panneau latéral du Cockpit), saisissez le code du groupe, puis **Ouvrir**.
- Pendant une vérification, le banc est en lecture seule (« Banc en lecture seule pendant l'essai. »).

9.7 Consigner la mesure

Pendant un incident (E1 à E3), la fiche impose une grandeur à relever, la **mesure clé**. La consigner, c'est faire enregistrer par l'application la preuve de votre mesure. Le bouton « Consigner la mesure » existe à deux endroits qui font la même chose : dans la fiche, section « Mesure clé », et sous le multimètre, bloc « Mesure clé ». Une fiche d'incident ouverte est enregistrée d'abord.

Conditions : un incident en cours, non validé, hors vérification ; la carte connectée ; une session démarrée (**Start**, barre du haut) ; le bouton **RUN** du Physical Lab ne suffit pas. L'application n'impose pas de durée minimale : elle refuse seulement s'il n'y a encore aucune trame de la session en cours. Attendez pourtant au moins 3 s après **Start** : la consignation scelle les trames des dernières secondes du bus, et une mesure calculée sur trop peu de trames est jugée fausse.

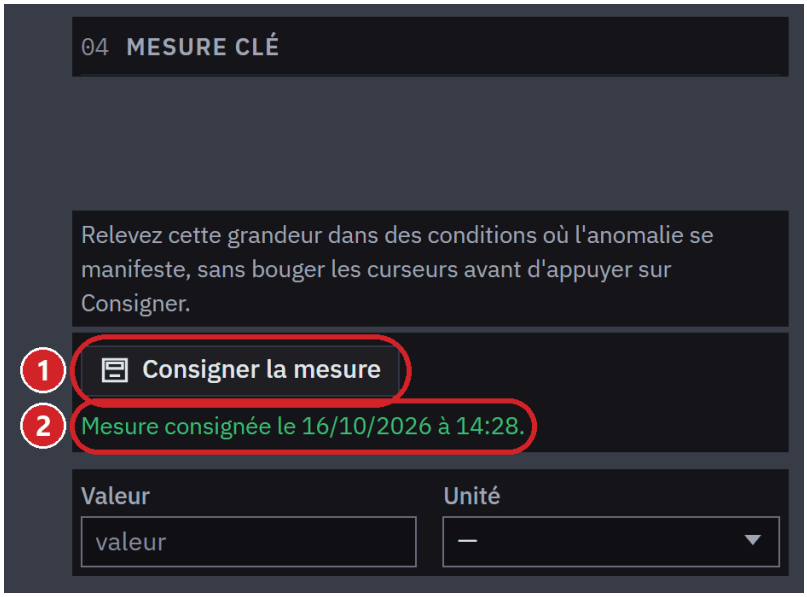


Figure 38. – La mesure clé consignée (contenu de la fiche masqué).

Étapes	1. Pendant que l'anomalie se manifeste, relevez la grandeur imposée, sans toucher aux curseurs. 2. Cliquez « Consigner la mesure » 1. 3. Saisissez la valeur relevée dans « Valeur », choisissez l'unité, puis **Enregistrer la fiche**.
Ce qu'on doit observer	« Mesure consignée le … » 2 donne la date de la consignation (« Mesure non consignée. » tant qu'aucune ne l'est). L'application a enregistré la grandeur imposée et les conditions (états des commandes du banc et, pour une tension, lecture du multimètre), et scellé les trames du bus des dernières secondes et votre circuit : la vérification juge votre valeur sur ces conditions-là.
Si cela échoue	« Aucune trame récente sur le bus : lancez le véhicule, puis consignez la mesure. » : cliquez **Start**, attendez au moins 3 s, consignez de nouveau. « La mesure ne peut être consignée que pendant l'incident en cours, hors vérification. » : attendez la fin de l'essai ; hors d'un incident, il n'y a rien à consigner.

Consignez **avant** de réparer : la mesure doit montrer l'anomalie. Une nouvelle consignation remplace la précédente. Exemple fictif : vous soupçonnez le conducteur R9.p1 – U7.in3 et l'état observé impose une tension ; vous relevez 2,48 V au multimètre pendant que l'anomalie se manifeste, vous consignez, puis vous saisissez `2,48` dans « Valeur » et choisissez `V` dans « Unité ». La valeur s'écrit sans unité, avec une virgule ou un point.

9.8 Reproduire l'anomalie

Vous avez réparé avant d'avoir mesuré, ou votre mesure ne montre plus rien ? Remettez le véhicule dans l'état où il est revenu d'essai.

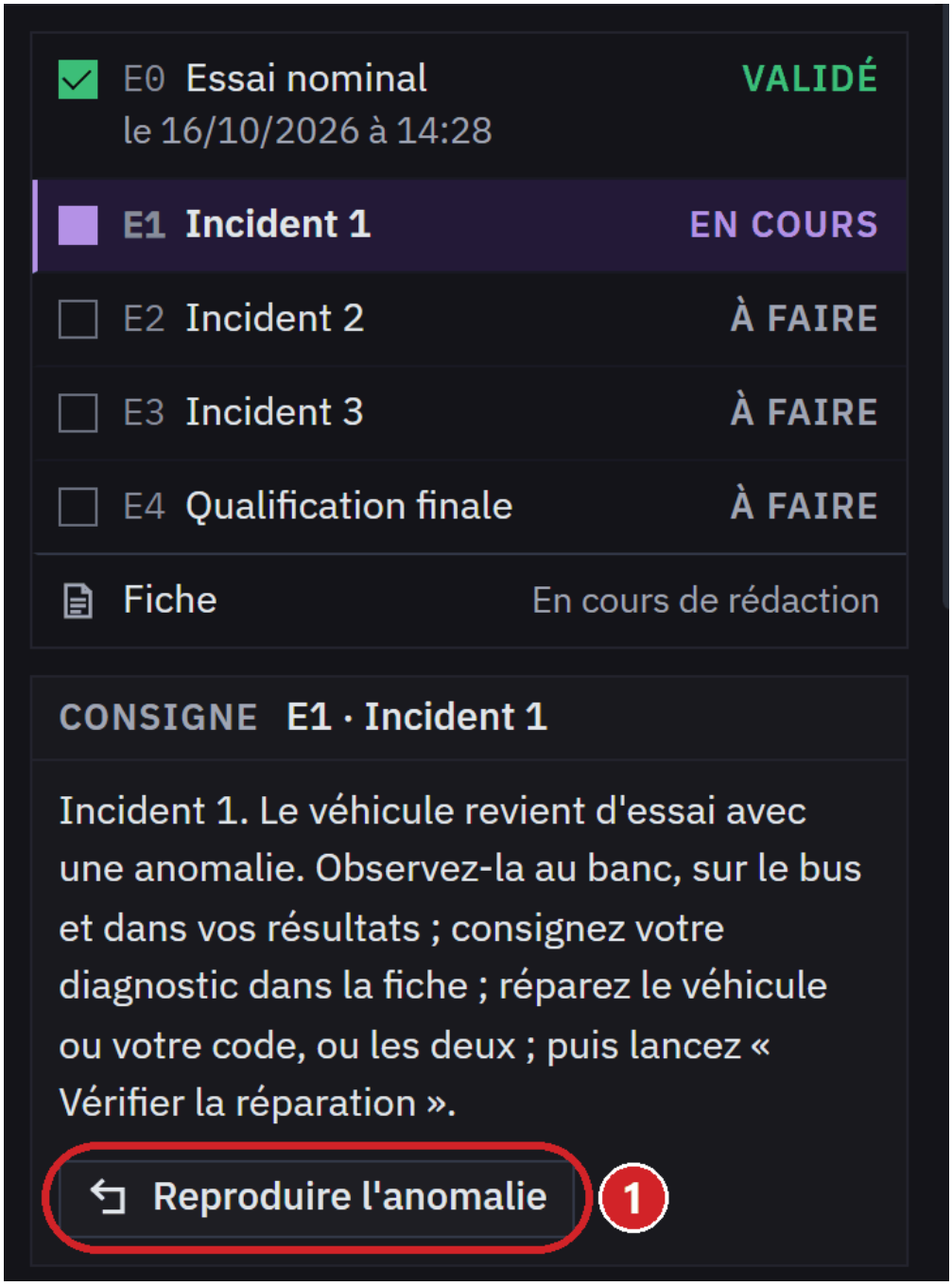


Figure 39. – Le bouton « Reproduire l'anomalie », sous la consigne d'un incident.

Étapes	1. Dans le panneau de la mission (Cockpit), sous la consigne de l'incident, cliquez « Reproduire l'anomalie » 1.
Ce qu'on doit observer	Le bouton 1 n'existe que pendant un incident non validé ; il est grisé pendant une vérification.
Si cela échoue	Un refus s'affiche sous le bouton de vérification (« n'existe que pendant un incident non validé, hors vérification. ») : l'étape en cours n'est pas un incident, ou un essai tourne.

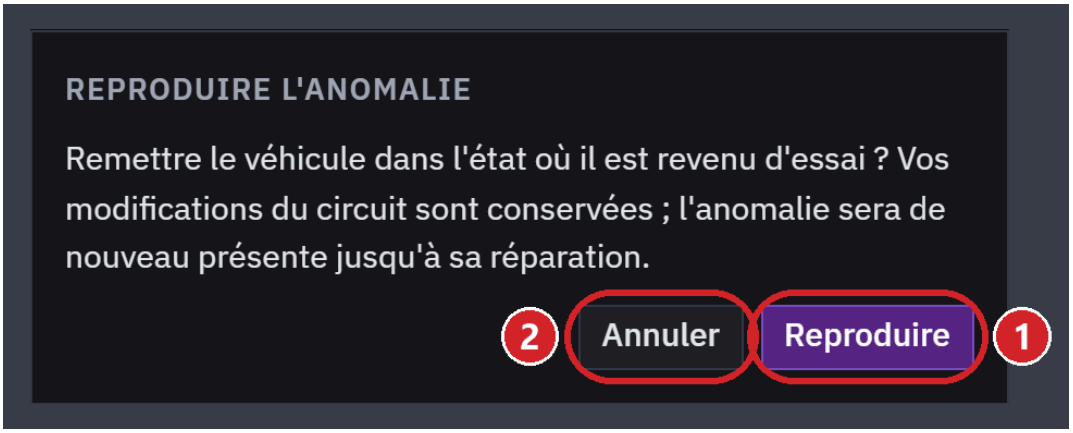


Figure 40. – La confirmation de « Reproduire l'anomalie ».

Étapes	1. Lisez la confirmation, puis cliquez « Reproduire » 1, ou « Annuler » 2 pour ne rien changer.
Ce qu'on doit observer	Après « Reproduire » 1, le véhicule revient dans l'état où il est revenu d'essai ; vos modifications du circuit sont conservées et l'anomalie est de nouveau présente jusqu'à sa réparation. Refaites la mesure, consignez-la, réparez de nouveau, puis vérifiez. Chaque reproduction figure dans la fiche finale.
Si cela échoue	La confirmation ne s'ouvre pas : le bouton était grisé (vérification en cours).

10. Le code STM32

La vue **Code STM32**, premier bouton de la barre d'activité, édite, compile et programme votre décodeur sans STM32CubeIDE. Les captures de cette section montrent le gabarit fourni, reproduit en texte plus loin (Le squelette fourni).

10.1 La page de code

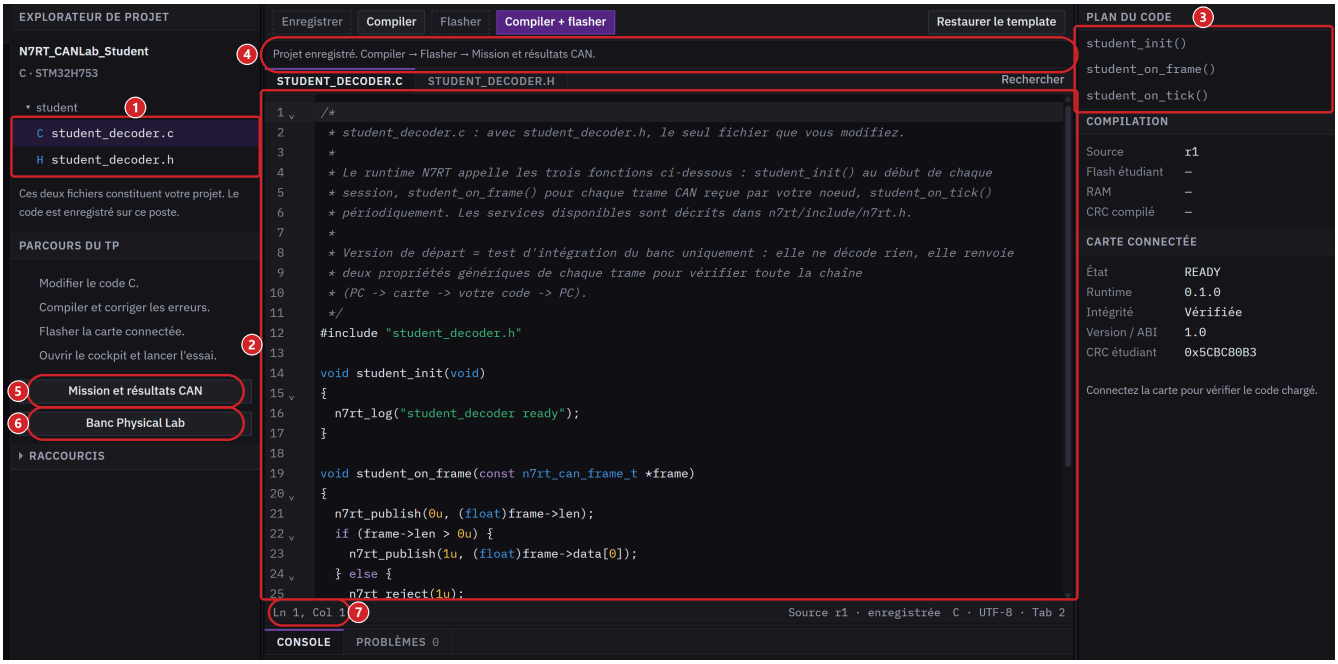


Figure 41. – La page de code, avec le gabarit fourni.

Étapes	1. Dans l'explorateur, cliquez le fichier à modifier (1). 2. Modifiez-le dans l'éditeur (2) ; cliquez une fonction dans « Plan du code » (3) pour aller à sa ligne. 3. Suivez la ligne d'état (4), qui annonce l'étape suivante. 4. Pour revenir au Cockpit ou au banc, cliquez « Mission et résultats CAN » (5) ou « Banc Physical Lab » (6).
Ce qu'on doit observer	(1) les deux fichiers du projet, `student_decoder.c` et `student_decoder.h` ; un point signale un fichier modifié. (3) les trois fonctions du gabarit, `student_init`, `student_on_frame` et `student_on_tick` (Le squelette fourni). (4) ici « Projet enregistré. Compiler → Flasher → Mission et résultats CAN. » (7) la position du curseur (« Ln 1, Col 1 »), suivie, à droite, de la révision de la source et de son état (enregistrée ou modifiée). Sous l'éditeur, les onglets « Console » et « Problèmes ».
Si cela échoue	La page reste sur « Chargement du projet… » ou affiche une erreur : voyez la FAQ de démarrage.

« Parcours du TP », dans l'explorateur, rappelle l'enchaînement : modifier le code, compiler et corriger les erreurs, flasher la carte, ouvrir le Cockpit et lancer l'essai. « Raccourcis » liste les touches de l'éditeur : Ctrl+S enregistre, Ctrl+F recherche, Ctrl+H remplace, Ctrl+Z et Ctrl+Y annulent et rétablissent, Tab indente, Ctrl+Espace propose une complétion des fonctions du SDK. Vos deux fichiers sont enregistrés dans le profil local de l'application, sur ce poste, et non dans le dossier décompressé.

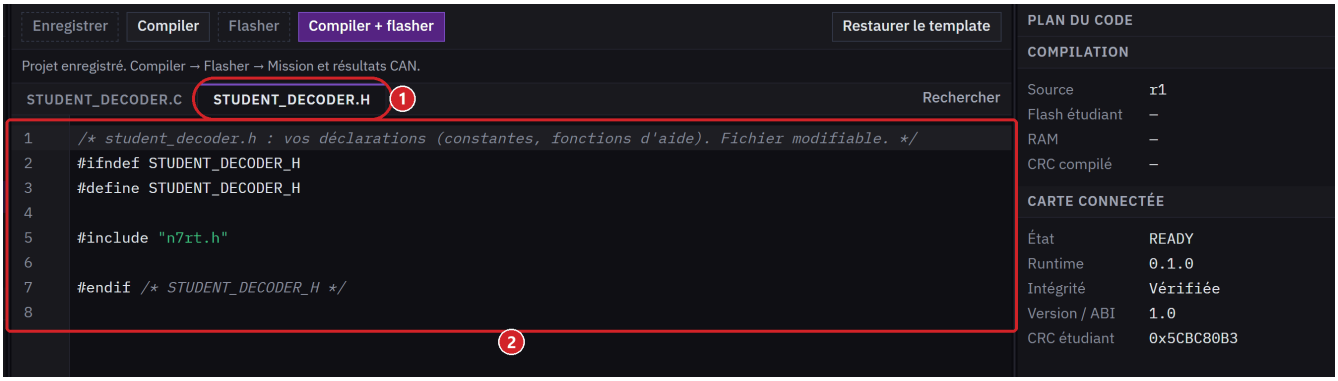


Figure 42. – Le fichier `student_decoder.h` du gabarit.

Étapes	1. Cliquez l'onglet `student_decoder.h` 1 au-dessus de l'éditeur.
Ce qu'on doit observer	L'éditeur 2 montre l'en-tête : vos déclarations (constantes, fonctions d'aide) et l'inclusion de `n7rt.h`, l'API publique du SDK. Le plan du code est vide pour un en-tête sans fonction.
Si cela échoue	Les deux onglets ont un point de modification après un essai malheureux : **Restaurer le template** (Figure 43) revient aux fichiers de départ.

10.2 Les boutons

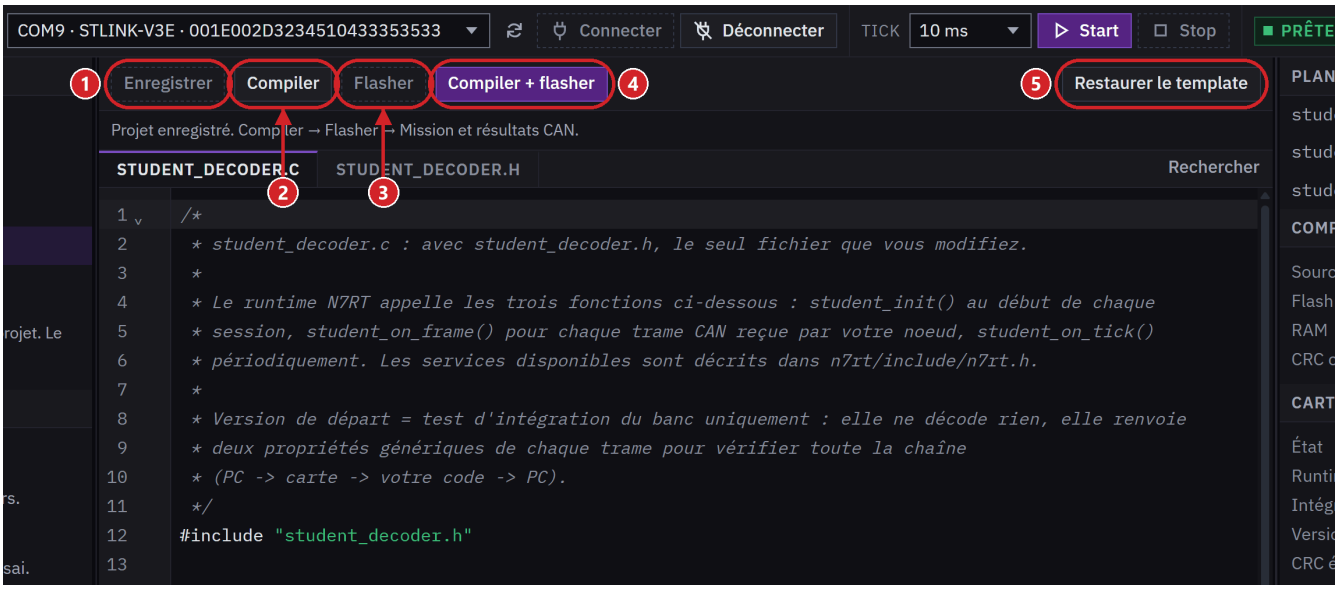


Figure 43. – Les boutons de la page de code.

Étapes	1. **Enregistrer** 1 (Ctrl+S) enregistre les deux fichiers ; il n'est actif qu'après une modification. 2. **Compiler** 2 enregistre, puis compile la source enregistrée. 3. **Flasher** 3 programme la carte connectée avec la dernière compilation. 4. **Compiler + flasher** 4 enchaîne l'enregistrement, la compilation, la programmation et le contrôle de la reconnexion. 5. **Restaurer le template** 5 remplace les deux fichiers par leur version de départ, après confirmation.
Ce qu'on doit observer	Un bouton grisé donne sa raison au survol. Ici, **Enregistrer** 1 et **Flasher** 3 sont grisés : rien n'est modifié, et rien n'a encore été compilé dans cette session.
Si cela échoue	**Flasher** 3 reste grisé : survolez-le (« Compilez les modifications avant de flasher. », « Arrêtez la session et attendez la fin de l'essai avant de flasher. », « Connectez une carte avec un runtime compatible et vérifié. », « La carte doit être prête avant le flash. », « Une opération est déjà en cours. »).

Restaurer le template ouvre la confirmation « Restaurer le template » : « Remplacer les deux fichiers étudiants par leur version de départ ? Le code présent sur la carte ne change pas. » Le bouton « Restaurer les deux fichiers » confirme, « Annuler » renonce. Copiez votre travail ailleurs avant de restaurer, puis recompilez avant de programmer la carte.

10.3 Compiler et corriger une erreur

Compiler écrit d'abord « Compilation de la source enregistrée… » dans l'onglet « Console », puis le résultat. Une compilation réussie se termine par une ligne de la forme `CRC étudiant 0x… · … octets de flash · … octets de RAM`. Une compilation qui échoue affiche `Compilation refusée.` suivi des diagnostics du compilateur.

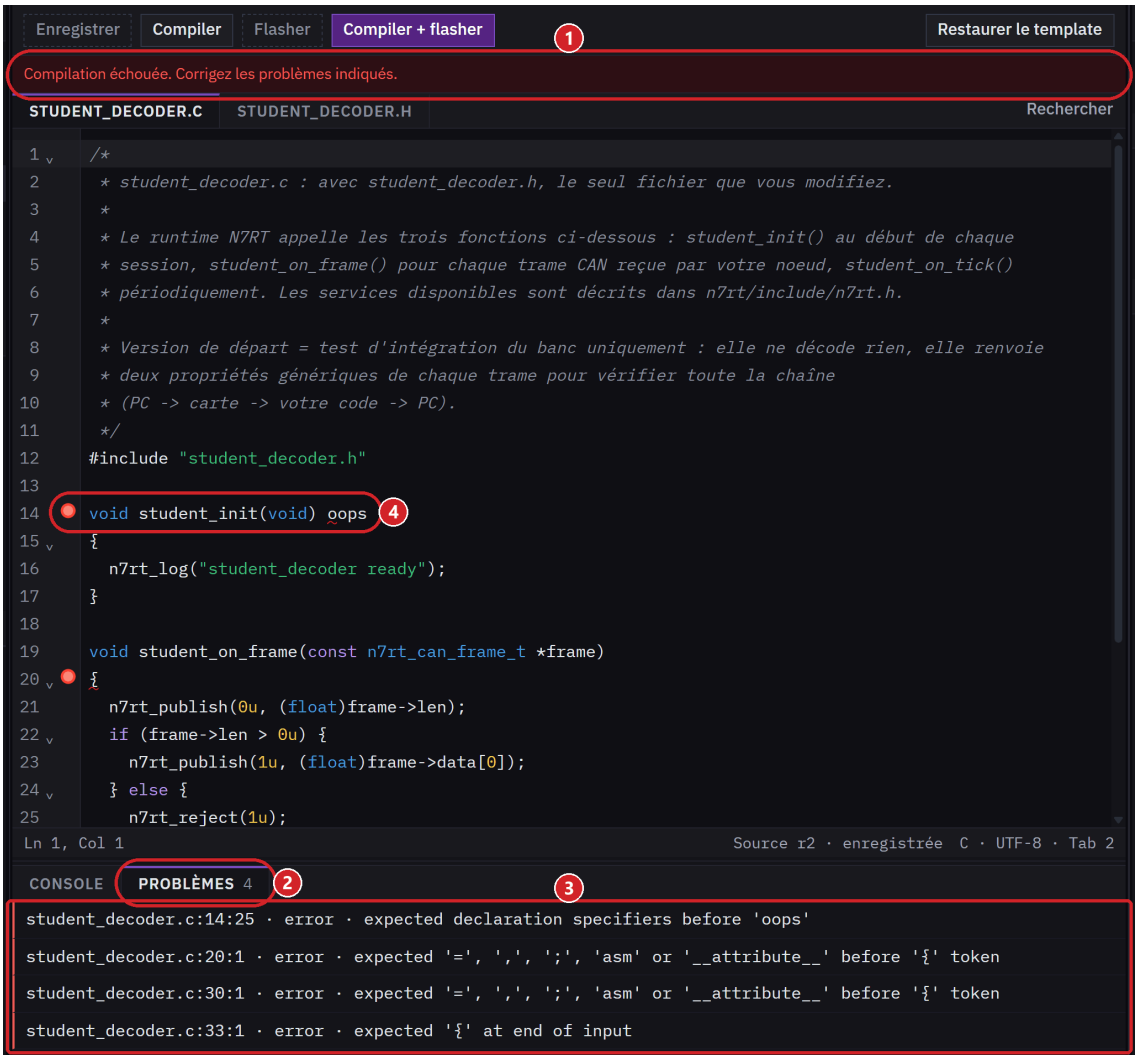


Figure 44. – Une compilation échouée et l'onglet « Problèmes ».

Étapes	1. Lisez la ligne d'état 1. 2. Ouvrez l'onglet « Problèmes » 2 et cliquez le premier diagnostic 3 : l'éditeur va au fichier et à la ligne concernés. 3. Corrigez, enregistrez, recompilez.
Ce qu'on doit observer	1 « Compilation échouée. Corrigez les problèmes indiqués. » 2 le nombre de diagnostics, ici 4. 3 chaque ligne se lit `fichier:ligne:colonne · error · message` (ou `warning`). 4 dans l'éditeur, un point rouge marque chaque ligne en cause et le mot fautif est souligné : ici un mot `oops` ajouté exprès à la ligne 14 du gabarit, qui entraîne aussi les trois erreurs suivantes.
Si cela échoue	Corrigez d'abord la première erreur : les suivantes en découlent souvent. Pour découvrir l'onglet sans risque, ajoutez `#error N7RT_TEST_DIAGNOSTIC` à la fin de `student_decoder.c`, compilez, lisez le diagnostic, puis retirez la ligne.

Une compilation réussie remplit la rubrique « Compilation » du panneau de droite :

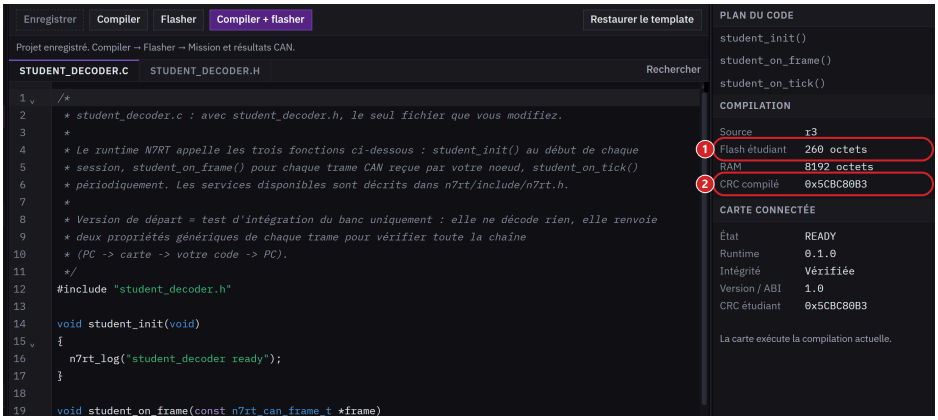


Figure 45. – La rubrique « Compilation » après une compilation réussie.

Étapes	1. Après **Compiler**, lisez la rubrique « Compilation » : « Source », « Flash étudiant » 1, « RAM », « CRC compilé » 2.
Ce qu'on doit observer	« Source » donne la révision compilée (`r3` : numéro de révision de la source). « Flash étudiant » 1 et, juste dessous, « RAM » donnent la taille de votre programme en mémoire. « CRC compilé » 2 est l'empreinte de cette compilation : la carte devra afficher la même après le flash. **Flasher** devient actif.
Si cela échoue	Les champs restent à « — » : la compilation a échoué ; lisez « Problèmes ».

10.4 Flasher la carte et contrôler le CRC

Avant de programmer : carte connectée, runtime vérifié et compatible (état « Prête », « Pas de code étudiant » ou « Code étudiant désactivé »), session arrêtée (**Stop**), aucune vérification en cours. Cliquez **Flasher**, ou **Compiler + flasher**. L'application libère le port série pour le programmateur, écrit seulement la zone étudiante de la mémoire (le runtime N7RT est contrôlé avant et après, et n'est jamais réécrit), puis reconnecte et vérifie la carte.

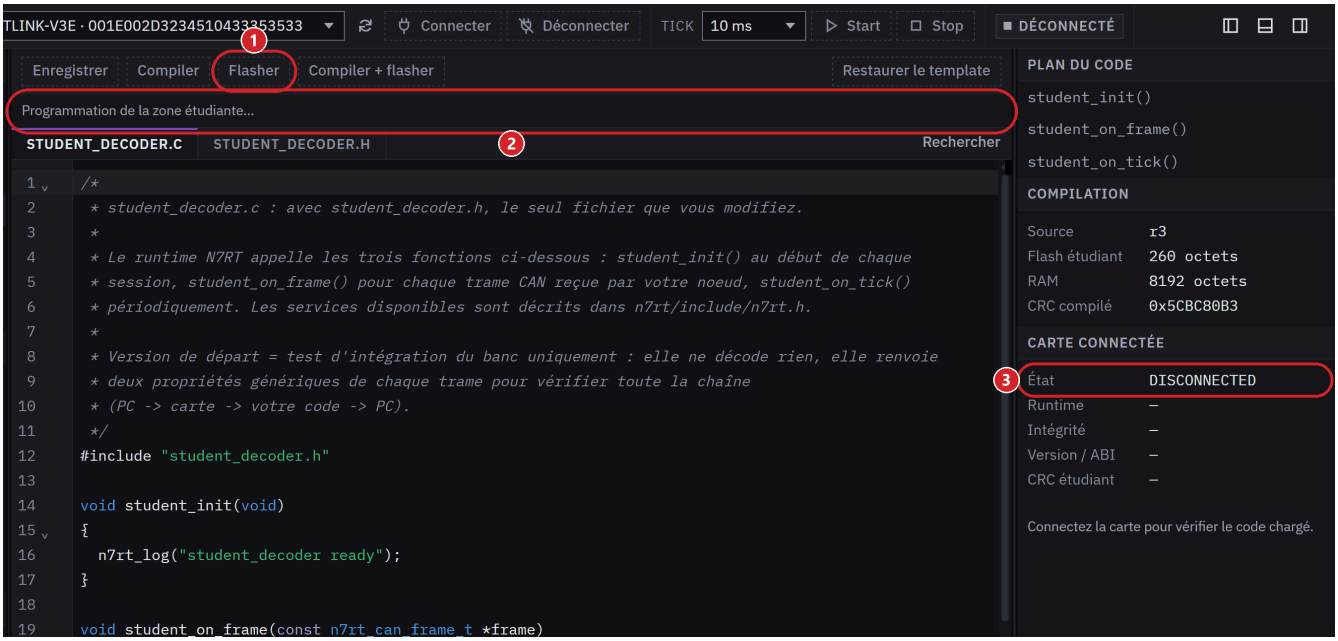


Figure 46. – La programmation en cours.

Étapes	1. Cliquez **Flasher** 1, puis attendez sans débrancher la carte.
Ce qu'on doit observer	La ligne d'état 2 affiche « Programmation de la zone étudiante… », puis « Reconnexion et vérification de la carte… ». Pendant la programmation, l'« État » de la carte 3 passe à `DISCONNECTED` et tous les boutons sont grisés, **Flasher** 1 compris.
Si cela échoue	« Programmation ou vérification échouée. Consultez la console. » : lisez « Console » ; débranchez et rebranchez la carte, connectez-la dans le Cockpit, puis recommencez.

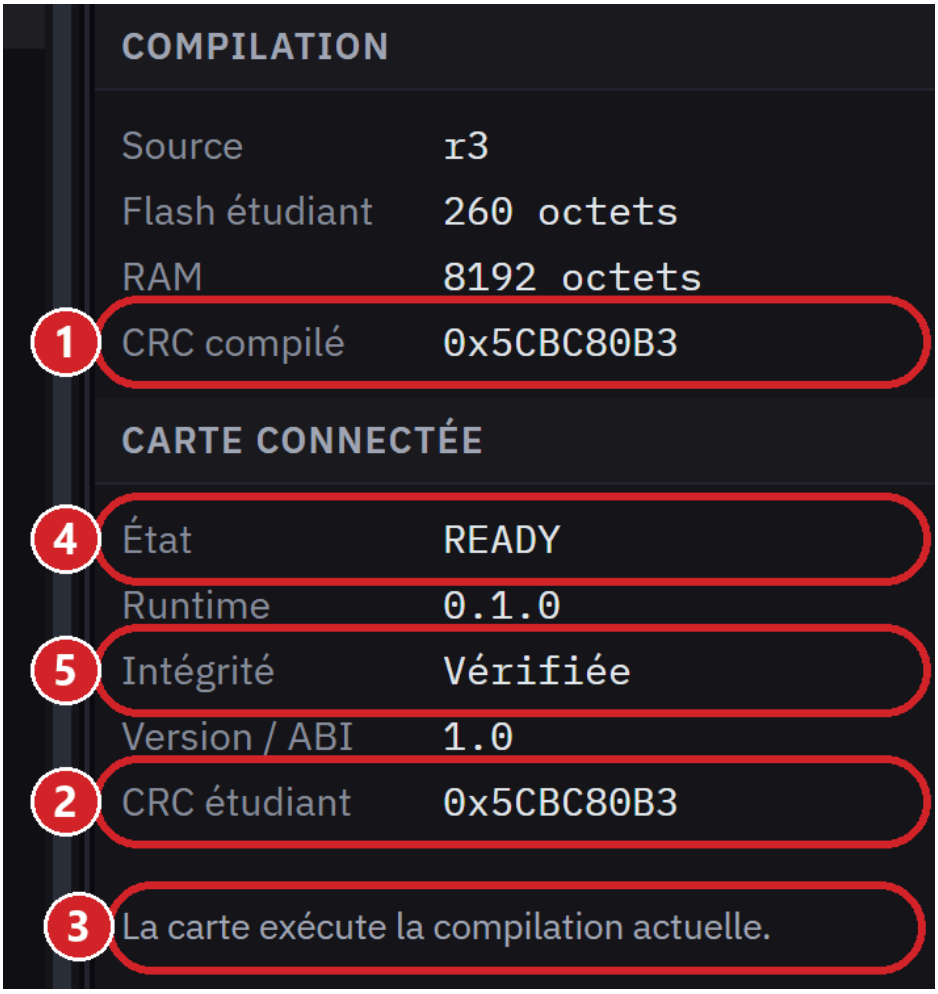


Figure 47. – La carte reconnectée exécute la compilation actuelle.

Étapes	1. Comparez le « CRC compilé » 1 et le « CRC étudiant » lu sur la carte 2. 2. Lisez la phrase de contrôle 3.
Ce qu'on doit observer	4 « État » revient à `READY` ; 5 « Intégrité » indique « Vérifiée » (runtime contrôlé). Les deux CRC 1 2 sont égaux et 3 affiche « La carte exécute la compilation actuelle. »
Si cela échoue	3 affiche « Le code sur la carte diffère de la source actuelle. » : la carte n'exécute pas votre dernière source ; compilez et flashez. « Connectez la carte pour vérifier le code chargé. » : la carte n'est pas connectée.

Le CRC de la zone étudiante, relu sur la carte après chaque programmation, est enregistré avec chaque vérification. Après un redémarrage de l'application, **Compiler** est de nouveau nécessaire pour disposer d'une compilation validée, même si la carte exécute déjà votre programme. Ce CRC identifie votre programme ; il n'a rien à voir avec le CRC qui protège certaines trames de votre mission, que votre code contrôle lui-même (Protection : le principe du CRC).

10.5 Le squelette fourni

Vous ne modifiez que `student_decoder.c` et `student_decoder.h`. Le dossier `n7rt/` du projet (en-tête `n7rt.h`, runtime précompilé, script de liaison) est vérifié à chaque compilation et sur la carte : n'y touchez pas. Voici le gabarit fourni :

```c
/*
 * student_decoder.c : avec student_decoder.h, le seul fichier que vous modifiez.
 *
 * Le runtime N7RT appelle les trois fonctions ci-dessous : student_init() au début de chaque
 * session, student_on_frame() pour chaque trame CAN reçue par votre noeud, student_on_tick()
 * périodiquement. Les services disponibles sont décrits dans n7rt/include/n7rt.h.
 *
 * Version de départ = test d'intégration du banc uniquement : elle ne décode rien, elle renvoie
 * deux propriétés génériques de chaque trame pour vérifier toute la chaîne
 * (PC -> carte -> votre code -> PC).
 */
#include "student_decoder.h"
```

```
13
14 void student_init(void)
15 {
16   n7rt_log("student_decoder ready");
17 }
18
19 void student_on_frame(const n7rt_can_frame_t *frame)
20 {
21   n7rt_publish(0u, (float)frame->len);
22   if (frame->len > 0u) {
23     n7rt_publish(1u, (float)frame->data[0]);
24   } else {
25     n7rt_reject(1u);
26   }
27 }
28
29 void student_on_tick(uint64_t now_us)
30 {
31   (void)now_us;
32 }
```

Ce gabarit est un test d'intégration du banc : il ne décode rien. À chaque trame, il publie sa longueur dans le slot 0 et son premier octet dans le slot 1 ; il ne publie rien aux ticks. Il ne passe donc aucune vérification de mission.

Le runtime appelle trois fonctions, toujours hors interruption et jamais en même temps :

Fonction	Quand, et pour quoi faire
`void student_init(void)`	au démarrage de la carte et au début de chaque session (**Start**) : remettre votre état à zéro, écrire un message de démarrage ;
`void student_on_frame(const n7rt_can_frame_t *frame)`	pour chaque trame reçue par votre nœud : la filtrer, la contrôler, la décoder et mémoriser ce qu'elle apporte ;
`void student_on_tick(uint64_t now_us)`	à chaque tick de la session, `now_us` étant l'instant du tick en µs de temps bus (un multiple de la période du tick) : publier ou rejeter chaque slot de la mission.

La trame reçue et les services du runtime sont déclarés dans `n7rt.h` :

```
25 typedef struct {
26   uint64_t timestamp_us; /* reception time, bus time in us since the session start */
27   uint32_t id;           /* 11-bit identifier, or 29-bit with N7RT_CAN_EXT */
28   uint8_t dlc;           /* DLC code as transmitted on the bus (0..15) */
29   uint8_t len;           /* number of valid bytes in data[] (0..64) */
30   uint8_t flags;         /* N7RT_CAN_* */
31   uint8_t bus;           /* 0 */
32   uint8_t data[64];
33 } n7rt_can_frame_t;      /* 80 bytes, layout frozen */
34
35 _Static_assert(sizeof(n7rt_can_frame_t) == 80, "n7rt_can_frame_t layout is frozen");
36
37 /* Student hooks. */
38 void student_init(void);
39 void student_on_frame(const n7rt_can_frame_t *frame);
40 void student_on_tick(uint64_t now_us);
41
42 /* Runtime services. At most 8 publications per hook call; extra ones are dropped and flagged. */
43 void n7rt_publish(uint8_t slot, float value);
44 void n7rt_reject(uint8_t slot);
45 uint64_t n7rt_now_us(void);
46 void n7rt_log(const char *text); /* up to 120 characters, 20 lines/s */
```

- `timestamp_us` : instant de réception, en µs de temps bus depuis le début de la session ; `id` : identifiant (11 bits, ou 29 bits si le drapeau `N7RT_CAN_EXT` est levé) ; `dlc` : code de longueur transmis ; `len` : nombre d'octets valides dans `data` ; `flags` : drapeaux `N7RT_CAN_EXT` (identifiant étendu), `N7RT_CAN_RTR` (trame de requête), `N7RT_CAN_FD`, `N7RT_CAN_BRS`, `N7RT_CAN_ESI` ; `data` : les octets, `data[0]` en premier. Les trames d'erreur du bus ne sont jamais livrées à votre code.
- `n7rt_publish(slot, valeur)` publie une valeur physique valide pour un slot ; `n7rt_reject(slot)` déclare qu'il n'y a pas de valeur exploitable pour ce slot ; `n7rt_now_us()` rend le temps du bus ; `n7rt_log(texte)` écrit une ligne dans la console (au plus 120 caractères, 20 lignes par seconde).
- Un **slot** est une sortie numérotée définie par l'énoncé (« Sorties à publier ») : numéro, code, nom et unité. Publiez la valeur dans l'unité de l'énoncé. Un appel écrit au plus 8 entrées (publications et rejets confondus) ; au-delà, elles sont perdues (« +débordement » dans le tableau des trames).

À savoir Votre code tourne sur le même microcontrôleur que le runtime. Un appel qui ne rend pas la main (boucle infinie) affiche « Code étudiant occupé » ; au-delà de 2 s, le chien de garde réinitialise la carte. Après

une faute, la carte passe à « Code étudiant désactivé » jusqu'au prochain **Start** ou à une nouvelle image. Le budget de temps par trame est de 200 µs : un dépassement est signalé (« +budget ») sans être sanctionné.

10.6 Les règles de compilation de l'IDE

L'éditeur compile votre code avec la chaîne GNU Arm, en C (GNU11), sans optimisation (`-O0`), avec les avertissements `-Wall -Wextra` (ils s'affichent dans « Problèmes »). Il refuse une source qui sort de ces règles :

- `#include` seulement de `student_decoder.h`, `n7rt.h` ou d'un en-tête C standard : `assert.h`, `ctype.h`, `errno.h`, `float.h`, `inttypes.h`, `limits.h`, `math.h`, `stdarg.h`, `stdbool.h`, `stddef.h`, `stdint.h`, `stdio.h`, `stdlib.h`, `string.h`, `time.h` ;
- `#pragma once` est le seul `#pragma` accepté ; `#include_next`, `#embed` et `#import` sont refusés ;
- pas d'assembleur (`asm`, `__asm__`), ni de `_Pragma`, de `__has_include` ou de collage de symboles `##` ;
- pas de trigraphe (`??`, `%:`) hors des chaînes et des commentaires ;
- chaque fichier est un texte UTF-8 de 256 Kio au plus.

La bibliothèque C fournie n'affiche pas `%f` : écrivez vos valeurs en entiers (par exemple en millièmes). Évitez `malloc` (la réserve de tas est de 8 Kio). Votre programme dispose de 768 Kio de flash et de 128 Kio de RAM ; une image trop grande est refusée à la compilation.

10.7 Décoder une trame : méthodes générales

Le SDK ne fournit aucune fonction de décodage : vous les écrivez. Les fragments qui suivent sont génériques et s'appuient sur une trame **fictive**, qui n'existe dans aucune mission ; vos propres valeurs viennent de votre documentation et de vos mesures.

Élément fictif	Description
Trame	identifiant `0x3C7`, DLC 6, période 50 ms, protégée ;
CRC de protection	INTEL, LSB 40, longueur 8 ;
Compteur de protection	INTEL, LSB 36, longueur 4 ;
Statut	INTEL, LSB 6, longueur 2 ; code « OK » : 2 ;
Signal *température liquide (exemple)*	MOTOROLA, LSB 22, longueur 10, non signé, facteur 0,25, offset −40 (°C), code invalide 1023, slot 7 ;
Supervision	premier timeout 300 ms, timeout 150 ms, péremption 90 ms, 2 pertes consécutives tolérées.

10.7.1 Numéroter les bits

Le bit i de la trame (de 0 à 63) est le bit (i mod 8) de l'octet $\lfloor i/8 \rfloor$; l'octet 0 est transmis en premier et le bit 0 est le poids faible de l'octet. Le **LSB** d'un champ est la position de son bit de poids faible. En INTEL, les bits suivants du champ sont i+1, i+2… ; en MOTOROLA, ils remontent l'octet, puis continuent au bit 0 de l'octet précédent. Le tableau ci-dessous place les champs de la trame fictive :

Octet	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0	7 **S**	6 **S**	5	4	3	2	1	0
1	15 **T**	14 **T**	13 **T**	12 **T**	11 **T**	10 **T**	9 **T**	8 **T**
2	23 **T**	22 **T**	21	20	19	18	17	16
3	31	30	29	28	27	26	25	24
4	39 **K**	38 **K**	37 **K**	36 **K**	35	34	33	32
5	47 **C**	46 **C**	45 **C**	44 **C**	43 **C**	42 **C**	41 **C**	40 **C**

Tableau 1. – Position des bits de la trame fictive `0x3C7` (S : statut, T : signal *température liquide (exemple)*, K : compteur, C : CRC ; case blanche : bit inutilisé).

Lecture du signal fictif : MOTOROLA, LSB 22, longueur 10. Ses deux bits de poids faible sont les bits 22 et 23 (bits 6 et 7 de l'octet 2) ; il continue au bit 0 de l'octet précédent et prend tout l'octet 1 (bits 8 à 15). Sa valeur brute vaut donc `(data[1] << 2) | (data[2] >> 6)`. Le compteur fictif (INTEL, LSB 36, longueur 4) est le quartet haut de l'octet 4, `data[4] >> 4` ; le CRC fictif (INTEL, LSB 40, longueur 8) est l'octet 5 entier.

10.7.2 Lire un champ INTEL ou MOTOROLA

Deux fonctions générales suffisent pour tous les champs ; vérifiez d'abord que la trame contient assez d'octets (`frame->len`), pour ne jamais lire au-delà des octets reçus.

```c
#include <stdint.h>

/* Champ INTEL : lsb = position du bit de poids faible (0 à 63), len = longueur (1 à 32). */
static uint32_t lire_intel(const uint8_t *d, unsigned lsb, unsigned len)
{
  uint32_t v = 0u;
  for (unsigned i = 0u; i < len; i++) {
    unsigned bit = lsb + i;                    /* INTEL : lsb, lsb + 1, lsb + 2... */
    v |= (uint32_t)((d[bit / 8u] >> (bit % 8u)) & 1u) << i;
  }
  return v;
}

/* Champ MOTOROLA repéré par son bit de poids faible. */
static uint32_t lire_motorola(const uint8_t *d, unsigned lsb, unsigned len)
{
  uint32_t v = 0u;
  unsigned bit = lsb;
  for (unsigned i = 0u; i < len; i++) {
    v |= (uint32_t)((d[bit / 8u] >> (bit % 8u)) & 1u) << i;
    if (bit % 8u == 7u) {
      bit -= 15u;                              /* fin de l'octet : bit 0 de l'octet précédent */
    } else {
      bit++;                                   /* bit suivant du même octet */
    }
  }
  return v;
}
```

Sur la trame fictive :

- `lire_motorola(frame->data, 22u, 10u)` rend la valeur brute du signal ;
- `lire_intel(frame->data, 36u, 4u)` rend la valeur du compteur.

10.7.3 Signe : le complément à deux

Un champ déclaré non signé se lit tel quel. Un champ en complément à deux est négatif quand son bit de poids fort vaut 1 : on lui retire alors 2^n, n étant sa longueur. Sur 10 bits, la valeur brute 1023 vaut -1 et la valeur brute 512 vaut -512.

```c
/* Complément à deux sur len bits (1 à 31). */
static int32_t etendre_signe(uint32_t brut, unsigned len)
{
  if ((brut & (1u << (len - 1u))) != 0u) {
    return (int32_t)((int64_t)brut - ((int64_t)1 << len));
  }
  return (int32_t)brut;
}
```

10.7.4 Facteur, offset et code invalide

La valeur physique vaut **valeur brute × facteur + offset**. Avec le signal fictif (facteur 0,25, offset −40 °C), la valeur brute 300 donne $300 \times 0{,}25 - 40 = 35$ °C. Un **code invalide** (ou SNA) est une valeur brute réservée par la source pour dire *pas de valeur* : elle ne se convertit pas. Ce que votre décodeur en fait, rejeter ou publier, est votre décision de conception DD-1, tirée de l'énoncé et du règlement.

```c
#include <stdbool.h>

#define SLOT_TEMP      7u        /* slot fictif du signal d'exemple */
#define TEMP_INVALIDE  1023u     /* code invalide fictif */

static float temp_c;             /* dernière valeur physique décodée */
static bool  temp_code_invalide; /* true : la source a envoyé le code invalide */

/* Dans student_on_frame, une fois la trame acceptée : */
static void decoder_temperature(const uint8_t *d)
{
  uint32_t brut = lire_motorola(d, 22u, 10u);   /* MOTOROLA, LSB 22, longueur 10 */
  temp_code_invalide = (brut == TEMP_INVALIDE);  /* la conduite à tenir : votre décision DD-1 */
  if (!temp_code_invalide) {
    temp_c = (float)brut * 0.25f - 40.0f;       /* valeur brute x facteur + offset */
  }
}
```

10.7.5 Statut

Un champ de statut dit si la source considère sa donnée comme bonne. Avec le statut fictif (INTEL, LSB 6, longueur 2, code « OK » : 2), `lire_intel(d, 6u, 2u) == 2u` signifie que la source n'annonce aucun défaut. La conduite à tenir pour un autre code vient de l'énoncé.

10.7.6 Protection : le principe du CRC

L'émetteur d'une trame protégée calcule un CRC sur les octets de la trame et le place dans le champ CRC. Le récepteur refait le même calcul sur les mêmes octets et compare : si les deux diffèrent, la trame n'est pas conforme (votre décision DD-2 dit ce que vous faites de ses données). Le calcul exact (polynôme, valeur de départ, octets couverts, valeur finale et tout ce que la spécification fait entrer dans le calcul) est défini par la spécification de protection que cite votre mission : cherchez-la dans la bibliothèque.

```c
/* Principe seulement : crc_selon_spec() est à écrire d'après la spécification de protection. */
uint8_t crc_selon_spec(const uint8_t *d, uint8_t len);

static bool protection_conforme(const uint8_t *d, uint8_t len)
{
  uint8_t recu = (uint8_t)lire_intel(d, 40u, 8u);    /* CRC fictif : INTEL, LSB 40, longueur 8 */
  return recu == crc_selon_spec(d, len);
}
```

10.7.7 Compteur et continuité

Un compteur de protection augmente d'une trame à la suivante et revient à 0 après sa valeur maximale (15 pour 4 bits). L'écart entre deux trames reçues se calcule modulo 2^n : un écart nul signale une trame répétée, un écart supérieur à 1 des trames manquantes. L'écart toléré vient de la spécification de protection.

```c
/* Écart entre deux valeurs d'un compteur de 4 bits (0 à 15, puis retour à 0). */
static unsigned ecart_compteur(unsigned precedent, unsigned actuel)
{
  return (actuel - precedent) & 0x0Fu;
}
```

10.7.8 Fraîcheur et timeout

Le temps se mesure avec le temps du bus : `frame->timestamp_us` dans `student_on_frame`, `now_us` dans `student_on_tick`. Mémorisez l'instant de la dernière trame valide, puis calculez son âge à chaque tick. Les seuils de la fiche (premier timeout avant la première trame valide de la session, timeout entre deux trames valides, péremption, âge au-delà duquel une donnée reçue n'est plus considérée comme fraîche, pertes consécutives tolérées) et la conduite à tenir viennent de votre énoncé ; le fragment ci-dessous ne montre que la mesure d'un âge, avec la péremption fictive.

```c
#define PEREMPTION_US  90000u    /* péremption fictive : 90 ms */

static uint64_t derniere_us;     /* temps bus de la dernière trame valide */
static bool     deja_recue;      /* au moins une trame valide depuis le début de la session */

/* Dans student_on_frame, pour une trame que vos décisions retiennent : */
static void noter_reception(const n7rt_can_frame_t *frame)
{
  derniere_us = frame->timestamp_us;
  deja_recue = true;
}

/* Dans student_on_tick : l'âge de la dernière trame est-il dans la péremption ? */
static bool donnee_fraiche(uint64_t now_us)
{
  return deja_recue && (now_us - derniere_us) <= PEREMPTION_US;
}
```

10.7.9 Quand publier, quand rejeter

Les vérifications comparent, tick après tick, ce que votre carte publie dans `student_on_tick` à ce qu'attend l'essai caché. Répartissez donc le travail ainsi :

1. dans `student_on_frame` : ignorez les trames qui ne vous concernent pas (autre identifiant, trame étendue `N7RT_CAN_EXT` ou de requête `N7RT_CAN_RTR`, trop courte) ; contrôlez la protection, le compteur et le statut ; décodez ; mémorisez la valeur, sa validité et l'instant de réception ;
2. dans `student_on_tick` : écrivez **chaque** slot de la mission, une fois par tick : `n7rt_publish` ou `n7rt_reject`, selon l'état de la donnée et vos décisions DD-1 à DD-4 ;
3. ne publiez pas les sorties de la mission dans `student_on_frame` : la vérification ne les lit pas. Pour déboguer, écrivez plutôt dans la console avec `n7rt_log`.

```c
void student_on_frame(const n7rt_can_frame_t *frame)
{
  if (frame->id != 0x3C7u || (frame->flags & (N7RT_CAN_EXT | N7RT_CAN_RTR)) != 0u) {
    return;                     /* pas la trame attendue */
  }
  if (frame->len < 6u) {
    return;                     /* jamais lire au-delà des octets reçus */
  }
  bool conforme = protection_conforme(frame->data, frame->len);
  /* compteur et statut ici ; trame non conforme : appliquez votre DD-2 */
  memoriser(frame, conforme);    /* à écrire d'après votre fiche : décodage, validité, instant */
}

void student_on_tick(uint64_t now_us)
{
  if (publiable(now_us)) {       /* à écrire : état mémorisé, DD-1 à DD-4, supervision */
    n7rt_publish(SLOT_TEMP, temp_c);   /* une valeur physique, dans l'unité de l'énoncé */
  } else {
    n7rt_reject(SLOT_TEMP);
  }
  /* ... une entrée, et une seule, pour chacun des autres slots de la mission ... */
}
```

Ce squelette ne tranche aucune question de la mission : quelle trame accepter quand sa longueur est inattendue, que faire d'une protection non conforme, d'un compteur discontinu ou d'un statut en défaut, combien de temps garder une valeur. Ces choix figurent dans votre fiche (décisions DD-1 à DD-4, supervision) : les décisions se justifient par l'énoncé, le règlement et les spécifications de la bibliothèque ; les valeurs de supervision sont celles que votre encadrant vous a communiquées au lancement de l'E0.

10.7.10 Déboguer avec `n7rt_log`

`n7rt_log` écrit au plus 120 caractères par ligne et 20 lignes par seconde ; au-delà, les lignes sont perdues. Écrivez un décompte ou une moyenne par bloc de trames, jamais une ligne par trame, et des entiers plutôt que des nombres à virgule.

```c
#include <stdio.h>

static unsigned nb_trames;

/* Dans student_on_frame, une fois la valeur brute lue : */
static void journaliser(uint32_t brut)
{
  nb_trames++;
  if (nb_trames == 100u) {              /* une ligne par bloc de 100 trames */
    char ligne[64];
    snprintf(ligne, sizeof ligne, "temp brute : %lu", (unsigned long)brut);
    n7rt_log(ligne);
    nb_trames = 0u;
  }
}
```

11. Vérifier une étape

Chaque étape se termine par une vérification que l'application conduit seule : votre fiche contre la vérité scellée dans votre pack, votre banc contre le banc de référence, votre carte contre un essai caché. Le texte libre des justifications n'entre jamais dans un verdict.

11.1 Quand le bouton est actif

Le bouton de vérification se trouve sous la consigne, dans le panneau de la mission (Cockpit). Son libellé dépend de l'étape : « Lancer la vérification » (E0), « Vérifier la réparation » (E1 à E3), « Lancer la qualification » (E4).

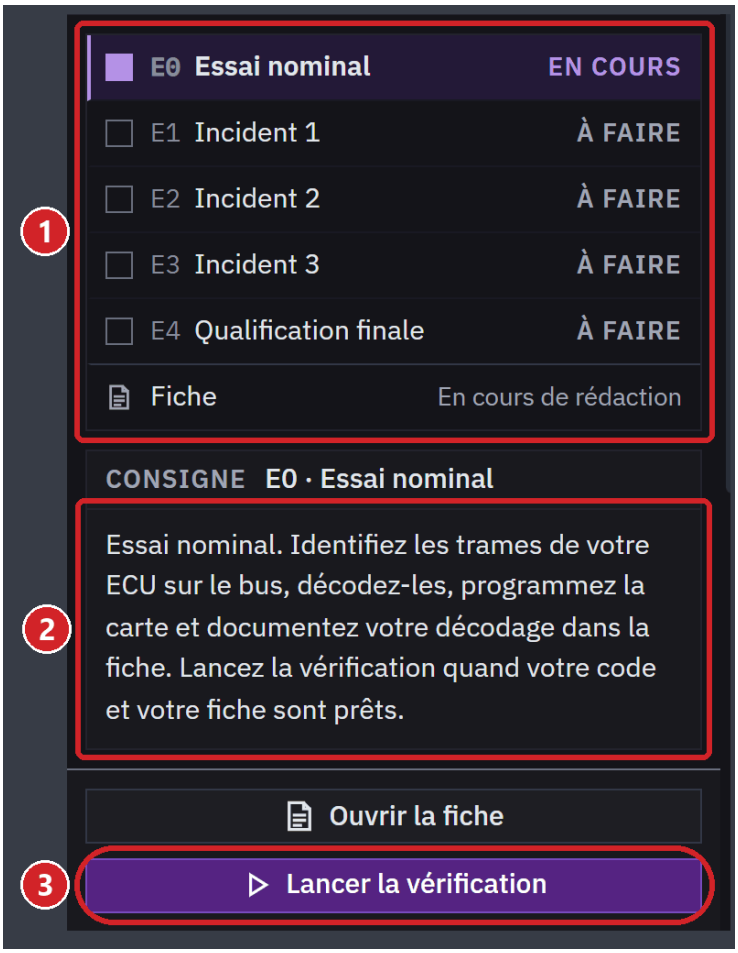


Figure 48. – Le bouton de vérification de l'essai nominal.

Étapes	1. Repérez l'étape en cours (1) et relisez sa consigne (2). 2. Quand votre code est programmé et votre fiche prête, cliquez « Lancer la vérification » (3).
Ce qu'on doit observer	(1) l'étape en cours est « EN COURS » ; la dernière ligne donne l'état de la fiche. (2) la consigne de l'étape. (3) le bouton, actif ici. Une fiche ouverte est enregistrée avant la vérification (« La fiche sera enregistrée avant la vérification. »).
Si cela échoue	Le bouton est grisé : le message affiché dessous dit pourquoi (liste ci-dessous).

Quand le bouton est grisé, le message affiché dessous en donne la raison :

Message	Que faire
« Complétez la fiche de cette étape avant la vérification. Champs à renseigner : … »	Renseignez les champs listés, puis enregistrez : la liste porte sur la fiche **enregistrée**.
« Corrigez les champs signalés dans la fiche. »	Un champ est mal formé (encadré en rouge dans la fiche) : corrigez-le.
« Connectez la carte et vérifiez que votre programme est chargé. »	Branchez et connectez la carte, attendez « Prête », programmez-la si besoin.
« Prochaine vérification possible dans … min … s. »	Patientez : 2 minutes entre deux vérifications pour E0 à E3, 5 minutes pour E4. Reprenez vos mesures en attendant.

Message	Que faire
« Vérification déjà en cours. »	Attendez le verdict.

11.2 Pendant l'essai

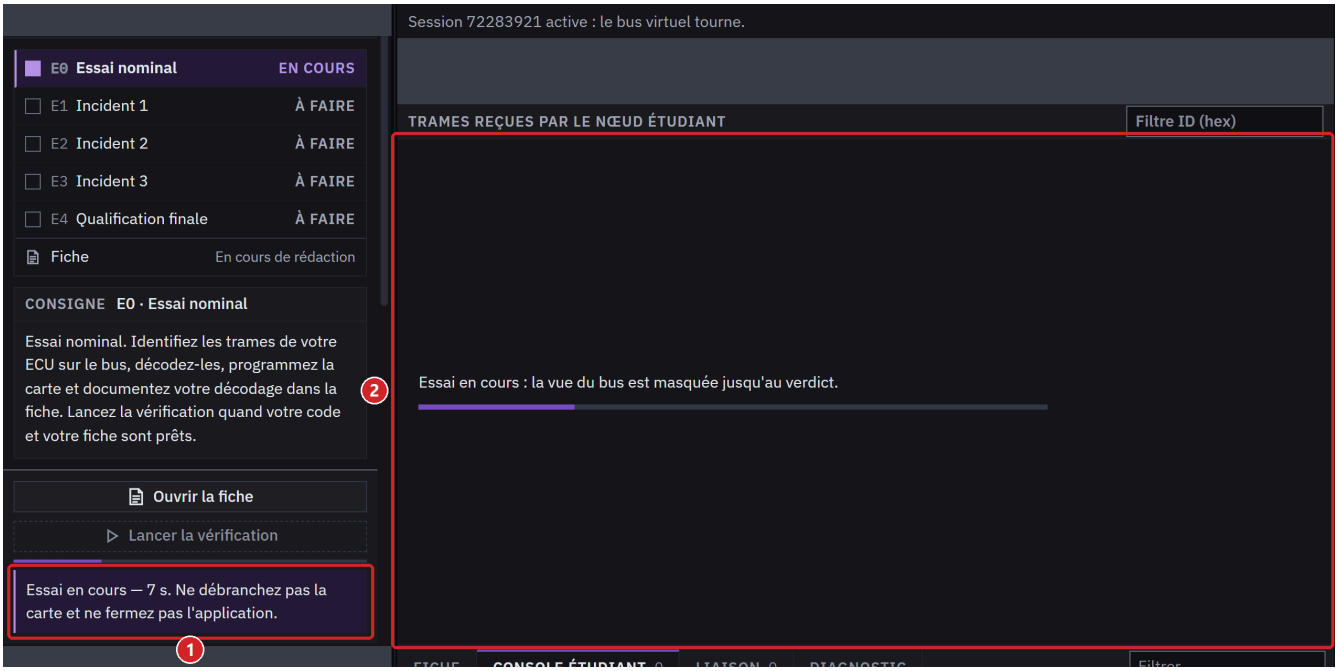


Figure 49. – Un essai en cours : bus et résultats masqués.

Étapes	1. Ne touchez à rien : ne débranchez pas la carte, ne fermez pas l'application, attendez le verdict.
Ce qu'on doit observer	1 le temps restant (« Essai en cours — 7 s. Ne débranchez pas la carte et ne fermez pas l'application. »), sous une barre de progression. 2 la vue du bus est masquée (« Essai en cours : la vue du bus est masquée jusqu'au verdict. ») ; les résultats, à droite, aussi.
Si cela échoue	Carte débranchée ou application fermée : « Vérification interrompue (…). Elle n'est pas comptée. Relancez quand le banc est prêt. » Rebranchez la carte ou rouvrez l'application, puis relancez la vérification.

Pendant l'essai, la carte et la session sont réservées (**Start**, **Stop**, **Connecter** et le choix du port sont grisés : « Vérification en cours : la carte et la session sont réservées à l'essai. »), la fiche est en lecture seule (« Vérification en cours : la fiche n'est pas modifiable. ») et le banc aussi. Une interruption n'est pas comptée ; au-delà de six interruptions d'une même étape dans l'heure, une interruption déclenche aussi le délai entre deux vérifications.

11.3 Lire le verdict

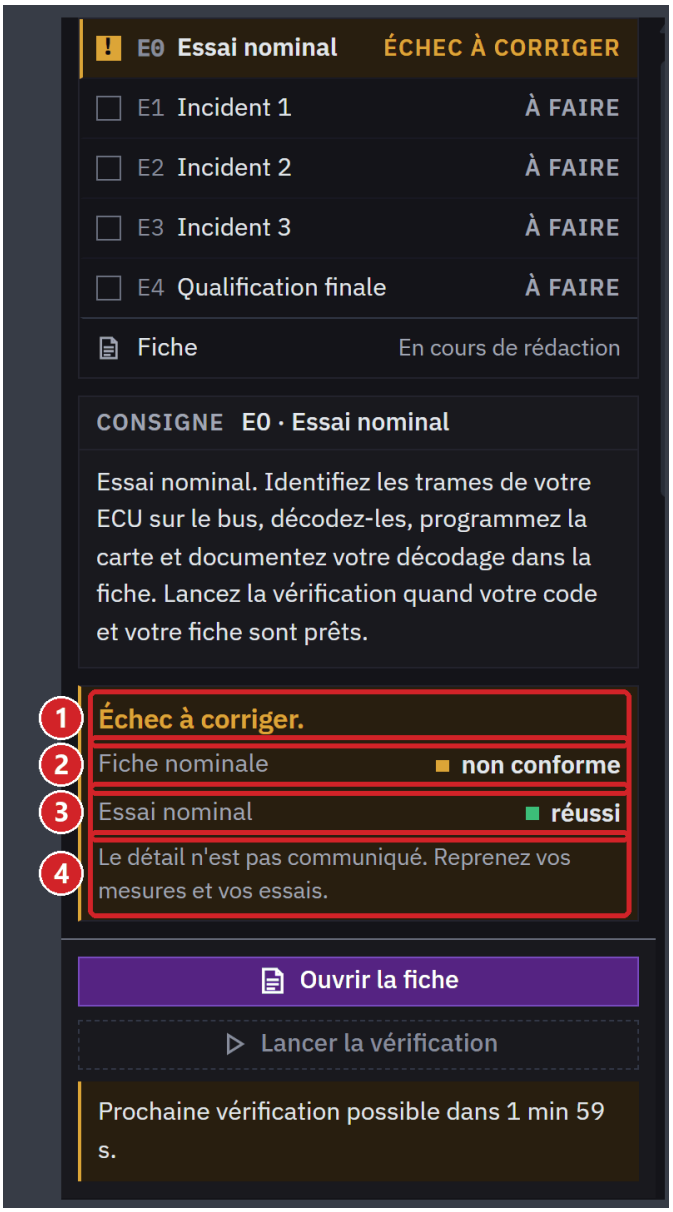


Figure 50. – Le verdict d'un essai nominal en échec.

Étapes	1. Lisez les lignes du verdict dans l'ordre : 1, puis 2 et 3, puis 4.
Ce qu'on doit observer	1 « Échec à corriger. » 2 « Fiche nominale : non conforme ». 3 « Essai nominal : réussi ». 4 « Le détail n'est pas communiqué. Reprenez vos mesures et vos essais. » L'étape passe à « ÉCHEC À CORRIGER » et le délai avant la vérification suivante s'affiche sous le bouton.
Si cela échoue	Ici, la carte a réussi l'essai mais la fiche contient au moins une erreur : c'est la fiche qu'il faut reprendre, champ par champ, depuis vos sources (Que faire quand ça échoue ?).

Chaque ligne dit **où** chercher, jamais **quoi** corriger :

Ligne	Ce qu'elle dit	Ce qu'elle ne dit pas
« Vérification réussie. Étape validée le … »	l'étape est validée, à cette date ; la ligne suivante annonce l'étape qui s'ouvre	—
« Échec à corriger. »	au moins un contrôle de l'étape a échoué	lequel
« Fiche nominale : conforme / non conforme » (E0)	si tous les champs contrôlés de la partie nominale sont justes	quel champ est faux, ni combien
« Essai nominal : réussi / non réussi » (E0)	si votre carte a passé l'essai caché, sur un banc identique au banc de référence	quel cas a échoué

Ligne	Ce qu'elle dit	Ce qu'elle ne dit pas
« Diagnostic (fiche) : conforme / non conforme » (E1 à E3)	si cible, catégorie, état observé et mesure clé sont justes et pertinents	lequel est faux
« Réparation (essai caché) : vérifiée / non vérifiée » (E1 à E3)	si votre circuit est identique au banc de référence, l'anomalie réellement supprimée, et si votre carte passe un essai caché qui rejoue l'anomalie réparée	ce qui manque
« Le détail n'est pas communiqué. Reprenez vos mesures et vos essais. »	un rappel : la réponse ne viendra pas de l'application	—

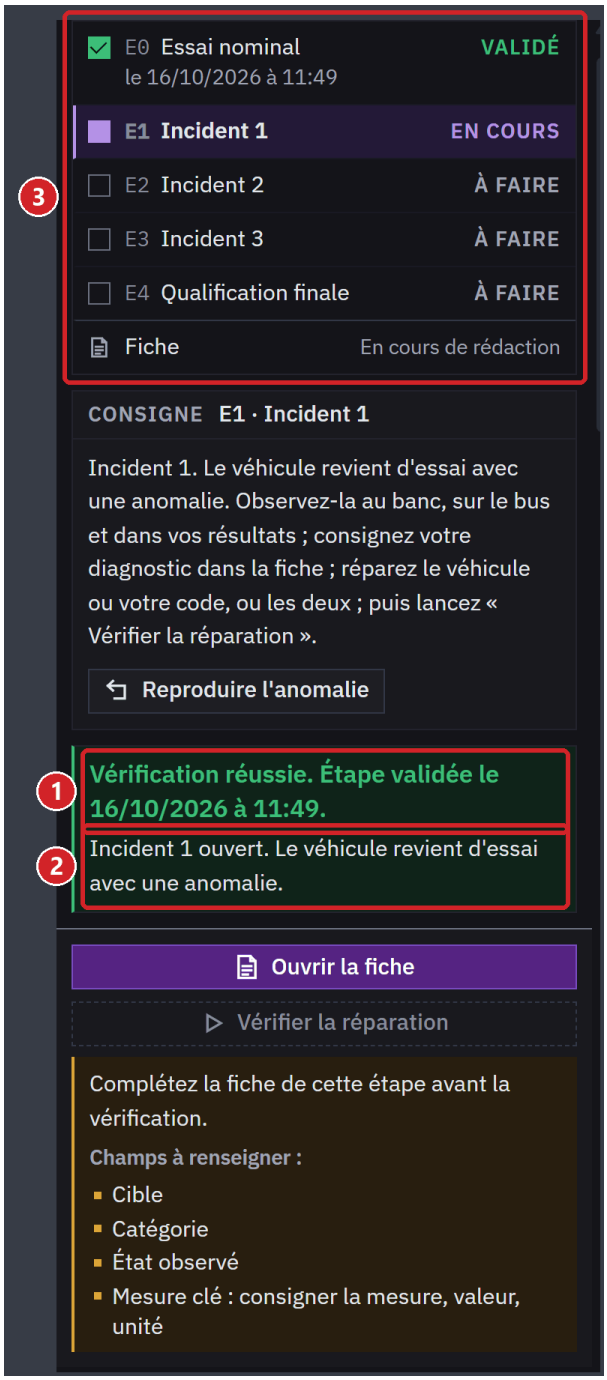


Figure 51. – Le verdict d'un essai nominal réussi.

Étapes	1. Lisez la première ligne 1, puis la suivante 2. 2. Contrôlez la liste des étapes 3.
Ce qu'on doit observer	1 « Vérification réussie. Étape validée le … » avec la date ; 2 « Incident 1 ouvert. Le véhicule revient d'essai avec une anomalie. » 3 E0 passe à « VALIDÉ » avec sa date, E1 à « EN COURS ». Sous le bouton, la liste des champs à renseigner concerne déjà l'étape suivante.

Si cela échoue	Une étape validée ne se rouvre pas : ses champs contrôlés se figent ; la justification, les pages et les noms des signaux restent modifiables.

12. La qualification E4

La qualification finale s'ouvre quand l'incident 3 est validé (« Qualification finale ouverte. »). Sa consigne : « Qualification finale. Une séquence d'essais cachée évalue votre code. Le résultat est donné par compétence. » Le véhicule n'a plus d'anomalie et la fiche n'a pas de partie propre à E4 : seule votre carte est évaluée.

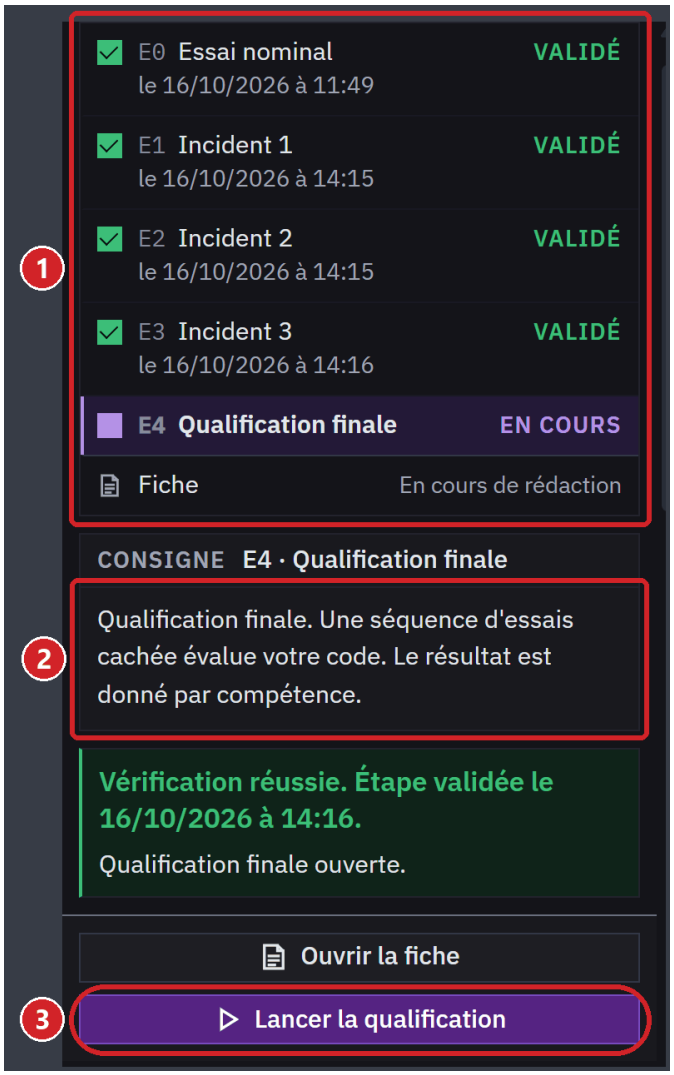


Figure 52. – La qualification finale ouverte.

Étapes	1. Vérifiez que E0 à E3 sont validés 1 et lisez la consigne 2. 2. Programmez la version finale de votre décodeur et contrôlez le CRC (Figure 47). 3. Cliquez « Lancer la qualification » 3.
Ce qu'on doit observer	1 E4 « Qualification finale » est « EN COURS ». 3 le bouton est actif.
Si cela échoue	Le bouton est grisé : lisez le message dessous (carte, délai), comme pour les autres étapes.

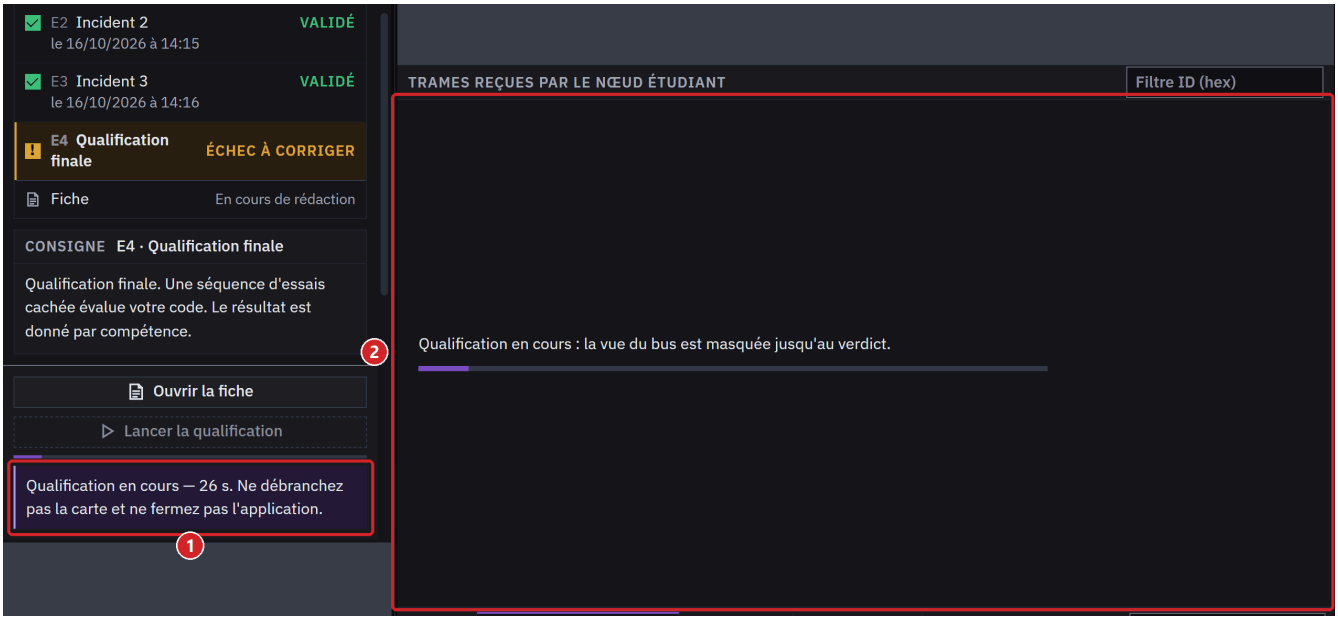


Figure 53. – Une qualification en cours.

Étapes	1. Attendez la fin, carte branchée, application ouverte.
Ce qu'on doit observer	1 le temps restant (« Qualification en cours — … s. »), jusqu'à la fin de la séquence d'essais ; 2 la vue du bus et, à droite, les résultats sont masqués jusqu'au verdict. Ici, E4 affiche encore « ÉCHEC À CORRIGER », d'une qualification précédente.
Si cela échoue	Interruption (carte débranchée, application fermée) : la qualification n'est pas comptée ; relancez-la.

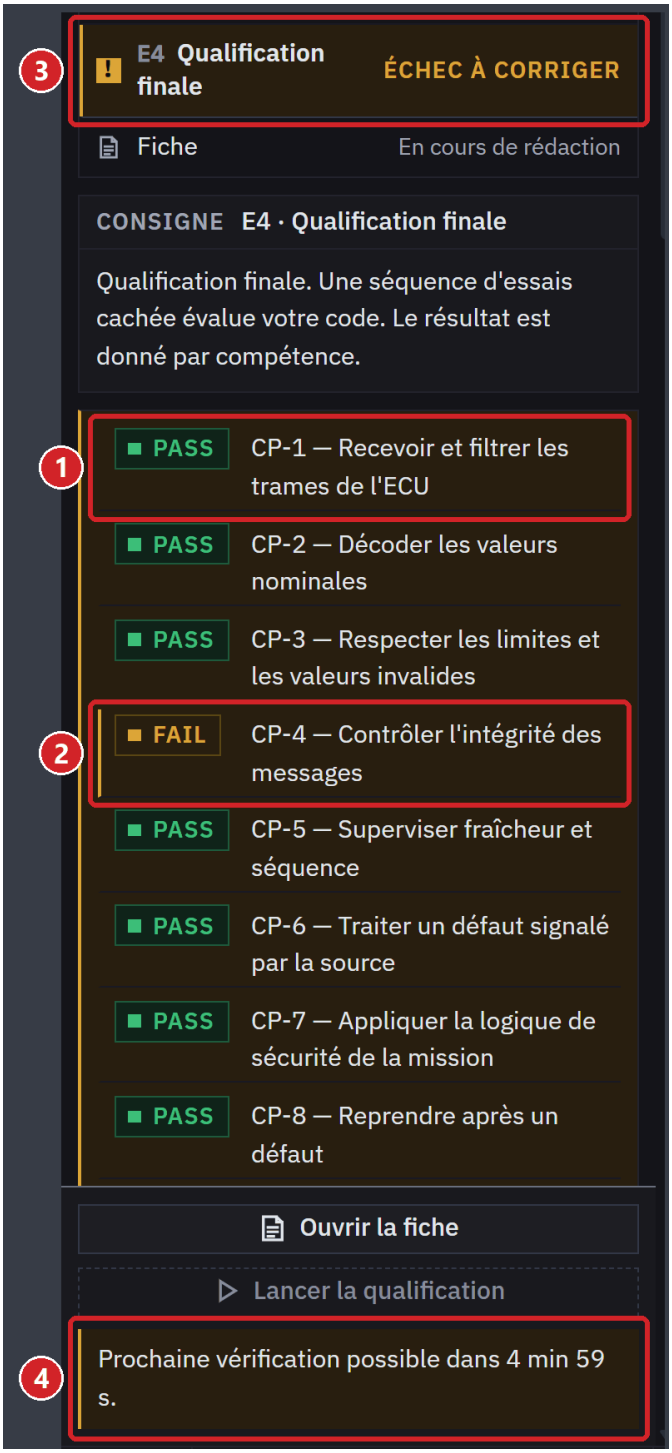


Figure 54. – Le résultat d'une qualification en échec, une ligne par compétence.

Étapes	1. Lisez les huit lignes de compétence, dont la première (1), et repérez celles en FAIL (2). 2. Contrôlez l'état de E4 (3) et le délai affiché sous le bouton (4).
Ce qu'on doit observer	(1) chaque ligne porte un badge PASS ou FAIL, puis « CP-n — compétence » ; aucune note ni détail des essais. (2) une ligne en FAIL suffit : (3) E4 passe à « ÉCHEC À CORRIGER » et (4) la qualification suivante est possible après 5 minutes. Les huit en PASS valident E4 et clôturent la mission : « Mission clôturée. », puis « La fiche finale est exportée dans le dossier Documents. » (Figure 55).
Si cela échoue	Une ligne en FAIL : reprenez la famille de situations que nomme la compétence (ci-dessous), corrigez, flashez, puis relancez la qualification une fois le délai écoulé.

Les huit compétences, toujours dans cet ordre :

Ligne	Compétence
CP-1	Recevoir et filtrer les trames de l'ECU

Ligne	Compétence
CP-2	Décoder les valeurs nominales
CP-3	Respecter les limites et les valeurs invalides
CP-4	Contrôler l'intégrité des messages
CP-5	Superviser fraîcheur et séquence
CP-6	Traiter un défaut signalé par la source
CP-7	Appliquer la logique de sécurité de la mission
CP-8	Reprendre après un défaut

Après un FAIL, relisez l'intitulé de la compétence : il désigne une famille de situations (trames à ignorer, valeurs invalides, protection, fraîcheur et séquence, défaut annoncé par la source, logique de sécurité de l'énoncé, retour à la normale). Reproduisez ces situations sur le banc et sur le bus, observez « Résultats étudiants » et vos lignes de console, corrigez votre code, compilez, flashez, contrôlez le CRC, puis relancez la qualification une fois le délai écoulé. Les huit compétences sont réévaluées à chaque qualification.

13. Clôture E5 et export

Quand la mission est clôturée, la fiche finale est exportée automatiquement et le panneau de la mission passe en « E5 · Clôture ».

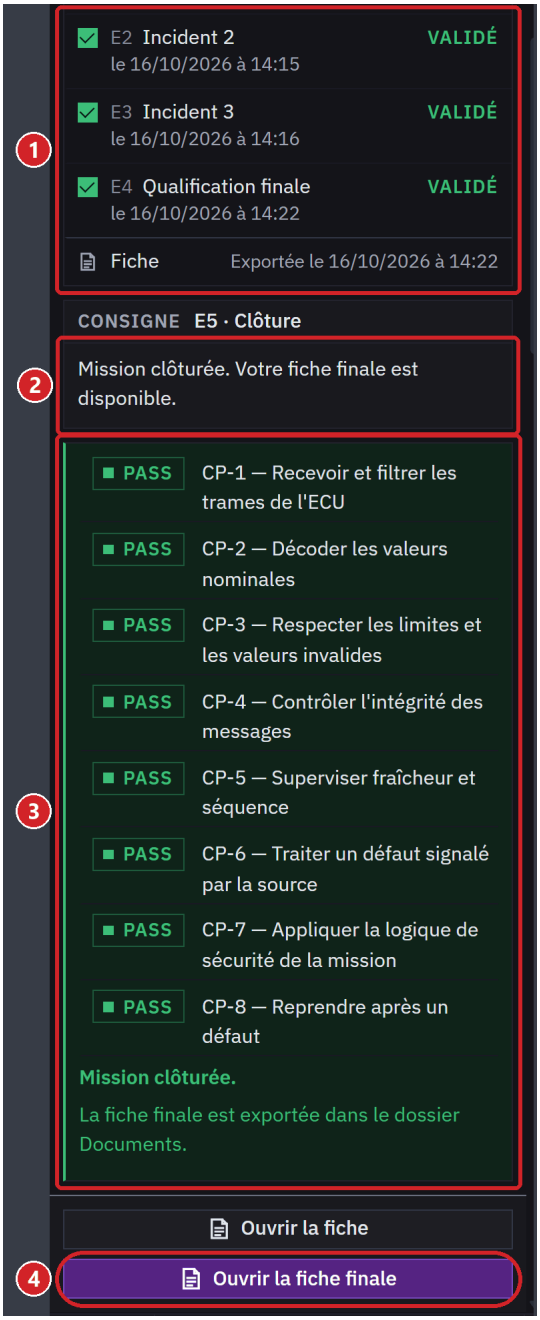


Figure 55. – Une mission clôturée.

Étapes	1. Contrôlez les étapes 1, la consigne 2 et le verdict 3. 2. Cliquez « Ouvrir la fiche finale » 4.
Ce qu'on doit observer	1 les étapes sont « VALIDÉ » (ici E2 à E4 ; E0 et E1, plus haut dans la liste, aussi) et la ligne **Fiche** indique « Exportée le … ». 2 « Mission clôturée. Votre fiche finale est disponible. » 3 les huit compétences, puis « Mission clôturée. » et « La fiche finale est exportée dans le dossier Documents. » 4 ouvre `fiche.html` dans votre navigateur. La barre d'état, en bas de la fenêtre, affiche « E5 · Clôture ».
Si cela échoue	L'export a échoué : le panneau affiche « Exportation de la fiche finale impossible » et le bouton « Réessayer l'export ». Causes signalées : « Dossier Documents introuvable. », « La fiche finale ne peut pas être écrite dans le dossier Documents. » Libérez de la place ou corrigez les droits, puis réessayez ; si cela persiste, prévenez l'encadrant.

L'export crée un dossier par clôture :

Emplacement	Contenu
`Documents\N7RT CAN Lab\fiches\`	un dossier `<mission>-<groupe>-<AAAAMMJJ-HHMM>`, nommé d'après la mission, le groupe et l'instant de la clôture ;
`fiche.html`	la fiche finale lisible, celle qu'ouvre « Ouvrir la fiche finale » ;
`fiche.json`	les mêmes données, lisibles par un programme ; leur empreinte SHA-256 figure au pied de `fiche.html` ;
`scellés\`	les fichiers scellés : la fiche finale et ceux de votre progression.

La fiche finale reprend vos parties de fiche, vos mesures consignées, les réparations enregistrées par l'application (CRC de la carte, modifications du circuit), les compétences, toutes vos tentatives, vos reproductions de l'anomalie et le temps actif.



Figure 56. – La fiche après la clôture, en lecture seule (contenu masqué).

Étapes	1. Pour relire votre fiche dans l'application, cliquez « Ouvrir la fiche » : elle s'ouvre dans l'onglet **Fiche**.
Ce qu'on doit observer	1 la note « La mission est clôturée : la fiche n'est plus modifiable. » Tous les champs sont figés.
Si cela échoue	Vous voulez corriger un champ après la clôture : ce n'est plus possible ; parlez-en à l'encadrant.

À remettre : si votre encadrant le demande, le dossier d'export entier, sans rien y modifier ; l'empreinte SHA-256 et les fichiers scellés permettent de contrôler qu'il est intact.

14. Que faire quand ça échoue ?

L'application ne dit jamais ce qui est faux : un verdict désigne une partie du travail (fiche, code, banc), un message désigne une condition à remplir. Changer des valeurs au hasard coûte une vérification toutes les 2 minutes (5 en E4) et n'apprend rien. Reprenez plutôt la démarche : **observer** (bus, résultats, console, banc), **formuler** une hypothèse, **mesurer ou tester** ce qui la confirme ou l'infirme, **corriger une seule chose**, puis **vérifier**.

Ce que vous voyez	Où chercher	Premières vérifications
Bouton de vérification grisé	message sous le bouton	Lisez-le : champs manquants (enregistrez la fiche), carte, délai, champ mal formé.
« Fiche nominale : non conforme »	fiche	Reprenez chaque champ depuis sa source : bus (identifiant, DLC, période moyennée sur plusieurs trames), documents (positions, ordre, signe, facteur, offset, code invalide, protection), énoncé et règlement (décisions DD-1 à DD-4), valeurs données par l'encadrant au lancement de l'E0 (supervision). Contrôlez la numérotation des bits.
« Essai nominal : non réussi »	code, carte	La carte exécute-t-elle la compilation actuelle (CRC) ? Chaque slot est-il écrit à chaque tick, valeur ou rejet, dans la bonne unité ? Testez vos cas limites sur le banc et lisez vos lignes de console.
« Diagnostic (fiche) : non conforme »	fiche d'incident	Cible (les bonnes broches ou le bon composant), catégorie (relisez sa définition), état observé (les états se lisent dans l'ordre de la liste : retenez le premier dont la définition s'applique), mesure clé consignée pendant l'anomalie, dans une unité acceptée.
« Réparation (essai caché) : non vérifiée »	banc, code	L'anomalie a-t-elle réellement disparu (refaites la mesure) ? Les « Paramètres » et les liaisons sont-ils ceux d'origine ? Votre code traite-t-il la situation réparée ?
Une ligne « CP-n — compétence » en FAIL (E4)	code	Reproduisez la famille de situations que nomme la compétence ; observez vos publications à chaque tick.
Rien dans « Résultats étudiants »	session, code	Session démarrée ? Carte « Prête » avec votre programme ? Publiez-vous dans `student_on_tick` ?
Aucune trame au tableau	session, filtre	Cliquez **Start** ; videz « Filtre ID (hex) ».

Pièges fréquents :

- **Mesure consignée après la réparation** : la mesure ne montre plus l'anomalie. Cliquez « Reproduire l'anomalie », remesurez, consignez, puis réparez de nouveau.
- **Fiche non enregistrée** : « Modifications non enregistrées » est affiché à côté du bouton d'enregistrement ; rien ne s'enregistre tout seul. Cliquez **Enregistrer la fiche** avant de vérifier.
- **Code non flashé** : « Le code sur la carte diffère de la source actuelle. » ; compilez, flashez, contrôlez le CRC.
- **Session arrêtée** : pas de trames, pas de consignation (« Aucune trame récente sur le bus : lancez le véhicule, puis consignez la mesure. »). Cliquez **Start** et attendez quelques secondes.
- **Carte dans un mauvais état** : « Pas de code étudiant », « Code étudiant désactivé », « Liaison perdue » ; voyez la FAQ de démarrage.
- **Banc remplacé** : **Nouveau**, **Ouvrir** ou **Charger** ont remplacé le banc de la mission ; fermez puis rouvrez la mission avec le code du groupe.
- **Paramètre modifié par erreur** : le banc ne correspond plus au banc de référence ; rétablissez la valeur d'origine (Ctrl+Z, ou la valeur que vous aviez notée).
- **Publications dans `student_on_frame` seulement** : elles s'affichent avec la source « trame … » mais ne comptent pas.

Le délai entre deux vérifications est de 2 minutes pour E0 à E3 et de 5 minutes pour E4, compté depuis le dernier verdict de l'étape. Mettez-le à profit : relisez vos sources, mesurez, testez votre code sur le banc.

15. FAQ de démarrage

L'état de la carte reste « Déconnecté ». Branchez la NUCLEO par son port ST-LINK, choisissez son port dans **CARTE**, puis cliquez **Connecter**. Le bouton « Rechercher les cartes », à droite de la liste, relit les ports disponibles.

L'état affiche « Carte absente ». La carte n'est pas détectée (« Carte … non détectée : vérifiez le câble USB (connecteur ST-LINK CN1). »). Vérifiez le câble et le connecteur, essayez un autre port USB, puis recherchez les cartes.

L'état affiche « Port occupé ». Un autre programme utilise le port série : console série de STM32CubeIDE, autre instance du CAN Lab, moniteur série. Fermez-le : la connexion reprend seule.

L'état reste sur « Poignée de main ». L'application attend la réponse du runtime de la carte. Patientez quelques secondes ; si rien ne change, débranchez et rebranchez la carte.

L'état affiche « Pas de code étudiant ». La carte n'a pas de programme étudiant valide (« Runtime vérifié, mais … : compilez et flashez votre projet depuis STM32CubeIDE. ») et **Start** reste grisé. Programmez-la depuis la vue **Code STM32** (**Compiler + flasher**) : STM32CubeIDE n'est pas nécessaire.

L'état affiche « Code étudiant désactivé ». Votre code a provoqué une faute ou bloqué la carte (« Code étudiant désactivé après une faute : corrigez-le, ou démarrez une session pour le réactiver. »). Cherchez un accès hors tableau ou une boucle sans fin, corrigez, compilez et flashez ; **Start** le réactive aussi.

« Carte muette » ou « Code étudiant occupé » s'affiche en haut. « Carte muette » : la carte ne répond plus (« Carte muette depuis 750 ms (cœur arrêté par un point d'arrêt ?) : envois suspendus. »). « Code étudiant occupé » : un appel de votre code ne rend pas la main (« Le code étudiant s'exécute depuis … ms (boucle infinie ?). »). Cherchez une boucle trop longue dans votre code.

L'état affiche « Carte incompatible ». Le runtime de la carte ne correspond pas au SDK de l'application. Seul un encadrant peut le réinstaller : prévenez-le.

L'état affiche « Liaison perdue ». Le câble a été débranché ou la carte a redémarré (« Liaison perdue sur … (carte débranchée ?) : reconnexion automatique dès son retour… »). Rebranchez : la reconnexion est automatique.

La compilation est impossible. Le message *Outil de compilation indisponible : installez les prérequis du projet étudiant.*, ou une console qui signale `arm-none-eabi-gcc not found`, indique un outil manquant :

1. installez Python 3.11 ou plus, la chaîne GNU Arm et STM32CubeProgrammer ; STM32CubeIDE ou STM32CubeCLT, installés dans leur dossier par défaut `C:\ST`, conviennent aussi ;
2. ouvrez PowerShell dans le dossier décompressé et lancez :
   `python -m pip install -r .\requirements-student-ide.txt`
3. relancez l'application.

Windows bloque le lancement. L'exécutable n'est pas signé : SmartScreen affiche « Windows a protégé votre ordinateur ». Cliquez « Informations complémentaires », puis « Exécuter quand même ».

La fenêtre reste blanche. Le composant WebView2 manque : installez « Microsoft Edge WebView2 Runtime », puis relancez.

Le pack est refusé. « Ce fichier n'est pas un pack de groupe N7RT. » : vous avez choisi un autre fichier ; prenez le fichier `.n7rtpack` de votre groupe. « Ce pack est destiné à une autre mission. » : le pack ne correspond pas à la mission choisie. « Pack du groupe illisible : demandez un nouveau fichier à votre encadrant. » : le fichier est abîmé.

« Code de groupe incorrect. » Ressaisissez le code reçu de l'encadrant, sans espace en trop. Le code n'est écrit nulle part dans le pack : demandez-le à l'encadrant si vous l'avez perdu.

« La mission de cette session est déjà choisie : ... » Le choix de la mission est définitif pour la session (« Ce choix est définitif pour la session : pour en changer, demandez à votre encadrant. »). Demandez à l'encadrant.

La fermeture demande « Fiche non enregistrée ». La fiche contient des modifications non enregistrées : « Enregistrer et fermer » les garde, « Fermer sans enregistrer » les abandonne, « Revenir à la mission » annule la fermeture. Pour du code non enregistré, la même question propose « Revenir à l'application ».

Le bouton de vérification reste grisé. Lisez le message dessous (Vérifier une étape) : champs manquants dans la fiche enregistrée, champ mal formé, carte non connectée ou sans programme, délai entre deux vérifications.

Le bouton Flasher reste grisé. Survolez-le : il faut une compilation de la source actuelle, une carte connectée dont le runtime est compatible et vérifié (état « Prête », « Pas de code étudiant » ou « Code étudiant désactivé », sonde ST-LINK identifiée), et aucune session ni vérification en cours (Figure 43).

Le panneau de code affiche « Le code sur la carte diffère de la source actuelle. » La carte exécute une autre version que votre source : compilez et flashez.

« Progression illisible sur ce poste : demandez à l'encadrant de réinitialiser la mission. » Prévenez l'encadrant.

Où sont mes fichiers ? Le code est enregistré dans le profil local de l'application, sur ce poste ; la fiche et la progression aussi. La fiche finale est exportée dans `Documents\N7RT CAN Lab\fiches\` à la clôture.